UNIVERSIDAD TECNOLÓGICA
DE XICOTEPEC DE JUÁREZ

INNOVACIÓN • DESARROLLO • FUTURO

Tecnología que transforma

MANUAL DE INSTALACIÓN

HERRAMIENTAS Y ENTORNOS DE DESARROLLO

INTEGRANTES:
Yazmin Gutierrez Hernandez
Obed Guzman Flores
Citlalli Perez Dionicio
Michelle Castro Otero
Jennifer Bautista Barrios

UNIVERSIDAD:
Universidad Tecnológica
de Xicotepec de Juárez

PROFESOR:
MTI. Marco Antonio
Ramirez Hernandez



ÍNDICE GENERAL

INTRODUCCIÓN 6
OBJETIVO DEL MANUAL 6
Objetivo general 6
Objetivos específicos 6
FLUJO Y ARQUITECTURA DEL ENTORNO 7
REQUISITOS DE HARDWARE 7
SOFTWARE REQUERIDO 8
PREPARACIÓN DEL SISTEMA OPERATIVO 8
Comprobar la versión de Windows 8
Comprobar la información del sistema 9
Comprobar la virtualización 10
Comprobación final 11
INSTALACIÓN Y CONFIGURACIÓN DE WSL2 11
Abrir PowerShell como administrador 12
Instalar WSL 12
Consultar las distribuciones disponibles 13
Instalar Ubuntu 24.04 14
Establecer WSL2 como versión predeterminada 15
Actualizar WSL 16
Comprobar el estado de WSL 17
Comprobar que Ubuntu utiliza WSL2 18
Comprobación final 18
INSTALACIÓN Y CONFIGURACIÓN DE VISUAL STUDIO CODE 19
Descargar Visual Studio Code 19
Instalar Visual Studio Code 20
Abrir Visual Studio Code 22
Instalar extensiones del entorno 23
INSTALACIÓN DEL JDK 21 26
Descarga del JDK 21 26
Instalación del JDK 27
Verificación de Java 28
INSTALACIÓN DE NODE.JS Y NPM 29
Evidencia E08 – Node.js y npm disponibles 30
INSTALACIÓN Y CONFIGURACIÓN DE ANDROID STUDIO 30
Descarga de Android Studio 30
Instalación 31
Configuración inicial 33
Verificación 35
Seguridad 35
CREACIÓN DE UN DISPOSITIVO VIRTUAL ANDROID 36
Abrir Device Manager 37

Crear un dispositivo virtual 37
Seleccionar la versión de Android 38
Configurar el AVD 39
Iniciar el emulador 40
EMULADOR FUNCIONANDO 41
INSTALACIÓN DE FLUTTER Y DART 42
Descargar Flutter 42
Extraer el Flutter SDK 44
Agregar Flutter al PATH de Windows 47
Comprobar que el PATH se actualizó 51
Verificar la versión de Flutter 52
Comprobar Dart 53
Comprobar Flutter con Flutter Doctor 54
Flutter instalado 54
Comprobación final 55
DIAGNÓSTICO DEL ENTORNO FLUTTER 56
Comprobaciones complementarias 56
VALIDACIÓN DE LA APLICACIÓN FLUTTER EXISTENTE 56
Abrir y revisar el proyecto existente 56
Resolver dependencias 57
Detectar el dispositivo Android 58
Ejecutar la aplicación real 59
Evidencia E15 – Aplicación Flutter existente funcionando 60
INSTALACIÓN Y CONFIGURACIÓN DE DOCKER DESKTOP 61
Descargar e instalar Docker Desktop 61
Verificar Docker y Docker Compose 63
Resultado esperado 63
Docker Desktop funcionando 64
DOCKERIZACIÓN DEL PROYECTO FLUTTER EXISTENTE 66
Revisar el proyecto Flutter 66
Verificar Docker 67
Crear el Dockerfile 68
Crear .dockerignore 70
Crear compose.yaml si aplica 71
Construir la imagen Docker 72
Ejecutar y verificar el contenedor 73
Detener los servicios 75
Comprobación Flutter + Docker 76
IMAGEN DOCKER 77
CONTENEDOR DOCKER 77
PUBLICACIÓN DEL PROYECTO EN GITHUB 78
Verificar Git y el repositorio local 78
Revisar .gitignore 79

Preparar los cambios 81
Crear el commit 81
Crear o conectar el repositorio de GitHub 82
Subir la rama actual 83
Comprobar el proyecto en GitHub 83
README del repositorio 84
PRUEBA FINAL Y PROBLEMAS FRECUENTES 84
Secuencia de validación final 84
Problemas frecuentes 85
Reglas permanentes para evidencias 86
CHECKLIST FINAL 86
REFERENCIAS OFICIALES 87
Nota final 88
EVIDENCIA FINAL DEL ENTORNO 88

ÍNDICE DE IMÁGENES

Figura 1. Verificación de la versión y compilación de Windows. 11
Figura 2. Información técnica del sistema mediante systeminfo. 12
Figura 3. Virtualización habilitada en el Administrador de tareas. 13
Figura 4. Apertura de PowerShell para configurar WSL2. 14
Figura 5. Instalación inicial de WSL desde PowerShell. 15
Figura 6. Consulta de distribuciones disponibles para WSL. 16
Figura 7. Instalación de Ubuntu 24.04 en WSL. 17
Figura 8. Configuración de WSL2 como versión predeterminada. 18
Figura 9. Actualización de WSL desde PowerShell. 19
Figura 10. Verificación del estado de WSL. 19
Figura 11. Ubuntu configurado para utilizar WSL2. 20
Figura 12. Descarga de Visual Studio Code desde el sitio oficial. 22
Figura 13. Aceptación de los términos de licencia de Visual Studio Code. 22
Figura 14. Selección de la carpeta de instalación de Visual Studio Code. 23
Figura 15. Proceso de instalación de Visual Studio Code en Windows. 24
Figura 16. Visual Studio Code abierto correctamente después de la instalación. 25
Figura 17. Extensión Flutter disponible en Visual Studio Code. 26
Figura 18. Extensión Dart disponible en Visual Studio Code. 26
Figura 19. Extensión WSL disponible en Visual Studio Code. 27
Figura 20. Extension Pack for Java disponible en Visual Studio Code. 27
Figura 21. Sitio oficial de Eclipse Temurin para descargar JDK 21. 28
Figura 22. Selección de JDK 21 LTS para Windows x64. 29
Figura 23. Inicio del asistente de instalación de Eclipse Temurin JDK 21. 29
Figura 24. Opciones de instalación para PATH y JAVA_HOME del JDK 21. 30
Figura 25. Proceso de instalación del JDK 21 en Windows. 30
Figura 26. JDK 21 instalado y reconocido desde la terminal de Windows. 31
Figura 27. Sitio oficial de descarga de Android Studio. 33
Figura 28. Opciones de descarga de Android Studio para Windows. 33
Figura 29. Selección de componentes durante la instalación de Android Studio. 34
Figura 30. Proceso de instalación de Android Studio. 35
Figura 31. Asistente de configuración inicial de Android Studio. 36
Figura 32. Descarga de componentes del Android SDK durante la configuración inicial. 37
Figura 33. Android Studio instalado y abierto correctamente. 38
Figura 34. Acceso al Device Manager de Android Studio. 39
Figura 35. Selección del perfil de dispositivo virtual Android. 40
Figura 36. Selección de la imagen del sistema Android para el AVD. 41
Figura 37. Configuración del dispositivo virtual Android. 42
Figura 38. Dispositivo virtual configurado en Device Manager. 43
Figura 39. Emulador Android iniciado y funcionando correctamente. 44
Figura 40. Documentación oficial de Flutter para iniciar la instalación. 45
Figura 41. Selección de Windows como plataforma de instalación de Flutter. 46

Figura 42. Descarga del SDK de Flutter para Windows. 46
Figura 43. Archivo del SDK de Flutter descargado en Windows. 47
Figura 44. Carpeta del SDK de Flutter extraída en el equipo. 48
Figura 45. Verificación de la estructura del SDK de Flutter. 49
Figura 46. Acceso a la configuración de variables de entorno de Windows. 50
Figura 47. Edición de las variables de entorno del sistema. 51
Figura 48. Configuración del PATH para Flutter. 52
Figura 49. Ruta de Flutter agregada al PATH de Windows. 52
Figura 50. Comprobación de la configuración de Flutter desde la consola. 54
Figura 51. Verificación de la versión de Flutter desde la consola. 55
Figura 52. Verificación de Dart desde la consola. 55
Figura 53. Diagnóstico del entorno mediante Flutter Doctor. 56
Figura 54. Evidencia de Flutter instalado y reconocido en la terminal. 57
Figura 55. Revisión de licencias Android y dispositivos disponibles en Flutter. 58
Figura 56. Revisión de la estructura del proyecto Flutter en Visual Studio Code. 59
Figura 57. Resolución de dependencias del proyecto con flutter pub get. 60
Figura 58. Detección del dispositivo Android desde Flutter. 61
Figura 59. Ejecución de la aplicación Flutter en el dispositivo Android. 62
Figura 60. Aplicación Flutter existente funcionando en Android. 63
Figura 61. Descarga de Docker Desktop para Windows desde el sitio oficial. 64
Figura 62. Configuración de instalación de Docker Desktop en Windows. 65
Figura 63. Verificación de Docker Engine y Docker Compose desde la terminal. 66
Figura 64. Docker Desktop iniciando Docker Engine. 67
Figura 65. Docker Desktop abierto con el motor disponible. 67
Figura 66. Revisión del proyecto Flutter existente antes de la dockerización. 69
Figura 67. Verificación de Docker y Docker Compose antes de construir la imagen. 70
Figura 68. Dockerfile creado para el proyecto Flutter. 71
Figura 69. Archivo .dockerignore configurado para el proyecto Flutter. 72
Figura 70. Archivo compose.yaml configurado para el proyecto Flutter. 73
Figura 71. Construcción de la imagen Docker del proyecto Flutter. 74
Figura 72. Imagen Docker del proyecto disponible en Docker Desktop. 75
Figura 73. Ejecución del contenedor y revisión de los logs de Docker. 76
Figura 74. Resultado del contenedor ejecutando la tarea definida en el Dockerfile. 77
Figura 75. Detención de los servicios Docker sin afectar el proyecto Flutter. 78
Figura 76. Evidencia de la imagen Docker construida para el proyecto. 79
Figura 77. Evidencia del contenedor Docker ejecutado. 80
Figura 78. Verificación del repositorio Git local antes de la publicación. 81
Figura 79. Revisión del archivo .gitignore del proyecto Flutter. 82
Figura 80. Preparación de cambios con git status y git add. 83
Figura 81. Creación del commit del proyecto mediante Git. 84
Figura 82. Configuración del repositorio remoto de GitHub. 84
Figura 83. Publicación de la rama actual en GitHub mediante git push. 85
Figura 84. Verificación del proyecto publicado en GitHub. 86

INTRODUCCIÓN

Este manual documenta la preparación de un entorno de desarrollo enfocado en una aplicación Flutter existente. El flujo de trabajo se concentra en Windows, WSL2 cuando sea necesario para Docker, Visual Studio Code como editor principal para revisar, modificar y ejecutar el proyecto Flutter, JDK 21 LTS y Maven 3.9.x como herramientas Java del entorno, Node.js y npm, Android Studio para administrar Android SDK y el emulador, Flutter, Docker, Git y GitHub.

La aplicación Flutter ya fue desarrollada previamente. Por ello, el manual no parte de crear una aplicación de ejemplo: primero valida el proyecto real, después documenta la incorporación de Docker y finalmente prepara y publica el repositorio en GitHub.

OBJETIVO DEL MANUAL

Objetivo general

Instalar, configurar y validar un entorno de desarrollo para ejecutar una aplicación móvil existente con Flutter en Android, utilizando Visual Studio Code como entorno principal de edición y ejecución, Android Studio para administrar el SDK y el emulador, JDK 21 LTS para la cadena de compilación Java/Android, Maven 3.9.x como herramienta Java adicional, Node.js y npm, Docker de acuerdo con las necesidades reales del proyecto y Git/GitHub para control de versiones y publicación.

Objetivos específicos

Comprobar que Windows, la arquitectura del sistema y la virtualización cumplen los requisitos del entorno.

Configurar WSL2 únicamente en la medida necesaria para Docker Desktop.

Instalar Visual Studio Code, configurar Flutter y Dart, y utilizar su terminal integrada para ejecutar el proyecto Flutter cuando corresponda.

Instalar JDK 21 LTS (Eclipse Temurin), configurar PATH y JAVA_HOME, y verificar Java desde la terminal.

Instalar Apache Maven 3.9.x, configurar MAVEN_HOME y PATH, y verificar su integración con Java 21.

Instalar y verificar Node.js LTS y npm.

Instalar Android Studio y configurar Android SDK, SDK Tools y un dispositivo virtual Android.

Instalar Flutter y Dart, configurar el PATH y revisar el diagnóstico con flutter doctor.

Validar la aplicación Flutter existente con flutter pub get, flutter devices y flutter run.

Instalar Docker Desktop y verificar Docker Engine y Docker Compose.

Documentar paso a paso la preparación del proyecto para Docker sin inventar servicios que no existan.

Preparar Git, crear el commit correspondiente y publicar el proyecto en GitHub.

Mantener evidencias reales, numeradas y sin información sensible.

FLUJO Y ARQUITECTURA DEL ENTORNO

El entorno se organiza en varios componentes complementarios. Visual Studio Code se utiliza para editar el código Flutter y puede ejecutar comandos como flutter pub get, flutter devices y flutter run; Android Studio administra Android SDK, las herramientas de compilación y el emulador; Flutter y Dart construyen la aplicación móvil; JDK 21 proporciona el entorno Java que utiliza la cadena de compilación Android mediante Gradle; Maven se conserva como herramienta Java adicional; Node.js y npm permanecen disponibles; y Docker se utiliza solamente para los componentes o tareas que realmente se decidan contenerizar.

WINDOWS
- ├─ Visual Studio Code → Flutter + Dart → edición y ejecución del proyecto
- ├─ Android Studio → Android SDK → Android Emulator
- ├─ JDK 21 LTS → Gradle / cadena de compilación Android
- ├─ Maven 3.9.x → herramienta Java adicional
- ├─ Node.js → npm
- └─ WSL2 → Docker Desktop → Docker Compose → Contenedor(es) del proyecto

Proyecto Flutter existente → Validación local → Dockerización real → Git → GitHub

No debe afirmarse que el emulador Android o el APK se ejecutan dentro de Docker si eso no ocurre en el proyecto real. La aplicación móvil seguirá ejecutándose mediante Android Studio y el emulador o dispositivo Android.

REQUISITOS DE HARDWARE

Nivel	Requisitos
Mínimos	CPU x64; 8 GB de RAM; 30 GB libres; virtualización habilitada; conexión a Internet.
Recomendados	Procesador Intel Core i5 / AMD Ryzen 5 o superior; 16 GB de RAM o más; SSD; 50 GB o más libres; virtualización habilitada.

Android Studio, el emulador y Docker pueden consumir recursos de manera simultánea. Cuando el equipo tenga memoria limitada, conviene cerrar servicios que no se utilicen y evitar ejecutar varios emuladores o contenedores innecesarios.

SOFTWARE REQUERIDO

Software	Uso dentro del manual
Windows 10/11	Sistema anfitrión.
WSL2 + Ubuntu 24.04 LTS	Soporte Linux y backend de Docker cuando sea necesario.

Visual Studio Code	Editor principal para modificar el código Flutter y ejecutar comandos del proyecto desde la terminal integrada.
JDK 21 LTS (Eclipse Temurin)	Entorno Java requerido por la cadena de compilación Android/Gradle y disponible para tareas Java.
Apache Maven 3.9.x	Herramienta adicional para gestión y automatización de proyectos Java; no sustituye a Gradle en Android.
Node.js LTS + npm	Herramientas de ejecución y gestión de paquetes disponibles en el entorno.
Android Studio	Administración del Android SDK, SDK Tools, Gradle y dispositivos virtuales Android (AVD).
Android SDK / SDK Tools / Emulator	Compilación y ejecución de la aplicación Android.
Flutter Stable + Dart	Framework y lenguaje del proyecto móvil.
Docker Desktop + Docker Compose	Construcción y ejecución de contenedores del proyecto.
Git + GitHub	Control de versiones y publicación del proyecto.

PREPARACIÓN DEL SISTEMA OPERATIVO

Antes de instalar las herramientas del entorno de desarrollo, se debe comprobar que Windows esté preparado y que el equipo sea compatible con WSL2. Se verificará la versión de Windows, la arquitectura del sistema y la virtualización.

Comprobar la versión de Windows

Paso 1. Presiona **Windows + R** para abrir la ventana Ejecutar.

Paso 2. Escribe `winver` y presiona **Enter**.

Paso 3. Se abrirá la ventana **Acerca de Windows**. Verifica la edición, versión y compilación.

Resultado esperado: Se muestra la ventana **Acerca de Windows** con la información de la versión instalada.

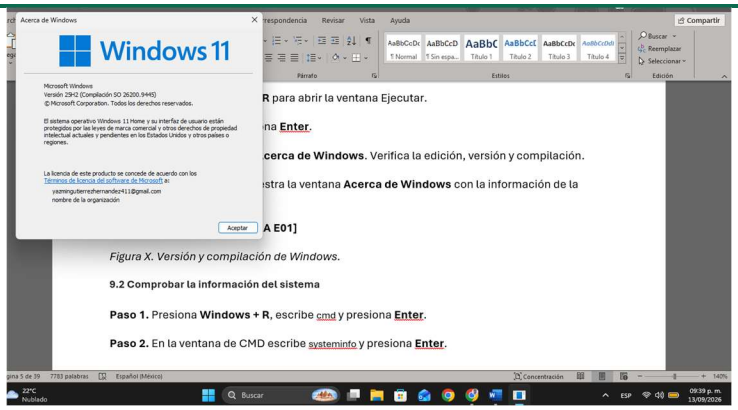


Figura 1. Verificación de la versión y compilación de Windows.

Fuente: Elaboración propia.

Comprobar la información del sistema

Paso 1. Presiona **Windows + R**, escribe `cmd` y presiona **Enter**.

Paso 2. En la ventana de CMD escribe `systeminfo` y presiona **Enter**.

Paso 3. Espera a que aparezca la información del sistema. Localiza datos como el sistema operativo, versión, tipo de sistema y arquitectura.

Paso 4. Verifica que el equipo utilice una arquitectura compatible con **x64**.

Paso 5. Toma una captura de la consola y guárdala como E02_SystemInfo.png.

Resultado esperado: Se muestra el resumen técnico del sistema.

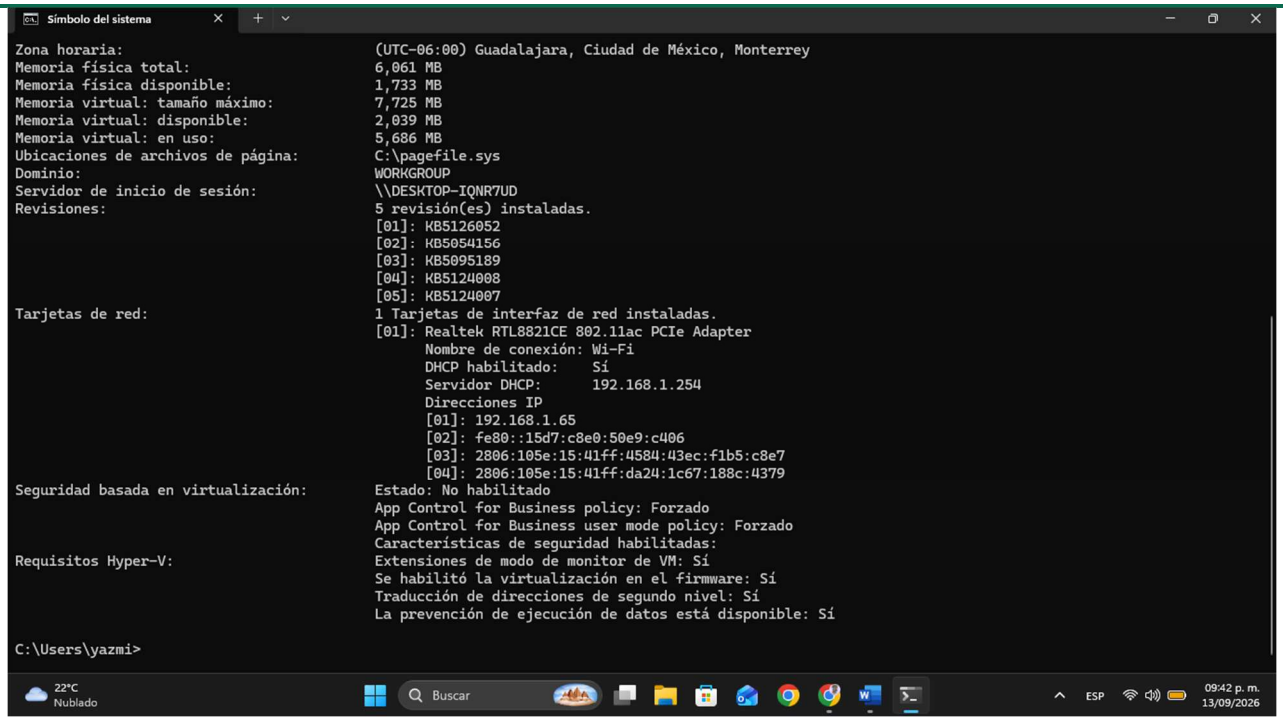


Figura 2. Información técnica del sistema mediante systeminfo.

Fuente: Elaboración propia.

Comprobar la virtualización

Paso 1. Presiona **Ctrl + Shift + Esc** para abrir el Administrador de tareas.

Paso 2. Selecciona **Rendimiento** y después **CPU**.

Paso 3. Localiza el campo **Virtualización**.

Paso 4. Verifica que aparezca como **Habilitada** o **Enabled**.

Paso 5. Toma una captura donde se observe el estado de la virtualización.

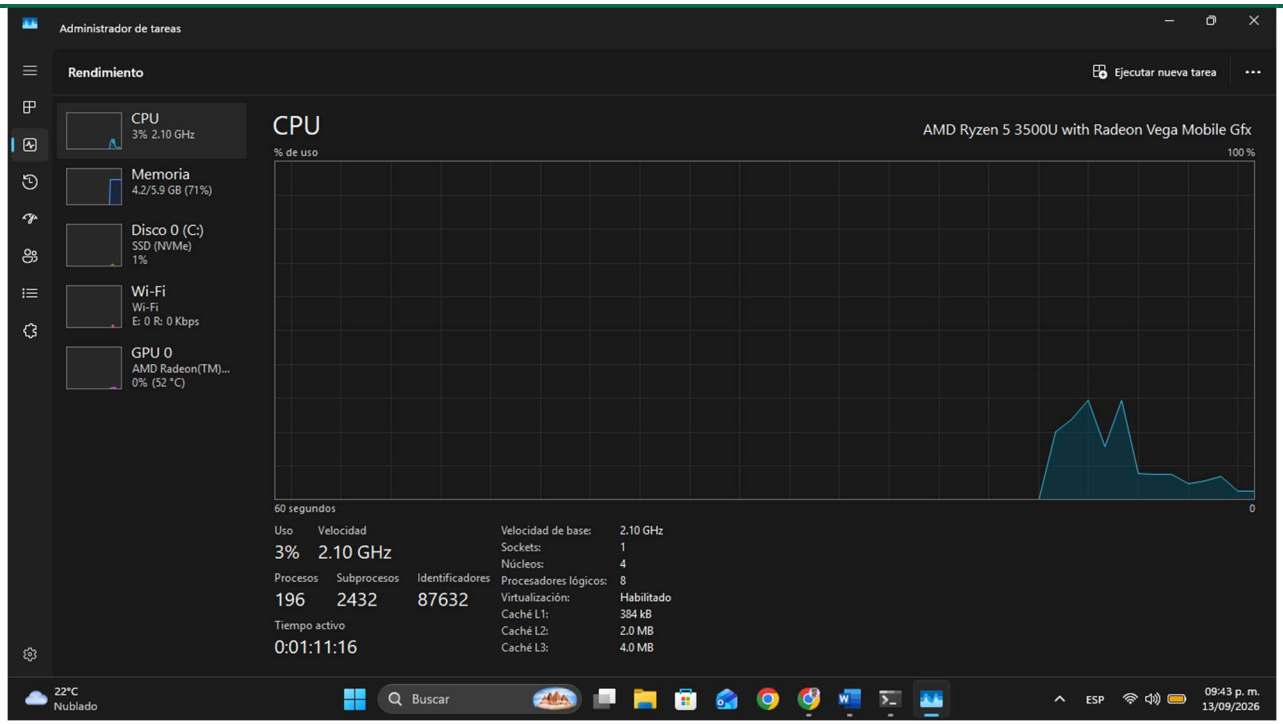


Figura 3. Virtualización habilitada en el Administrador de tareas.

Fuente: Elaboración propia.

Resultado esperado: La virtualización aparece como **Habilitada**.

En algunos equipos esta función puede aparecer en BIOS/UEFI como **Intel VT-x**, **AMD-V** o **SVM**. Si está deshabilitada, deberá activarse antes de continuar.

Comprobación final

Antes de continuar con el siguiente punto, verifica que la versión de Windows esté identificada, `systeminfo` se haya ejecutado correctamente, la arquitectura sea compatible con x64 y la virtualización esté habilitada.

La preparación del sistema permitió comprobar que Windows cuenta con las condiciones necesarias para continuar con la configuración del entorno de desarrollo. Se verificó la versión del sistema operativo, la arquitectura x64 y el estado de la virtualización, dejando el equipo listo para los siguientes pasos de instalación.

INSTALACIÓN Y CONFIGURACIÓN DE WSL2

WSL2 permite utilizar un entorno Linux integrado con Windows. En este manual se mantiene principalmente para soportar Docker Desktop cuando el equipo utilice el backend basado en WSL2. Se instalará y verificará Ubuntu 24.04 LTS como distribución de trabajo.

Abrir PowerShell como administrador

Paso 1. Abre el menú **Inicio**.

Paso 2. Escribe:

```
PowerShell
```

Paso 3. Haz clic derecho en **Windows PowerShell** y selecciona **Ejecutar como administrador**.

Paso 4. Si aparece una ventana de confirmación, selecciona **Sí**.

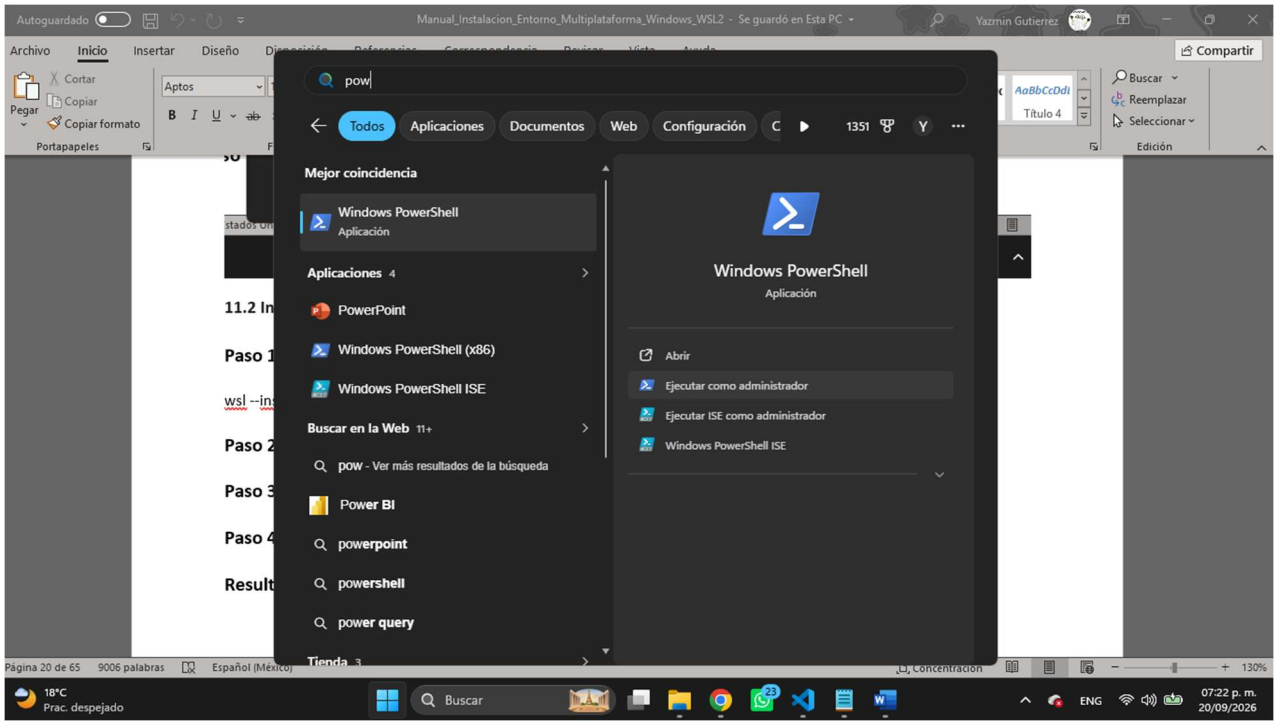


Figura 4. Apertura de PowerShell para configurar WSL2.

Fuente: Elaboración propia.

Instalar WSL

Paso 1. En PowerShell ejecuta:

```
wsl --install
```

Paso 2. Presiona **Enter**.

Paso 3. Espera a que termine la instalación.

Paso 4. Si Windows solicita reiniciar el equipo, reinícialo.

Resultado esperado: Se instala la configuración inicial de WSL.

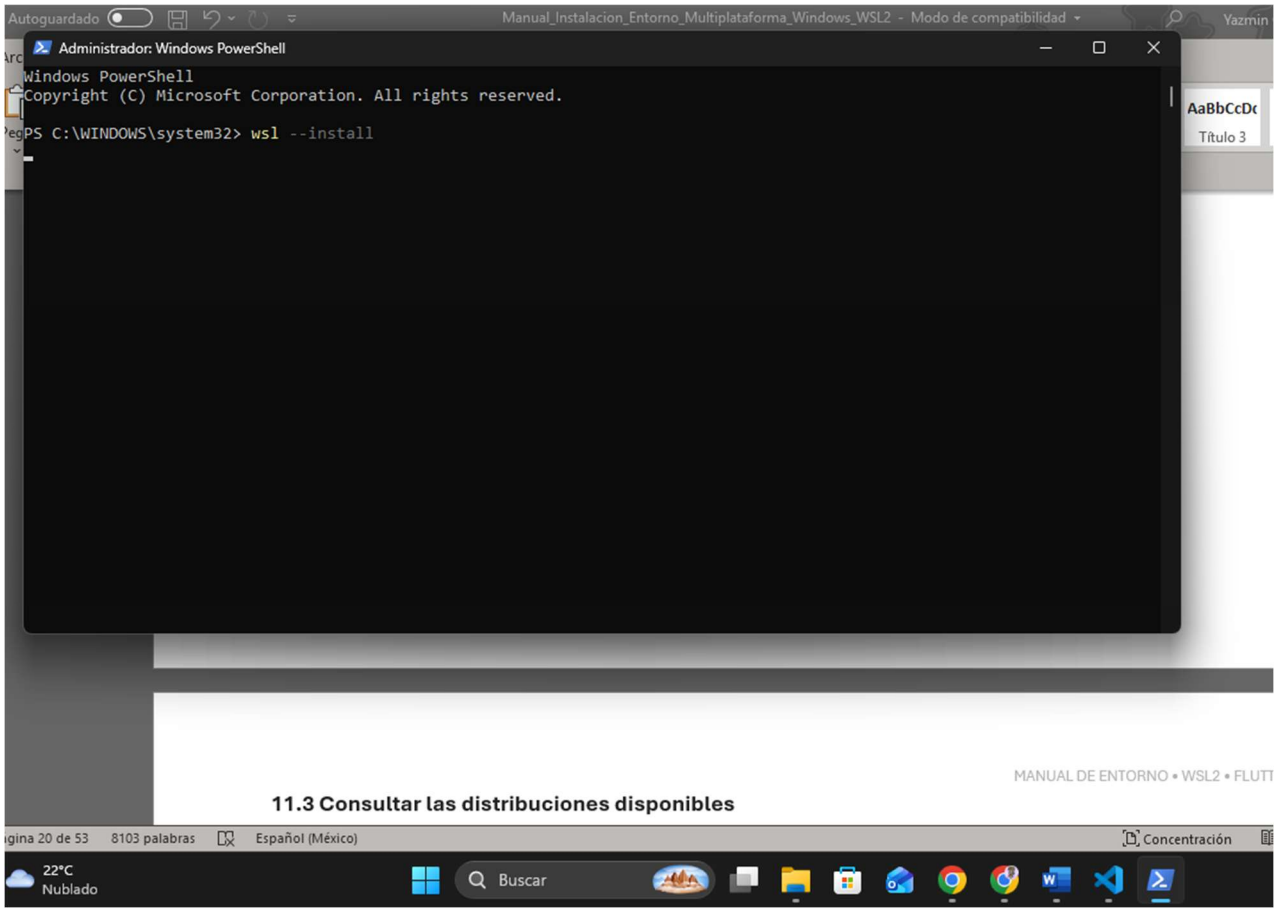


Figura 5. Instalación inicial de WSL desde PowerShell.
Fuente: Elaboración propia.

Consultar las distribuciones disponibles

Paso 1. Abre nuevamente **PowerShell**.

Paso 2. Ejecuta:

```
wsl --list --online
```

Paso 3. Presiona **Enter**.

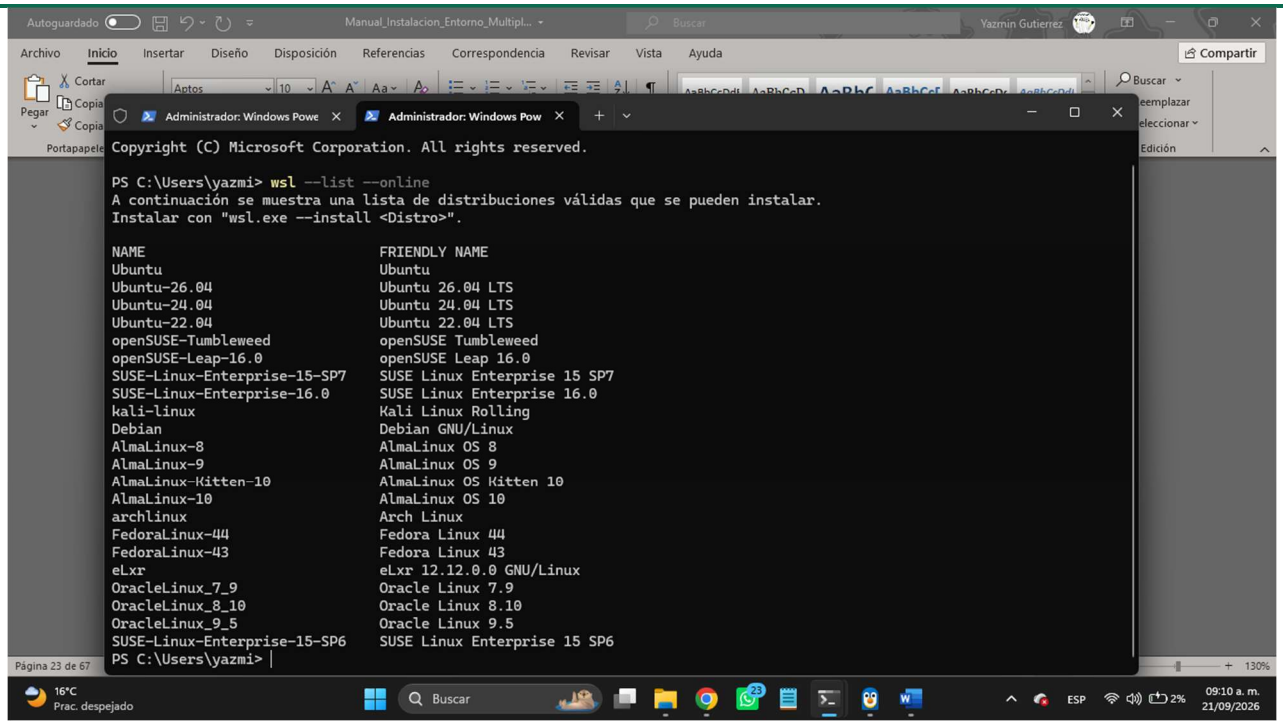


Figura 6. Consulta de distribuciones disponibles para WSL.

Fuente: Elaboración propia.

Resultado esperado: Aparece una lista de distribuciones Linux disponibles para instalar.

Instalar Ubuntu 24.04

Paso 1. En PowerShell como administrador ejecuta:

```
wsl --install -d Ubuntu-24.04
```

Paso 2. Presiona **Enter**.

Paso 3. Espera a que termine la instalación de Ubuntu.

Resultado esperado: Ubuntu 24.04 queda instalada en WSL.

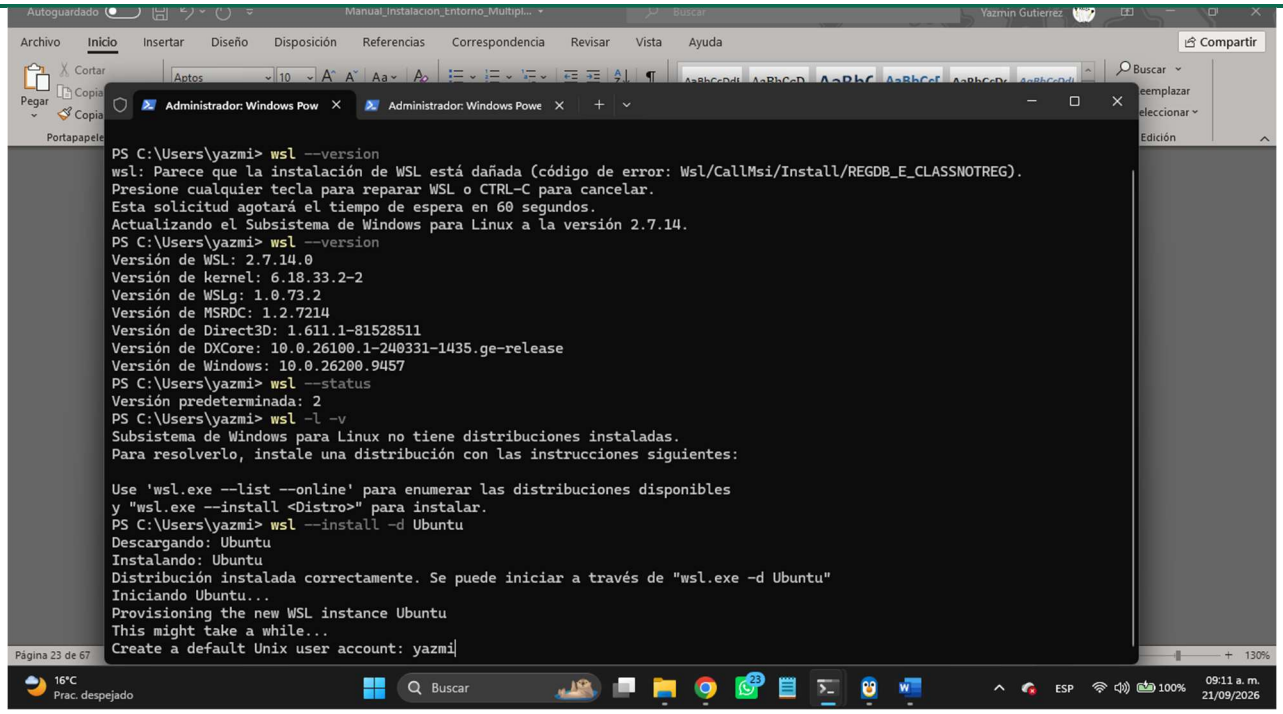


Figura 7. Instalación de Ubuntu 24.04 en WSL.

Fuente: Elaboración propia.

Establecer WSL2 como versión predeterminada

Paso 1. En PowerShell ejecuta:

```
wsl --set-default-version 2
```

Paso 2. Presiona **Enter**.

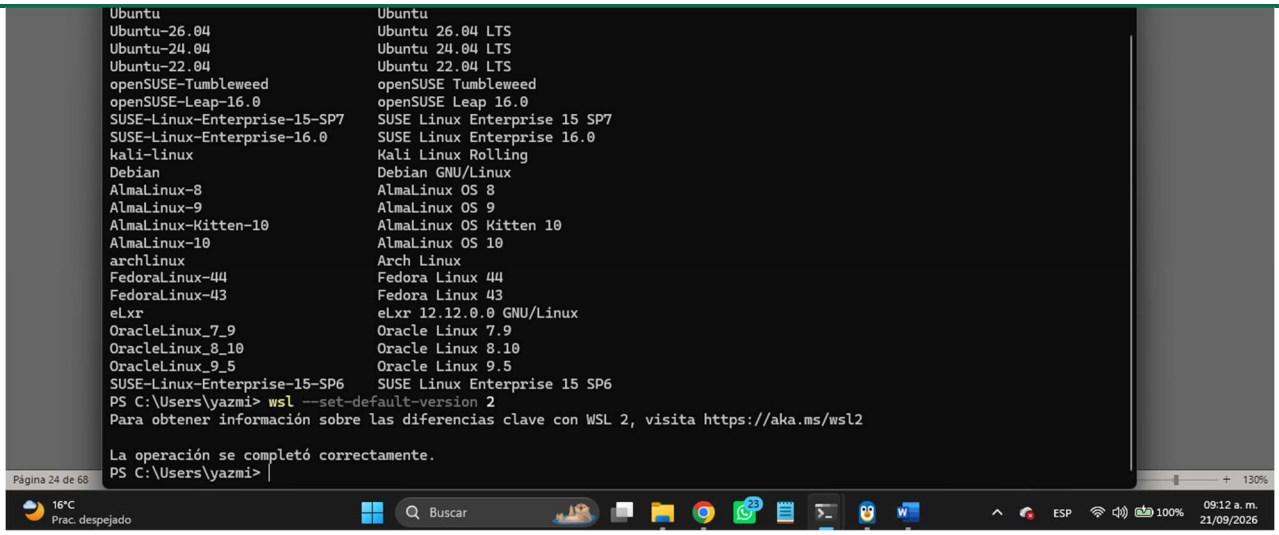


Figura 8. Configuración de WSL2 como versión predeterminada.

Fuente: Elaboración propia.

Resultado esperado: WSL2 queda configurado como la versión predeterminada para las nuevas distribuciones.

Actualizar WSL

Paso 1. Ejecuta:

```
wsl --update
```

Paso 2. Presiona **Enter**.

Paso 3. Espera a que finalice la actualización.

Resultado esperado: WSL queda actualizado.

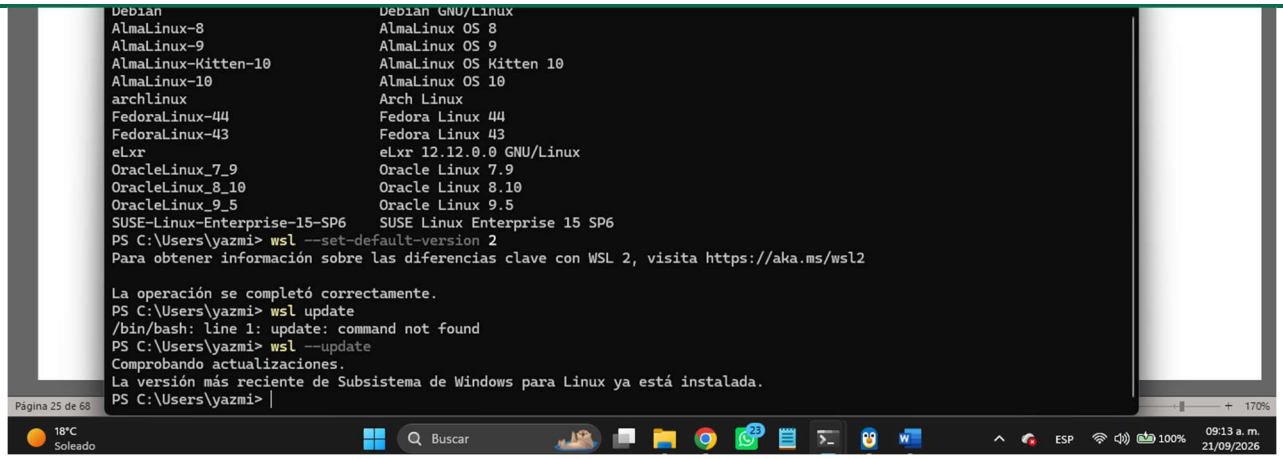


Figura 9. Actualización de WSL desde PowerShell.

Fuente: Elaboración propia.

Comprobar el estado de WSL

Paso 1. Ejecuta:

```
wsl --status
```

Paso 2. Presiona **Enter**.

Paso 3. Revisa la información mostrada sobre WSL y su configuración.

Resultado esperado: La consola muestra el estado de WSL y la configuración predeterminada.

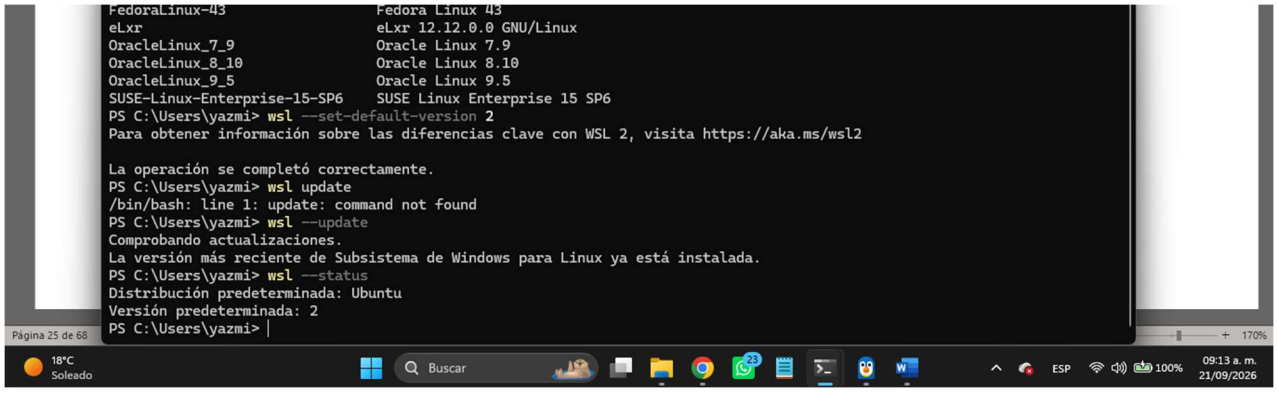


Figura 10. Verificación del estado de WSL.

Fuente: Elaboración propia.

Comprobar que Ubuntu utiliza WSL2

Paso 1. Ejecuta:

```
wsl --list --verbose
```

Paso 2. Presiona **Enter**.

Paso 3. Busca la distribución **Ubuntu-24.04**.

Paso 4. Revisa la columna **VERSION**.

Debe aparecer:

2

Resultado esperado: Ubuntu-24.04 aparece ejecutándose con **WSL2**.

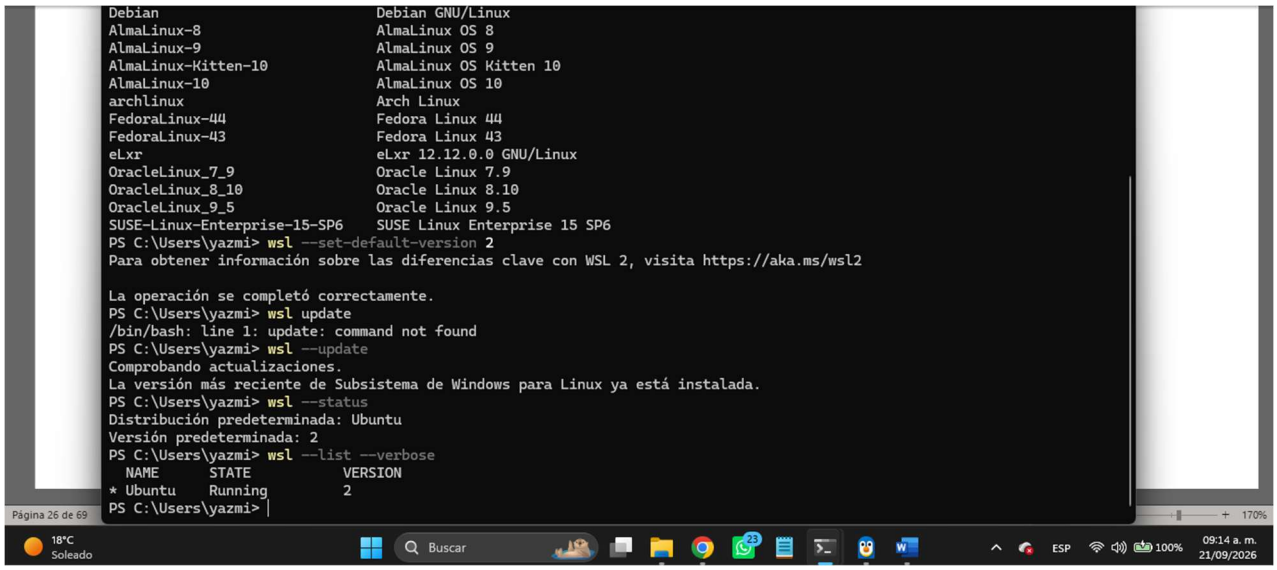


Figura 11. Ubuntu configurado para utilizar WSL2.
Fuente: Elaboración propia.

Comprobación final

Antes de continuar, verifica que:

- WSL está instalado.
- Ubuntu 24.04 está instalada.
- WSL2 está configurado como versión predeterminada.
- WSL está actualizado.
- Ubuntu aparece con **VERSION 2**.

INSTALACIÓN Y CONFIGURACIÓN DE VISUAL STUDIO CODE

Visual Studio Code se incorpora como editor principal para trabajar con el código del proyecto Flutter. Desde su terminal integrada se podrán ejecutar comandos de Flutter y Git. Android Studio se mantiene para administrar Android SDK, las herramientas Android y el emulador. De esta forma, la aplicación puede editarse y lanzarse desde VS Code mientras el dispositivo virtual es proporcionado por Android Studio.

Descargar Visual Studio Code

Paso 1. Abre el navegador web.

Paso 2. Ingresa al sitio oficial:

https://code.visualstudio.com/

Paso 3. Selecciona Download for Windows.

Paso 4. Espera a que termine la descarga del instalador.

RESULTADO ESPERADO: El instalador de Visual Studio Code queda disponible en la carpeta de descargas.

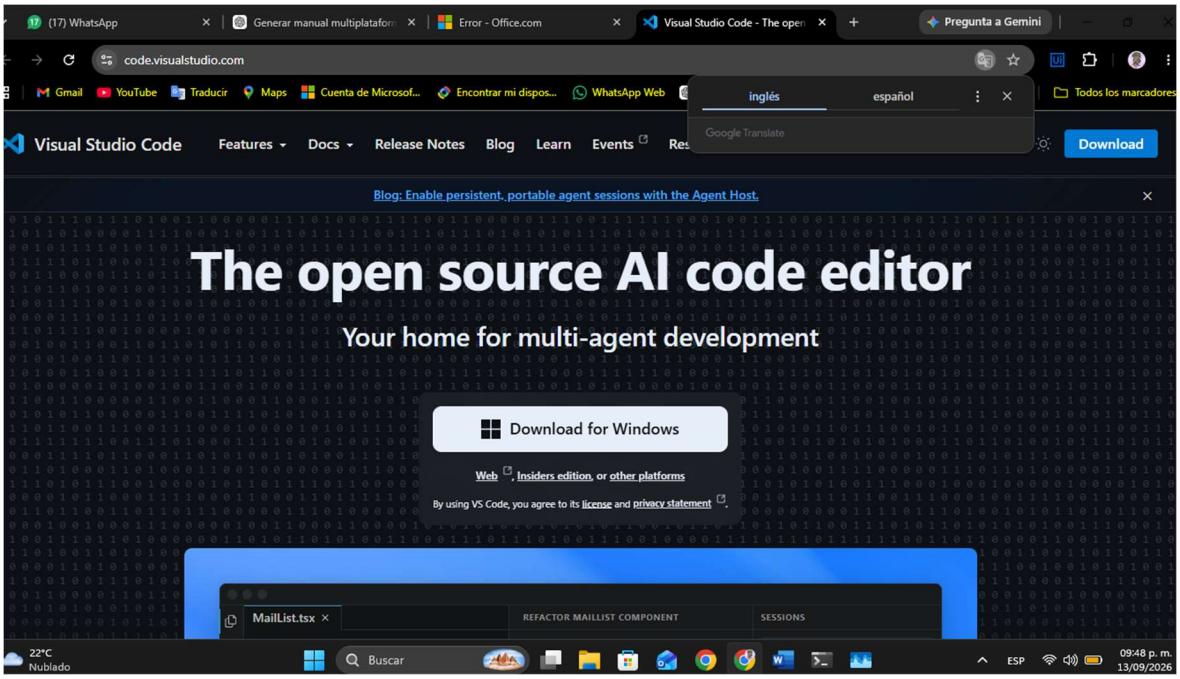


Figura 12. Descarga de Visual Studio Code desde el sitio oficial.

Fuente: Elaboración propia.

Instalar Visual Studio Code

Paso 1. Ejecuta el instalador descargado.

Paso 2. Lee y acepta los términos de licencia para continuar.

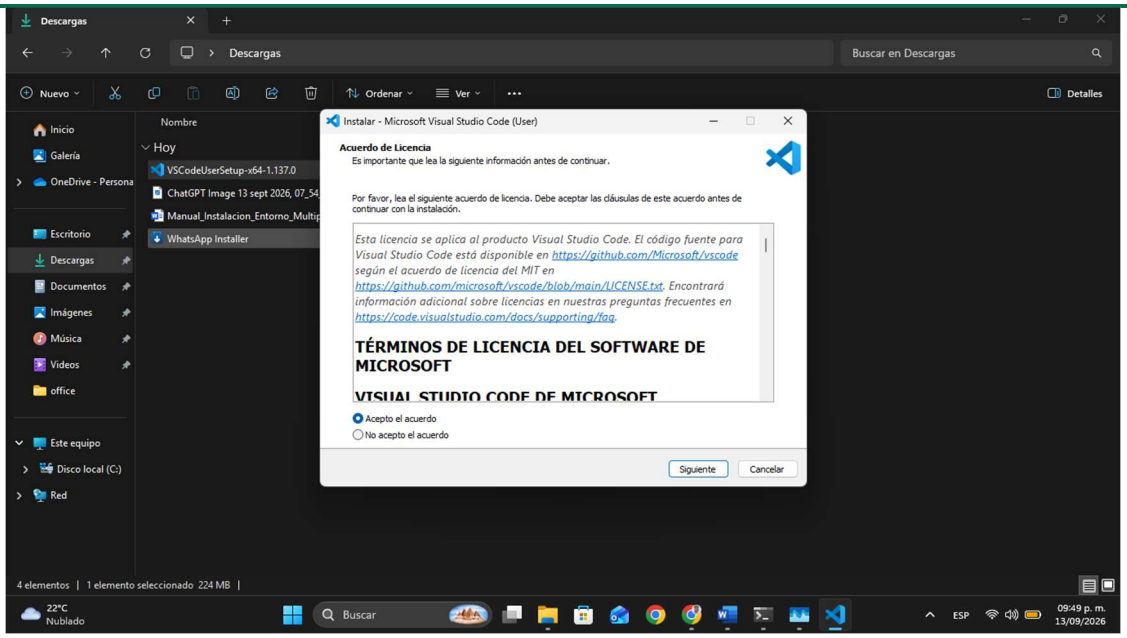


Figura 13. Aceptación de los términos de licencia de Visual Studio Code.

Fuente: Elaboración propia.

Paso 3. Selecciona la ubicación de instalación y continúa con Siguiente.

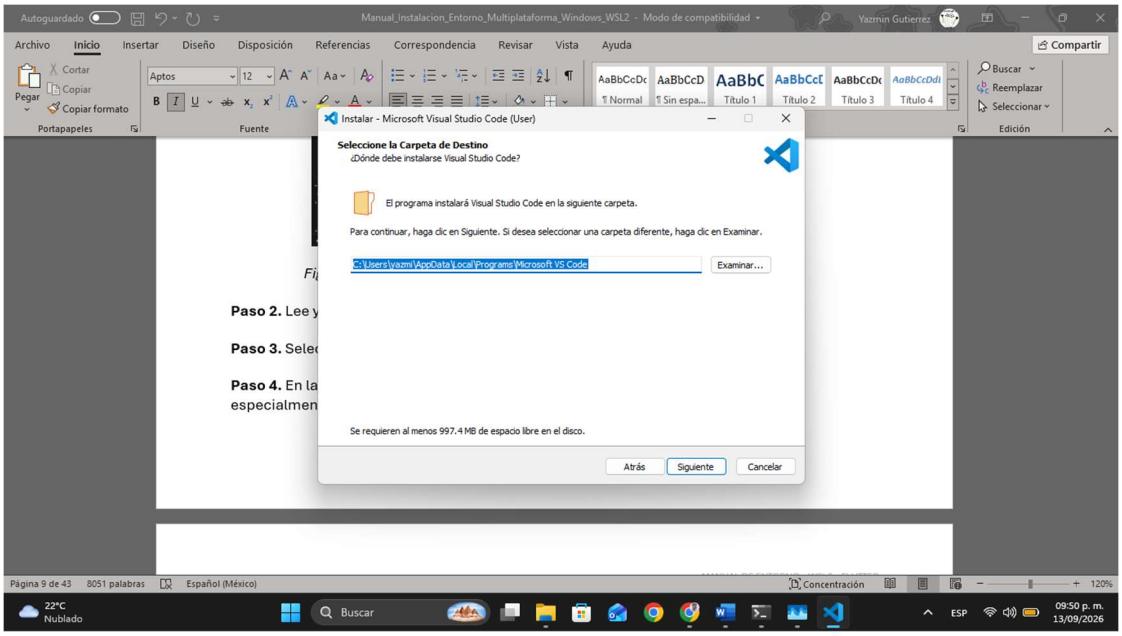


Figura 14. Selección de la carpeta de instalación de Visual Studio Code.

Fuente: Elaboración propia.

Paso 4. En las tareas adicionales, activa las opciones de integración necesarias, especialmente agregar Visual Studio Code al PATH y al menú contextual.

Paso 5. Selecciona Instalar y espera a que concluya el proceso.

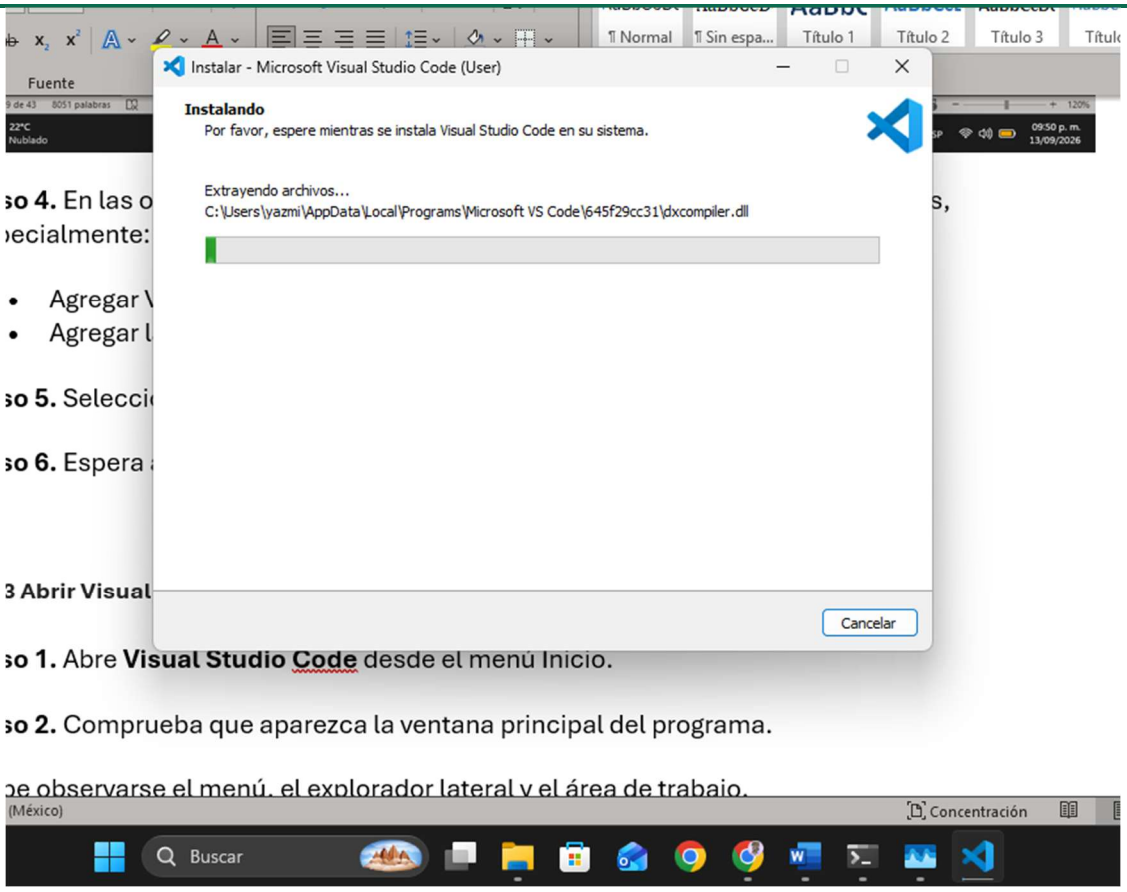


Figura 15. Proceso de instalación de Visual Studio Code en Windows.

Fuente: Elaboración propia.

Paso 6. Selecciona Finalizar.

RESULTADO ESPERADO: Visual Studio Code queda instalado y puede abrirse desde Windows.

Abrir Visual Studio Code

Paso 1. Abre Visual Studio Code desde el menú Inicio.

Paso 2. Comprueba que se muestre la ventana principal, el explorador lateral y el área de trabajo.

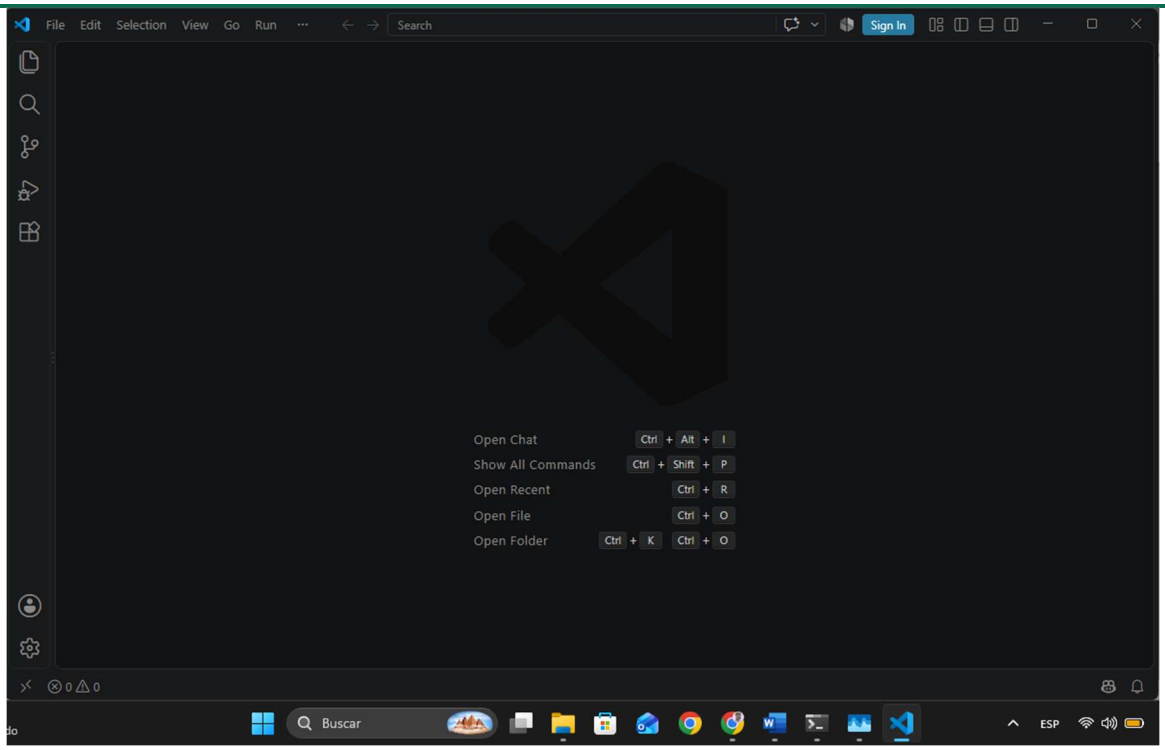


Figura 16. Visual Studio Code abierto correctamente después de la instalación.

Fuente: Elaboración propia.

Instalar extensiones del entorno

Instala únicamente las extensiones relacionadas con el entorno actual:

Flutter – soporte para proyectos Flutter.

Dart – análisis, formato y depuración de Dart.

WSL – integración con Windows Subsystem for Linux.

Extension Pack for Java – soporte para edición y herramientas Java.

Docker – apoyo opcional para revisar contenedores y archivos Docker desde el editor.

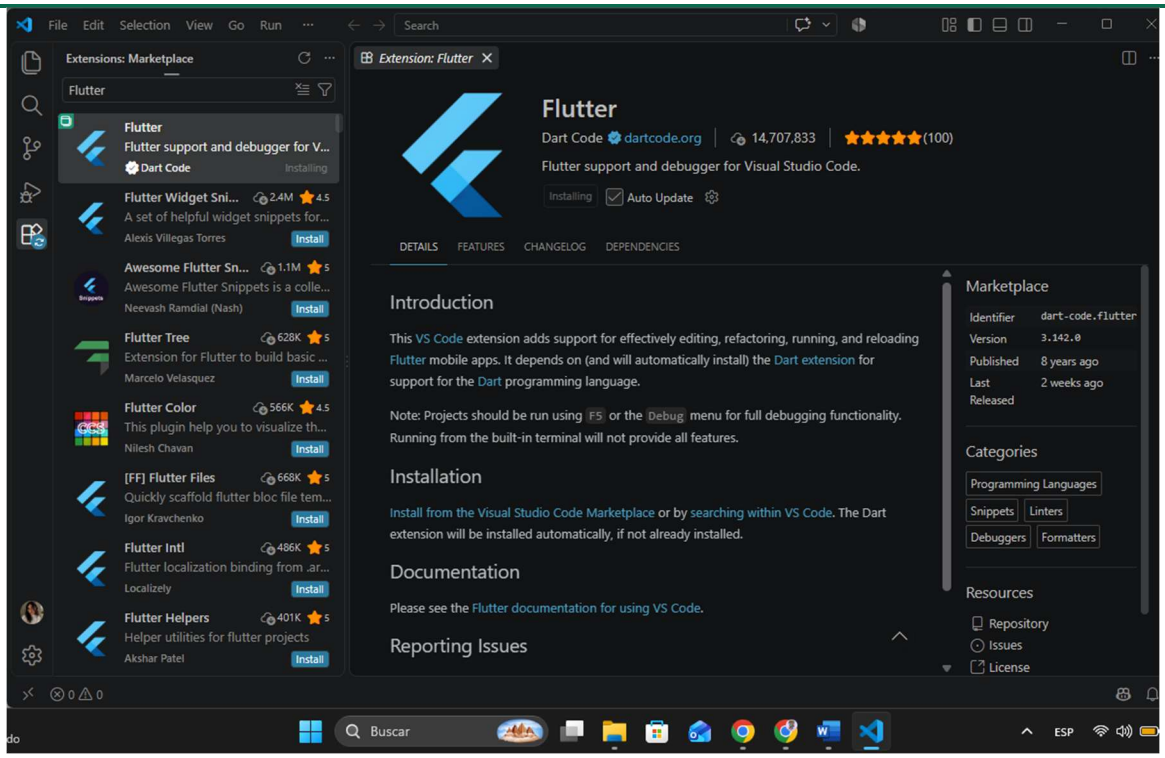


Figura 17. Extensión Flutter disponible en Visual Studio Code.

Fuente: Elaboración propia.

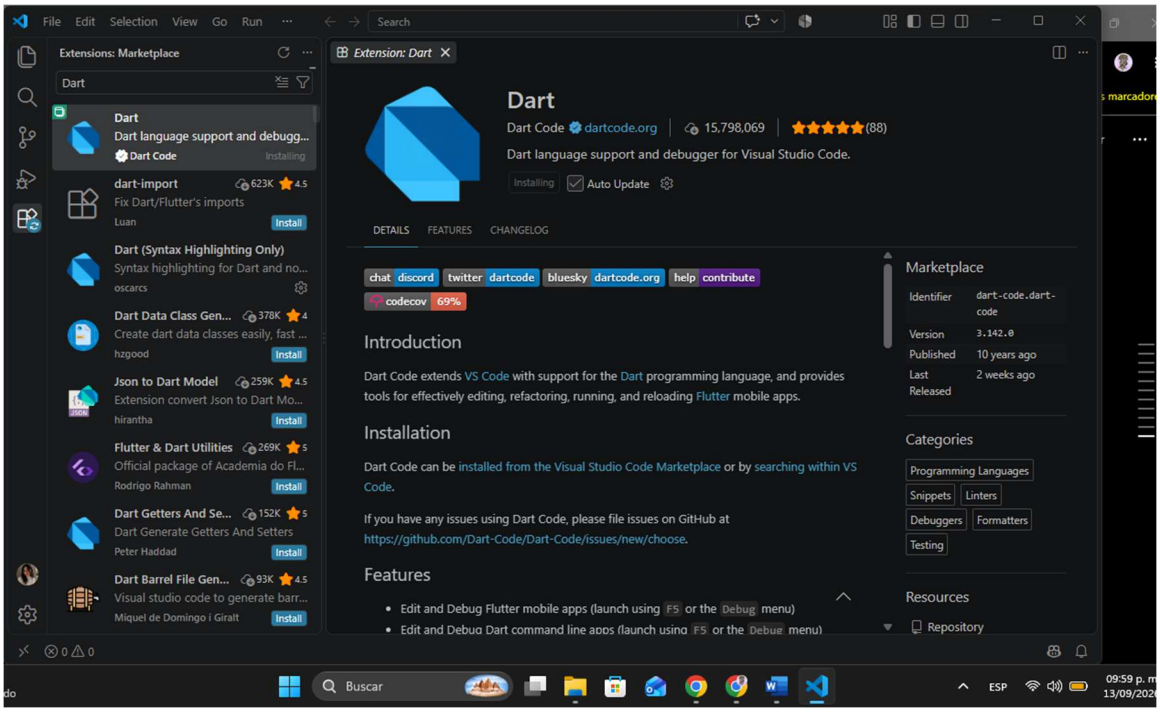


Figura 18. Extensión Dart disponible en Visual Studio Code.

Fuente: Elaboración propia.

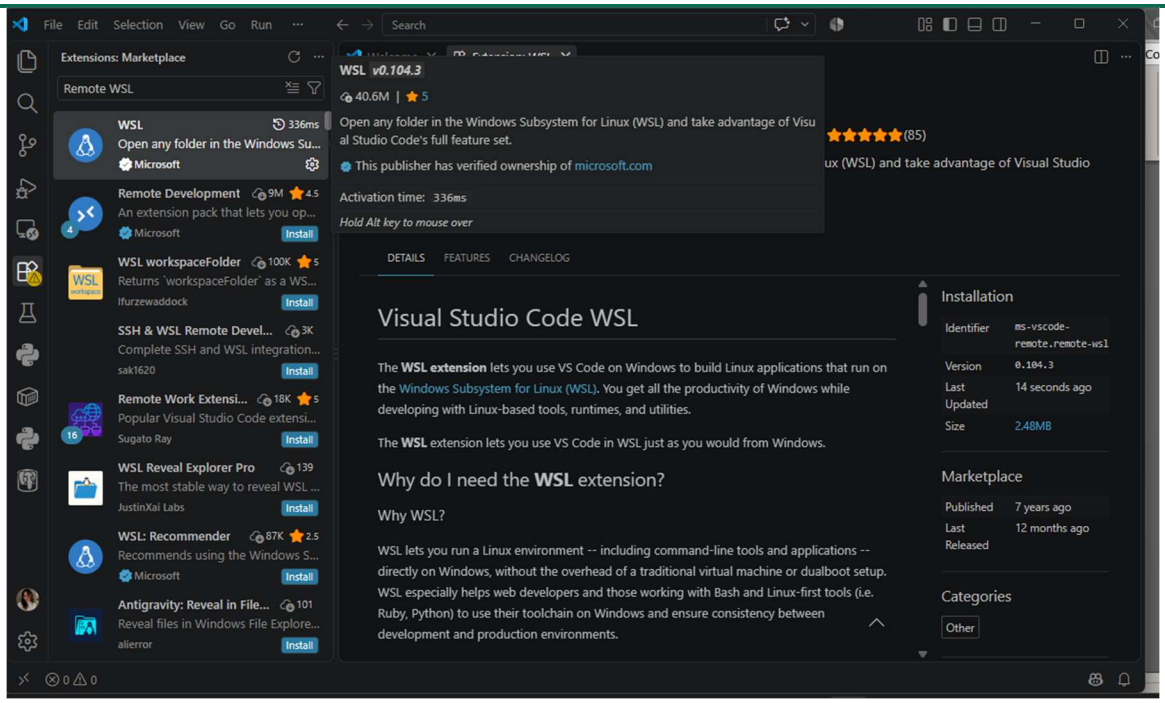


Figura 19. Extensión WSL disponible en Visual Studio Code.

Fuente: Elaboración propia.

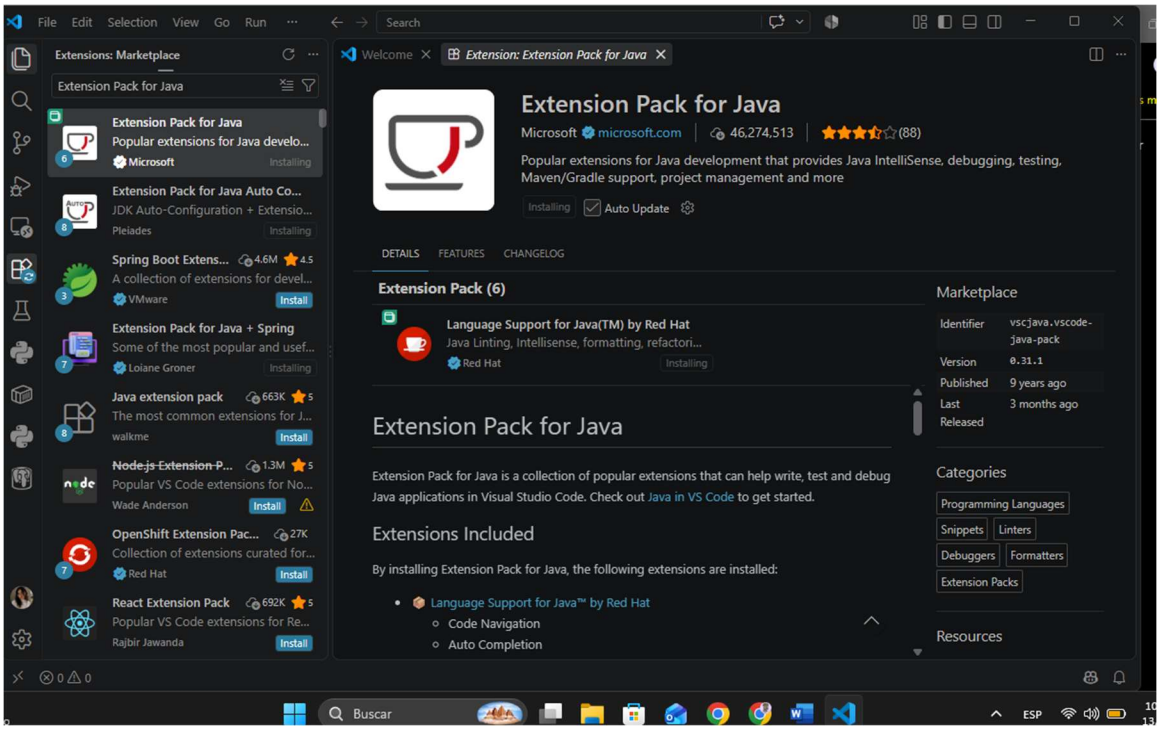


Figura 20. Extension Pack for Java disponible en Visual Studio Code.

Fuente: Elaboración propia.

INSTALACIÓN DEL JDK 21

El Java Development Kit (JDK) contiene las herramientas necesarias para desarrollar, compilar y ejecutar aplicaciones Java. En un proyecto Flutter para Android, la parte nativa de Android se compila mediante Gradle, y Gradle necesita un JDK compatible. En este entorno se instalará JDK 21 LTS mediante Eclipse Temurin, distribución de OpenJDK proporcionada por Adoptium. Android Studio también puede incluir su propio runtime Java; la instalación manual permite disponer de Java desde CMD, PowerShell, VS Code y Maven mediante JAVA_HOME.

Nota técnica: Android no se compila con Maven como sistema principal. Los proyectos Android modernos, incluido el módulo android/ de Flutter, utilizan Gradle. Si Android Studio o flutter doctor reportan una incompatibilidad entre JDK, Gradle o Android Gradle Plugin, debe utilizarse la versión de JDK compatible con las versiones instaladas.

Descarga del JDK 21

Paso 1. Abre el sitio oficial de Adoptium:

https://adoptium.net/

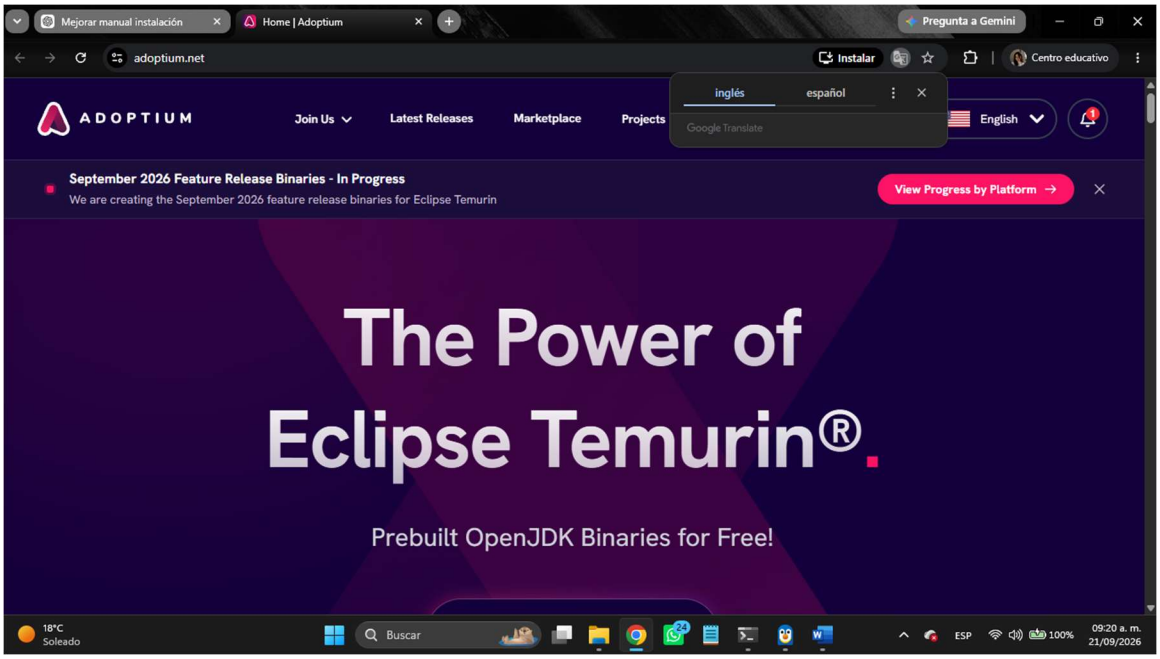


Figura 21. Sitio oficial de Eclipse Temurin para descargar JDK 21.

Fuente: Elaboración propia.

Paso 2. Selecciona JDK 21 LTS, Windows, arquitectura x64 y Eclipse Temurin.

Paso 3. Descarga el instalador disponible para Windows.

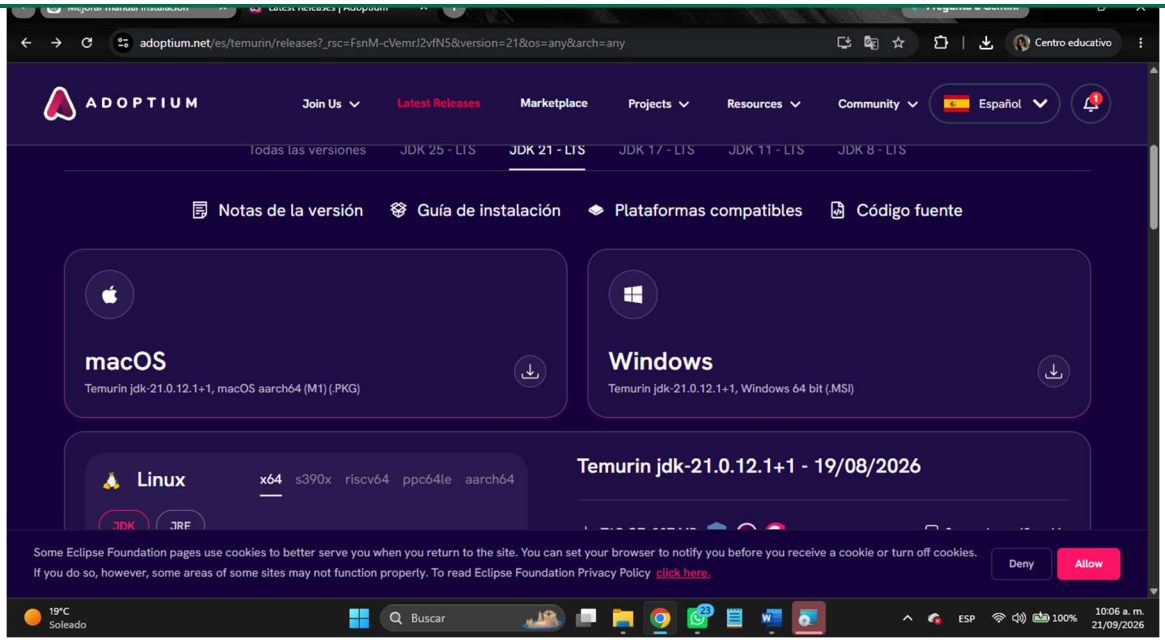


Figura 22. Selección de JDK 21 LTS para Windows x64.

Fuente: Elaboración propia.

Instalación del JDK

Paso 4. Ejecuta el instalador descargado y acepta las condiciones correspondientes.

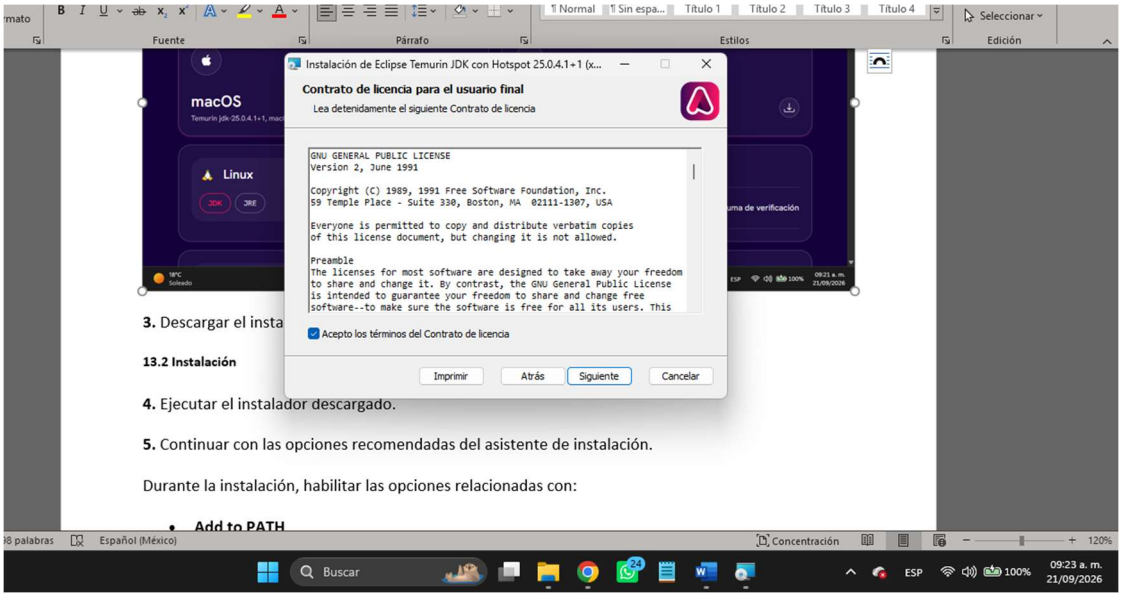


Figura 23. Inicio del asistente de instalación de Eclipse Temurin JDK 21.

Fuente: Elaboración propia.

Paso 5. Mantén las opciones recomendadas y habilita las opciones relacionadas con Add to PATH y Set JAVA_HOME variable.

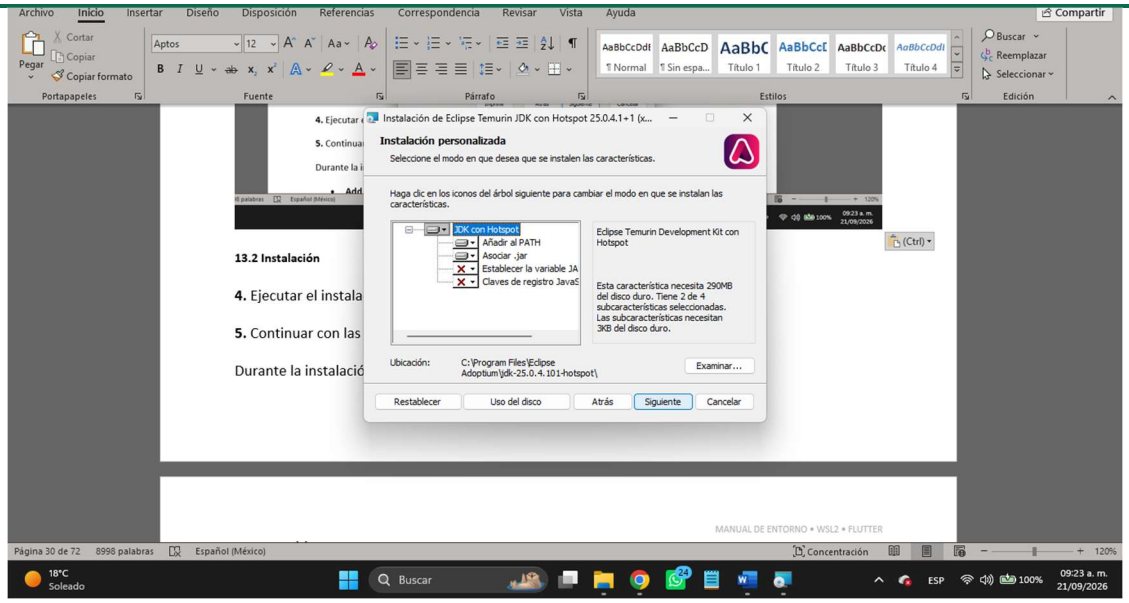


Figura 24. Opciones de instalación para PATH y JAVA_HOME del JDK 21.

Fuente: Elaboración propia.

Paso 6. Inicia la instalación y espera hasta que concluya.

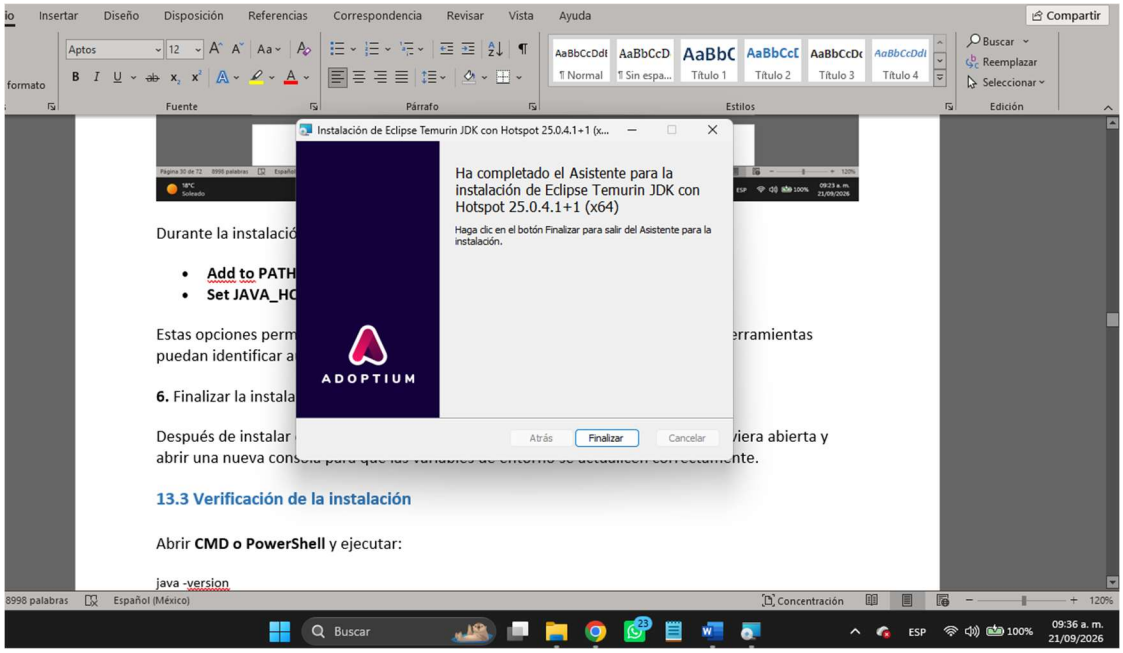


Figura 25. Proceso de instalación del JDK 21 en Windows.

Fuente: Elaboración propia.

Después de finalizar, cierra las consolas abiertas y abre una nueva ventana de CMD o PowerShell para que Windows cargue las variables actualizadas.

Verificación de Java

Abre CMD o PowerShell y ejecuta:

java -version

DÓNDE EJECUTARLO: Windows / CMD o PowerShell.

RESULTADO ESPERADO: La consola muestra Java 21 junto con información de Eclipse Temurin u OpenJDK.

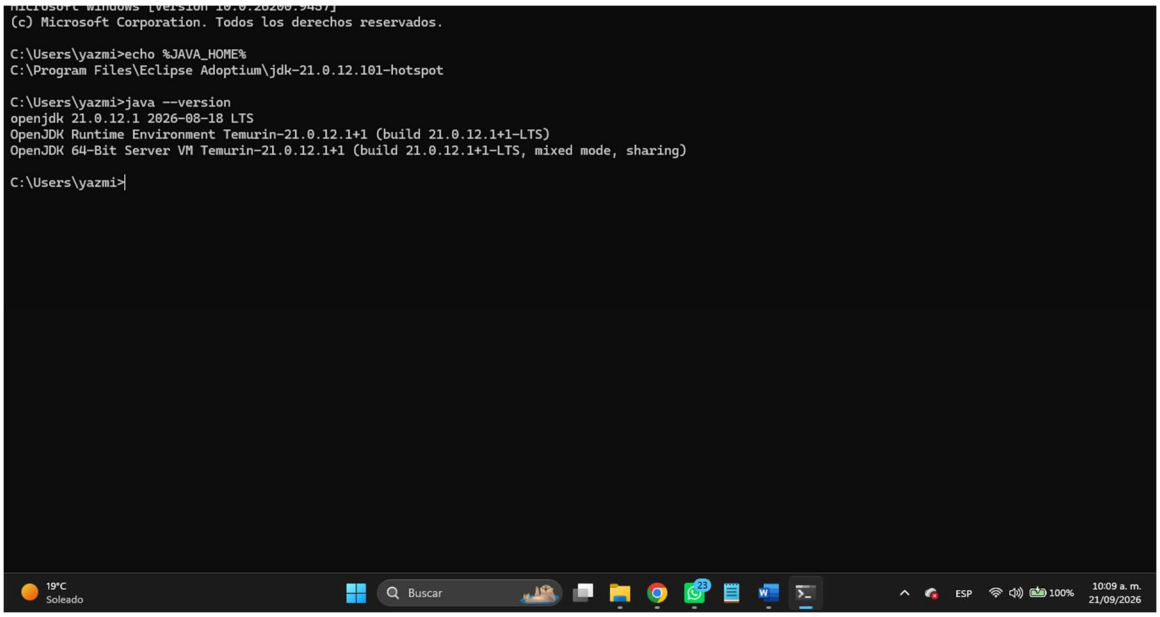


Figura 26. JDK 21 instalado y reconocido desde la terminal de Windows.
Fuente: Elaboración propia.

INSTALACIÓN DE NODE.JS Y NPM

Node.js proporciona un entorno de ejecución de JavaScript y npm administra paquetes. En este manual ambos se conservan únicamente como herramientas disponibles del entorno y se verifican mediante sus comandos de versión.

1. Abrir https://nodejs.org/en/download/.

2. Elegir LTS para Windows x64.

3. Ejecutar el instalador.

4. Mantener opciones recomendadas.

5. Abrir una consola nueva.

```
node --version
npm --version
```

DÓNDE EJECUTARLO: [WINDOWS / CMD o PowerShell]

RESULTADO ESPERADO: Se muestran ambas versiones.

EVIDENCIA: E08

Evidencia E08 – Node.js y npm disponibles

MOMENTO: Después de ejecutar node --version y npm --version.

ACCIÓN: Capturar la terminal con ambas versiones visibles.

PANTALLA: CMD o PowerShell.

LA CAPTURA DEBE MOSTRAR: Las versiones instaladas de Node.js y npm.

NO DEBE MOSTRAR: Contraseñas, tokens o credenciales.

NOMBRE DEL ARCHIVO: E05_Node_Npm.png

[INSERTAR AQUÍ LA CAPTURA DE EVIDENCIA E08]

INSTALACIÓN Y CONFIGURACIÓN DE ANDROID STUDIO

Android Studio es el entorno oficial de desarrollo para aplicaciones Android. Su instalación incluye las herramientas necesarias para trabajar con Android SDK, administrar versiones de Android y preparar emuladores. Para la compilación Android utiliza Gradle, que requiere un JDK compatible; por ello se conserva la instalación de JDK 21 documentada anteriormente. Maven permanece disponible como herramienta Java adicional, pero no reemplaza a Gradle en el proceso de compilación Android.

Descarga de Android Studio

1. Abrir el sitio oficial de Android Studio en el navegador:

https://developer.android.com/studio

2. Seleccionar la opción de descarga para **Windows**.

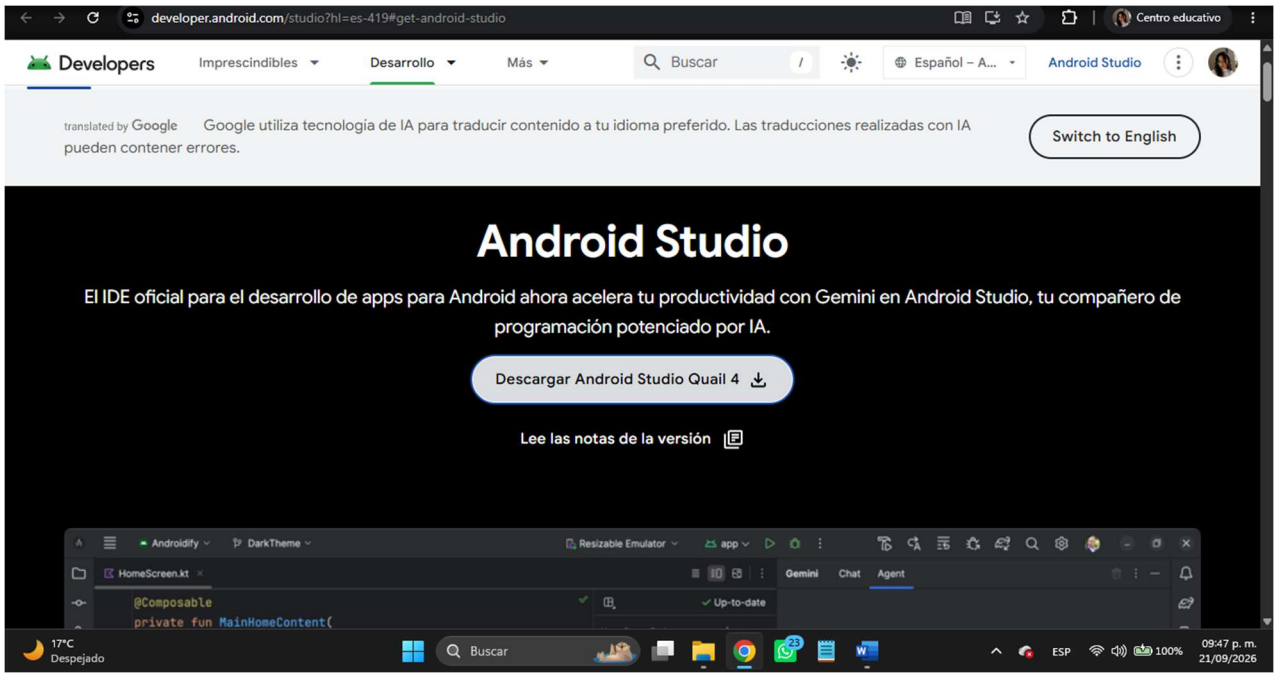


Figura 27. Sitio oficial de descarga de Android Studio.

Fuente: Elaboración propia.

3. Descargar el instalador ejecutable `.exe`.

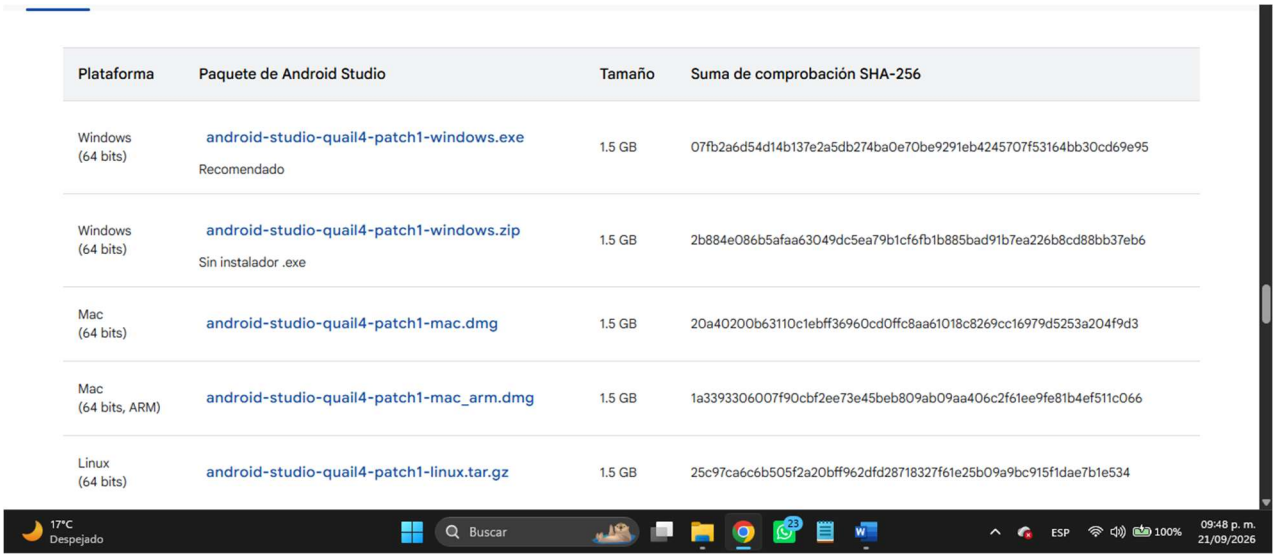


Figura 28. Opciones de descarga de Android Studio para Windows.

Fuente: Elaboración propia.

Instalación

4. Ejecutar el archivo descargado.

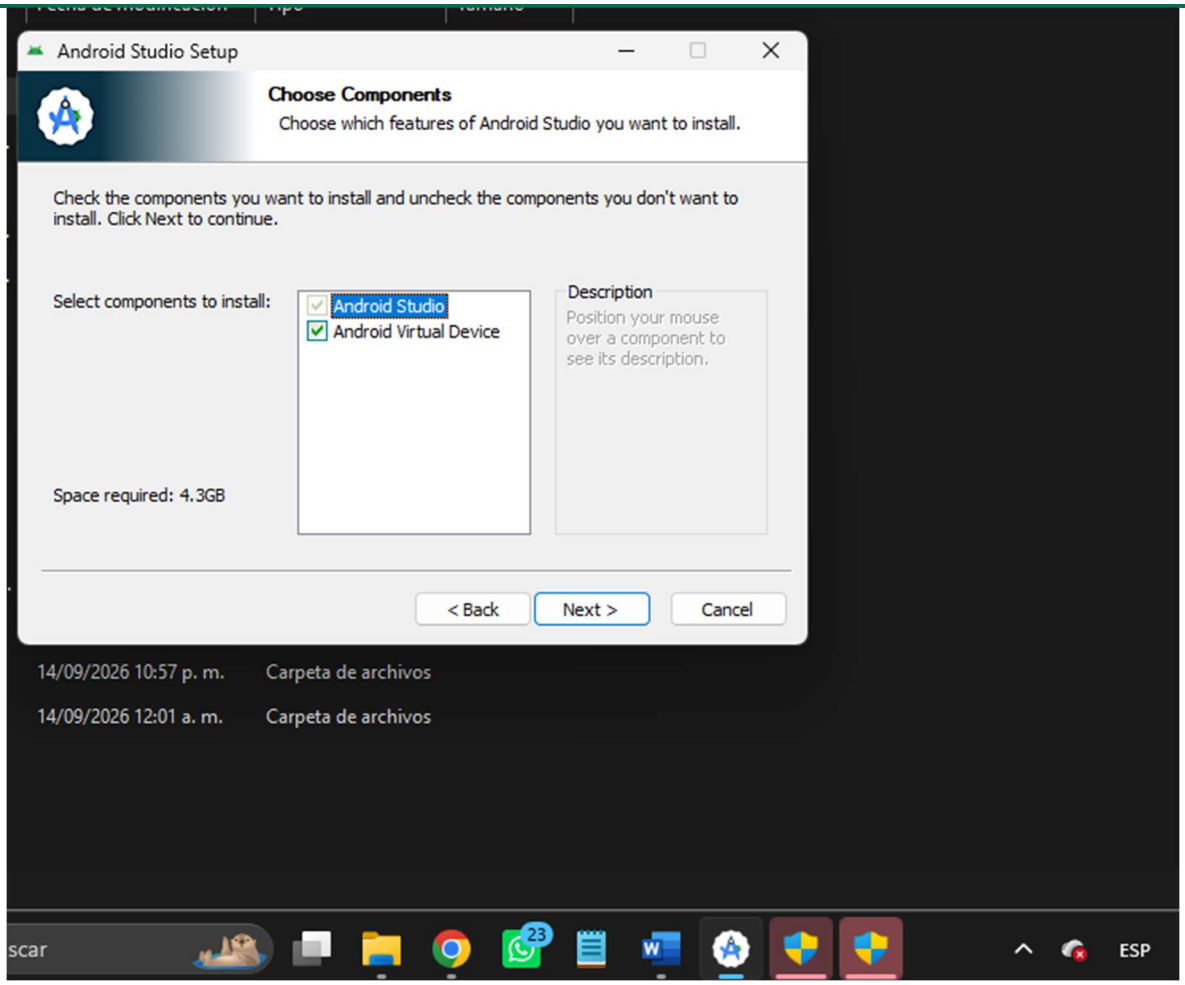


Figura 29. Selección de componentes durante la instalación de Android Studio.
Fuente: Elaboración propia.

5. Aceptar los términos y condiciones de la licencia cuando sean solicitados.

6. Mantener seleccionados los componentes recomendados, incluyendo:

- **Android Studio**
- **Android SDK**
- Herramientas adicionales requeridas por el instalador

7. Continuar con la instalación utilizando las opciones recomendadas.

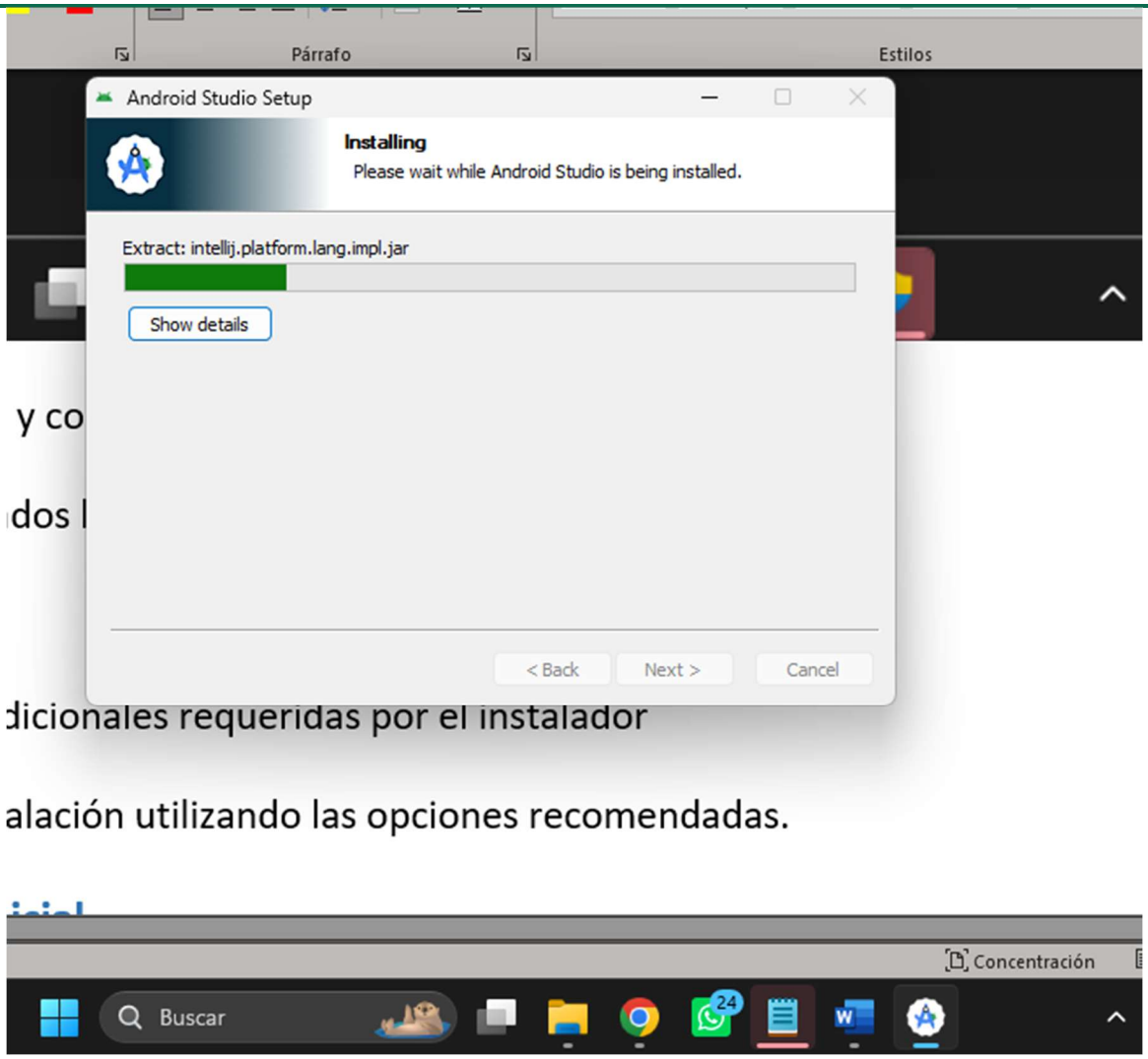


Figura 30. Proceso de instalación de Android Studio.

Fuente: Elaboración propia.

Configuración inicial

8. Al abrir Android Studio por primera vez, completar el **Setup Wizard**.

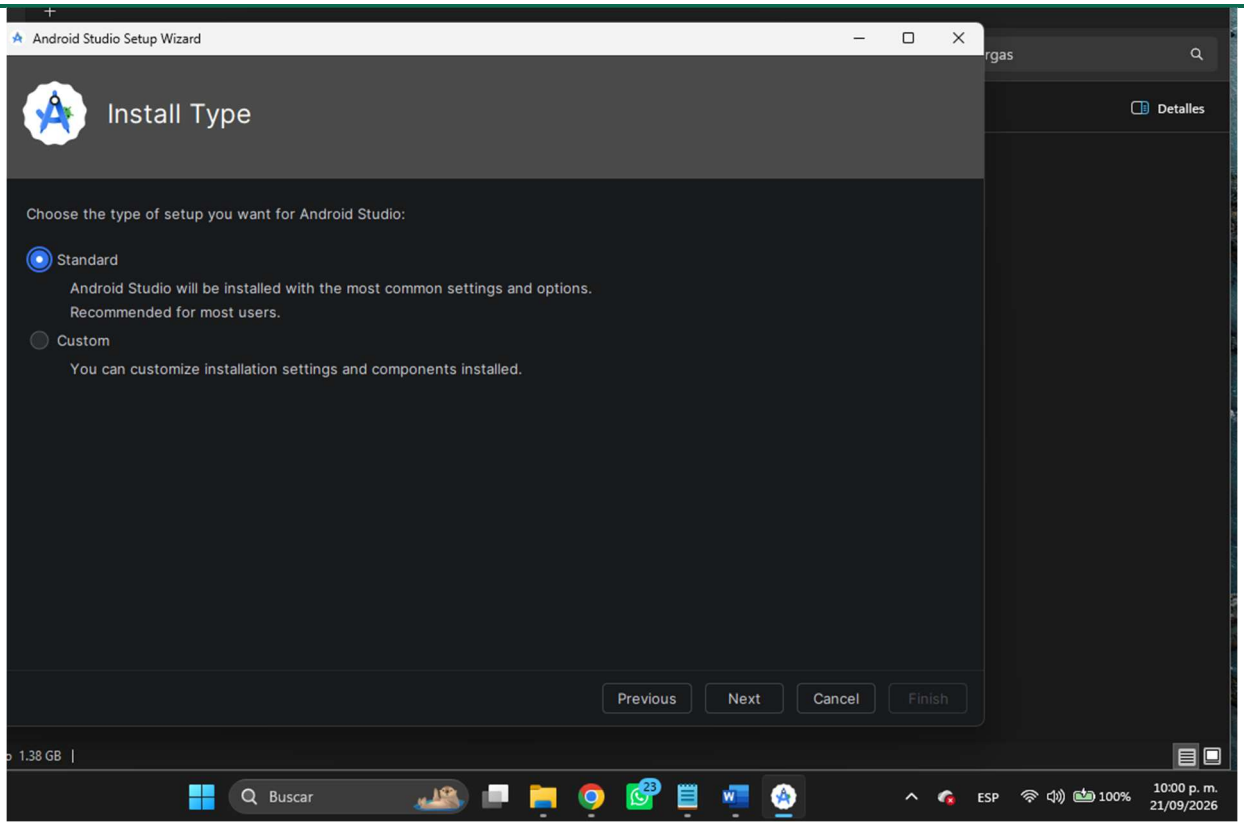


Figura 31. Asistente de configuración inicial de Android Studio.

Fuente: Elaboración propia.

Este asistente permite descargar y configurar los componentes necesarios para utilizar el entorno de desarrollo.

9. Una vez finalizada la configuración inicial, abrir el **SDK Manager**.

10. Revisar las secciones:

- **SDK Platforms**
- **SDK Tools**

Verificar que estén instalados los componentes necesarios para el desarrollo de aplicaciones Android.

11. Aplicar los cambios y esperar a que finalice la instalación de los componentes seleccionados.

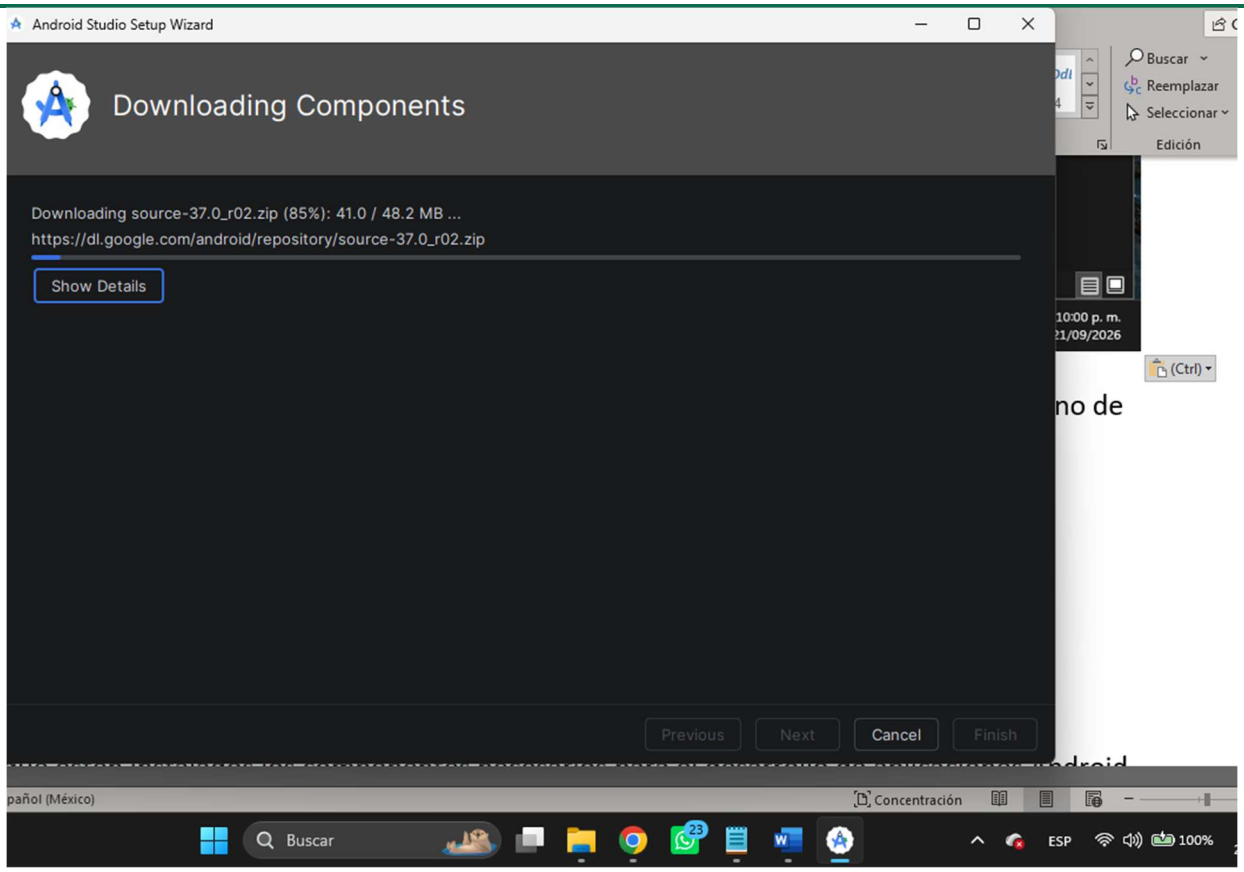


Figura 32. Descarga de componentes del Android SDK durante la configuración inicial.
Fuente: Elaboración propia.

Verificación

La instalación se considera correcta cuando Android Studio inicia sin errores y el **SDK Manager** permite visualizar las plataformas y herramientas del Android SDK instaladas.

Seguridad

Durante la elaboración de las evidencias del manual, evitar mostrar información sensible.

Las capturas de pantalla no deben incluir:

- Contraseñas.
- Tokens.
- Claves privadas.
- Credenciales.
- Información personal innecesaria.
- Datos de cuentas personales.

Si alguna información sensible aparece en pantalla, debe recortarse o difuminarse antes de insertar la imagen en el manual.

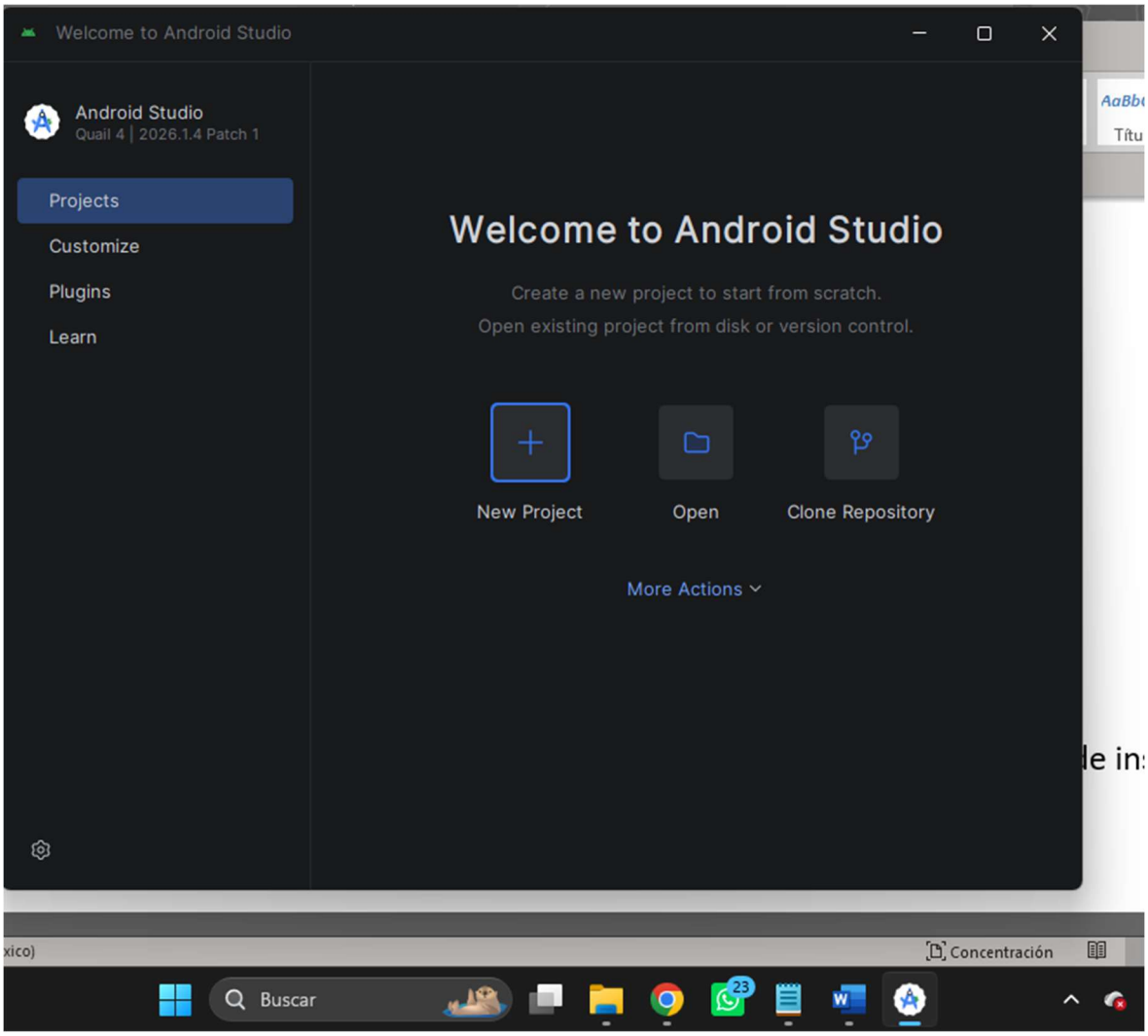


Figura 33. Android Studio instalado y abierto correctamente.
Fuente: Elaboración propia.

CREACIÓN DE UN DISPOSITIVO VIRTUAL ANDROID

Un **Android Virtual Device (AVD)** es un dispositivo virtual que permite ejecutar y probar aplicaciones Android sin necesidad de utilizar un teléfono físico.

El AVD combina un perfil de hardware, una versión de Android, memoria, almacenamiento y otras opciones de configuración. Para utilizar el emulador correctamente, la **virtualización del procesador** debe estar habilitada en el equipo.

En equipos con recursos limitados, puede ser más conveniente utilizar un dispositivo Android físico.

Abrir Device Manager

1. Abrir **Android Studio**.

2. Acceder a **Device Manager**.

Dependiendo de la versión de Android Studio, puede encontrarse en:

```
Tools > Device Manager
```

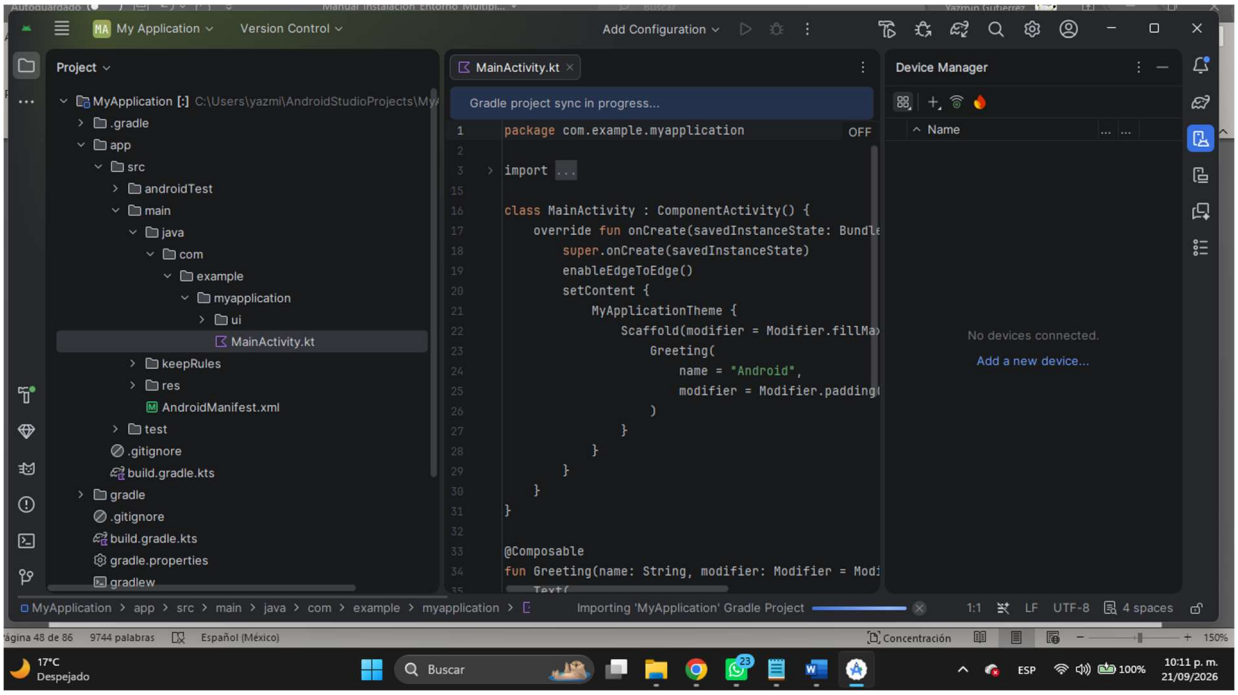


Figura 34. Acceso al Device Manager de Android Studio.
Fuente: Elaboración propia.

Crear un dispositivo virtual

3. Seleccionar la opción:

```
Create Virtual Device
```

4. Elegir un perfil de dispositivo.

Se recomienda utilizar un perfil con características moderadas para evitar un consumo excesivo de memoria y procesamiento.

Por ejemplo:

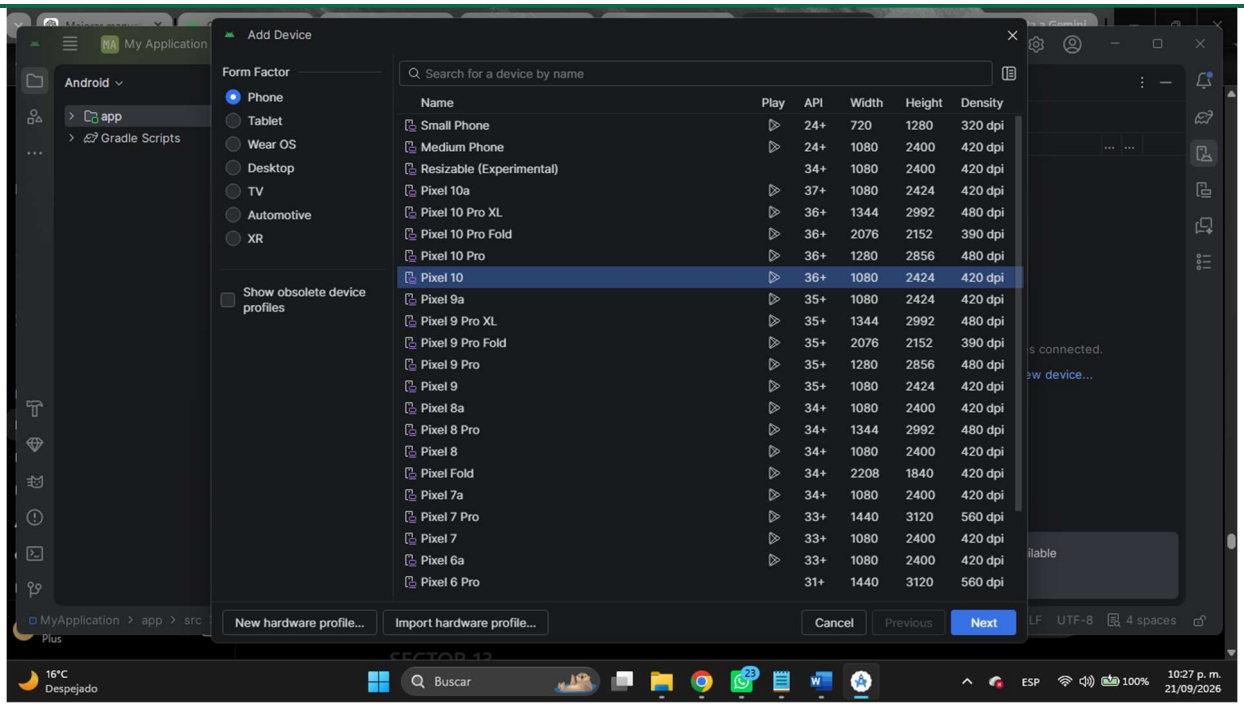


Figura 35. Selección del perfil de dispositivo virtual Android.
Fuente: Elaboración propia.

5. Continuar con **Next**.

Seleccionar la versión de Android

6. Elegir una imagen del sistema Android estable y compatible con el entorno de desarrollo.

Por ejemplo:

```
Android 14
Android 15
```

Si la imagen todavía no está instalada, seleccionar **Download** y esperar a que finalice la descarga.

7. Seleccionar la imagen instalada y continuar con **Next**.

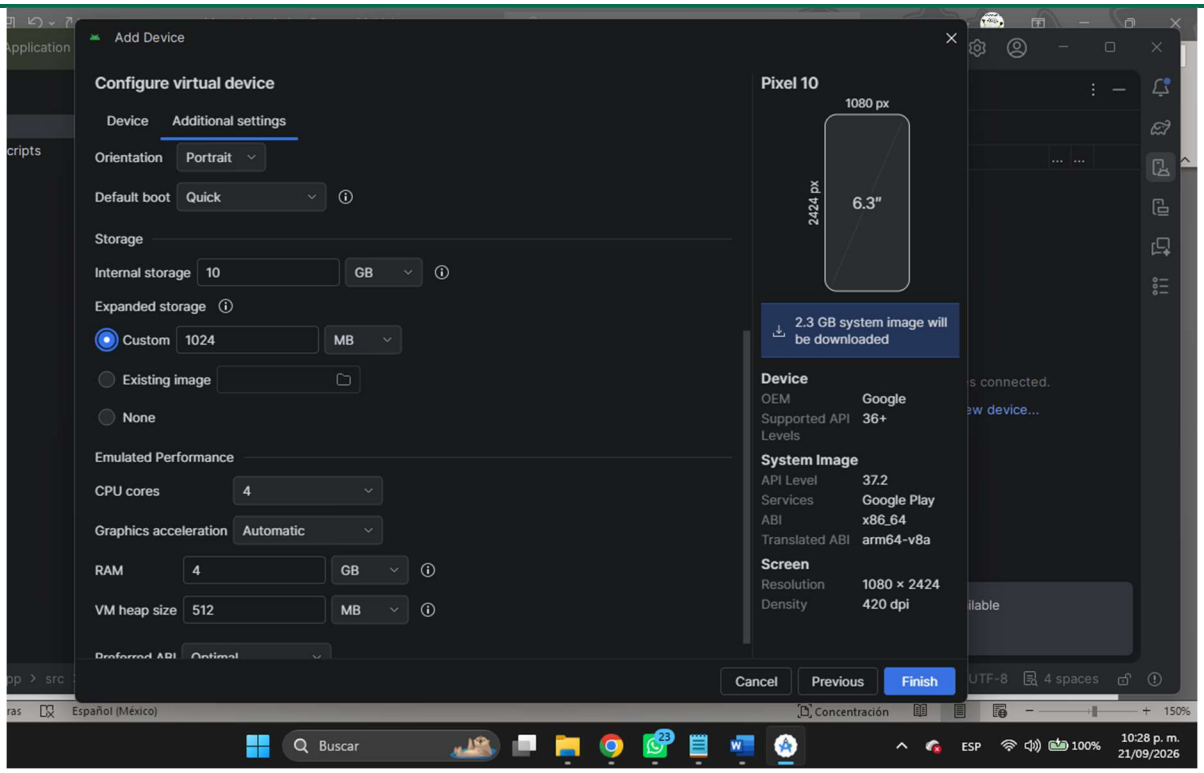


Figura 36. Selección de la imagen del sistema Android para el AVD.

Fuente: Elaboración propia.

Configurar el AVD

8. Revisar la configuración principal del dispositivo virtual.

Verificar principalmente:

- Nombre del dispositivo.
- Perfil de hardware.
- Versión de Android.
- Arquitectura de la imagen.
- Orientación.
- Memoria disponible.
- Almacenamiento.

En la mayoría de los casos, se pueden conservar los valores recomendados por Android Studio.

9. Seleccionar **Finish** para crear el dispositivo virtual.

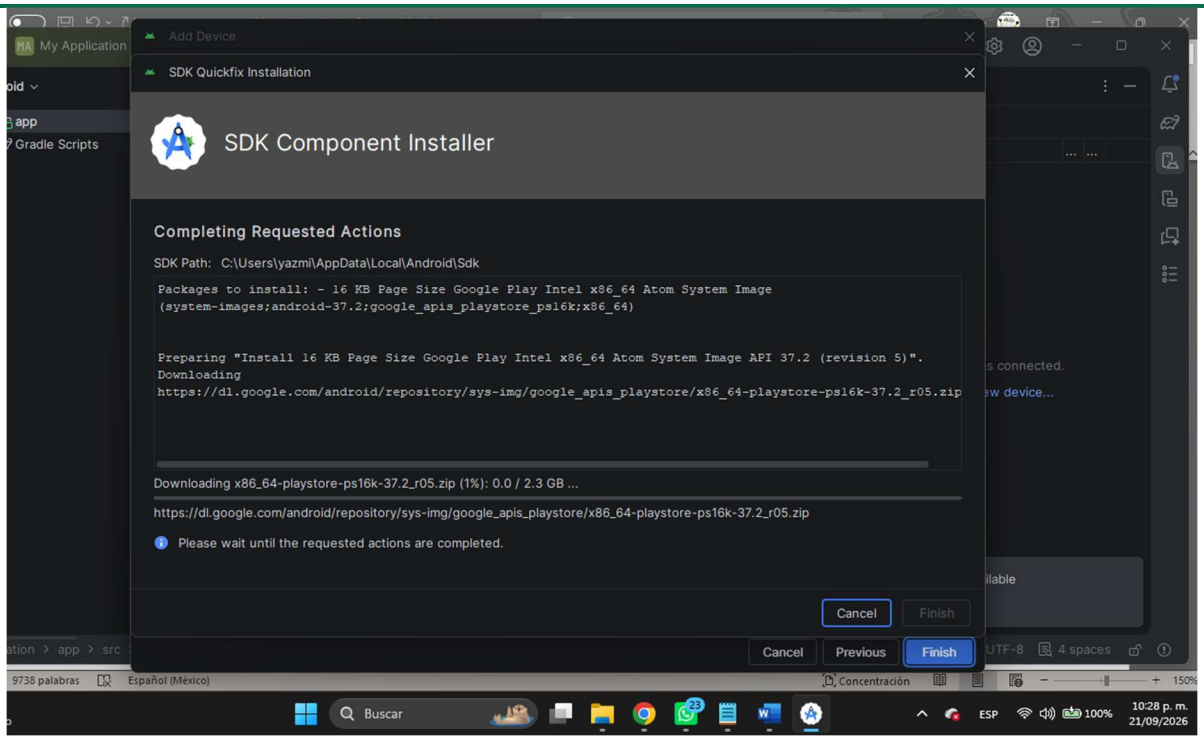


Figura 37. Configuración del dispositivo virtual Android.
Fuente: Elaboración propia.

Iniciar el emulador

10. En **Device Manager**, localizar el dispositivo virtual creado.

11. Presionar el botón de ejecución ▶ para iniciar el emulador.

12. Esperar a que el sistema Android termine de iniciar y muestre la pantalla principal.

La primera ejecución puede tardar algunos minutos, ya que el emulador debe completar la configuración inicial del sistema.

Si se muestra correctamente la pantalla principal de Android, el dispositivo virtual se encuentra listo para ejecutar aplicaciones.

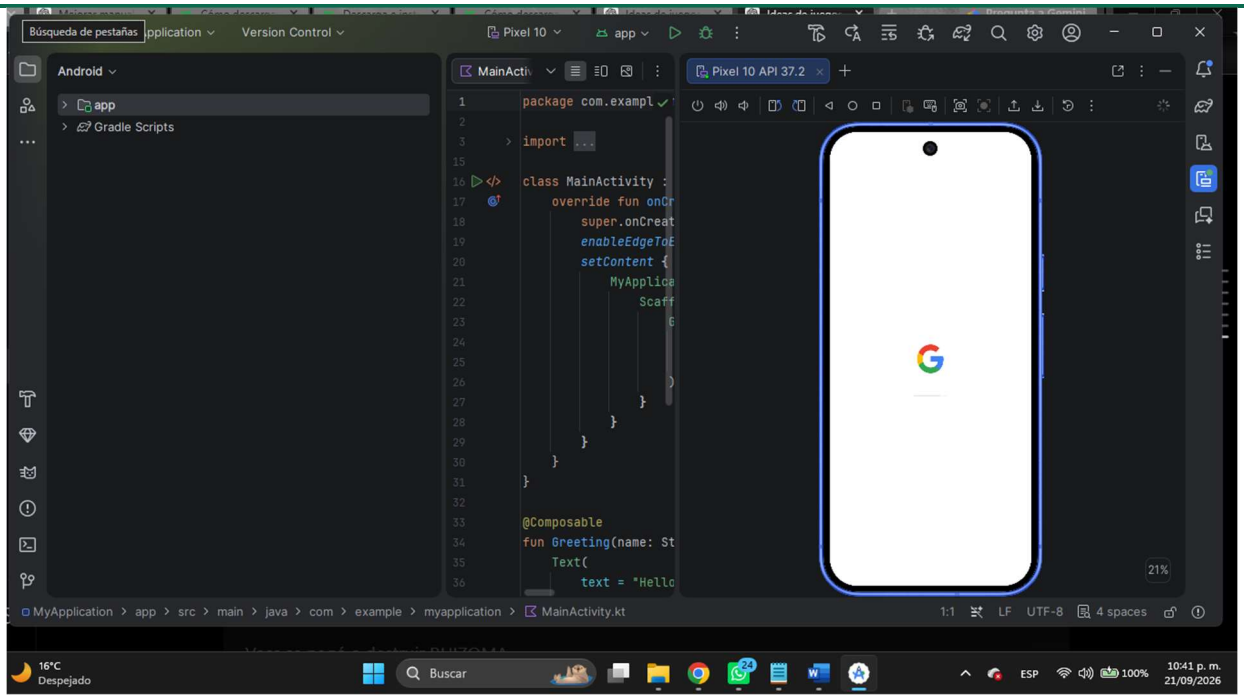


Figura 38. Dispositivo virtual configurado en Device Manager.

Fuente: Elaboración propia.

EMULADOR FUNCIONANDO

Pantalla: Android Emulator.

La captura debe mostrar:

- El dispositivo virtual encendido.
- La pantalla principal de Android.
- El sistema iniciado correctamente.
- El estado funcional del emulador.

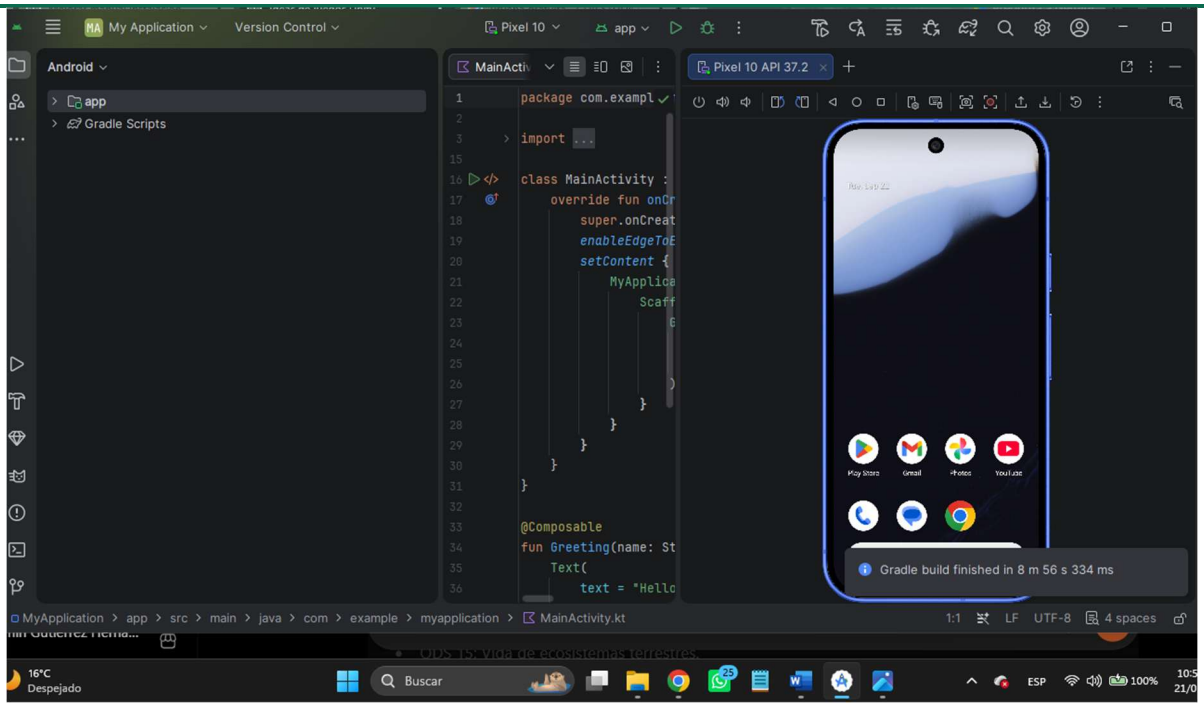


Figura 39. Emulador Android iniciado y funcionando correctamente.
Fuente: Elaboración propia.

INSTALACIÓN DE FLUTTER Y DART

Flutter será el framework utilizado para desarrollar la aplicación móvil y **Dart** será el lenguaje de programación utilizado por Flutter. Para esta instalación se utilizará el canal **Stable**, ya que está destinado a versiones estables del SDK.

El objetivo de este punto es instalar Flutter en Windows, agregarlo al **PATH** y comprobar que el sistema reconoce correctamente los comandos de Flutter y Dart.

Descargar Flutter

Paso 1. Abre **Google Chrome** desde el menú Inicio o desde el acceso directo del escritorio.

Paso 2. En la barra de direcciones escribe:

https://docs.flutter.dev/

y presiona **Enter**.

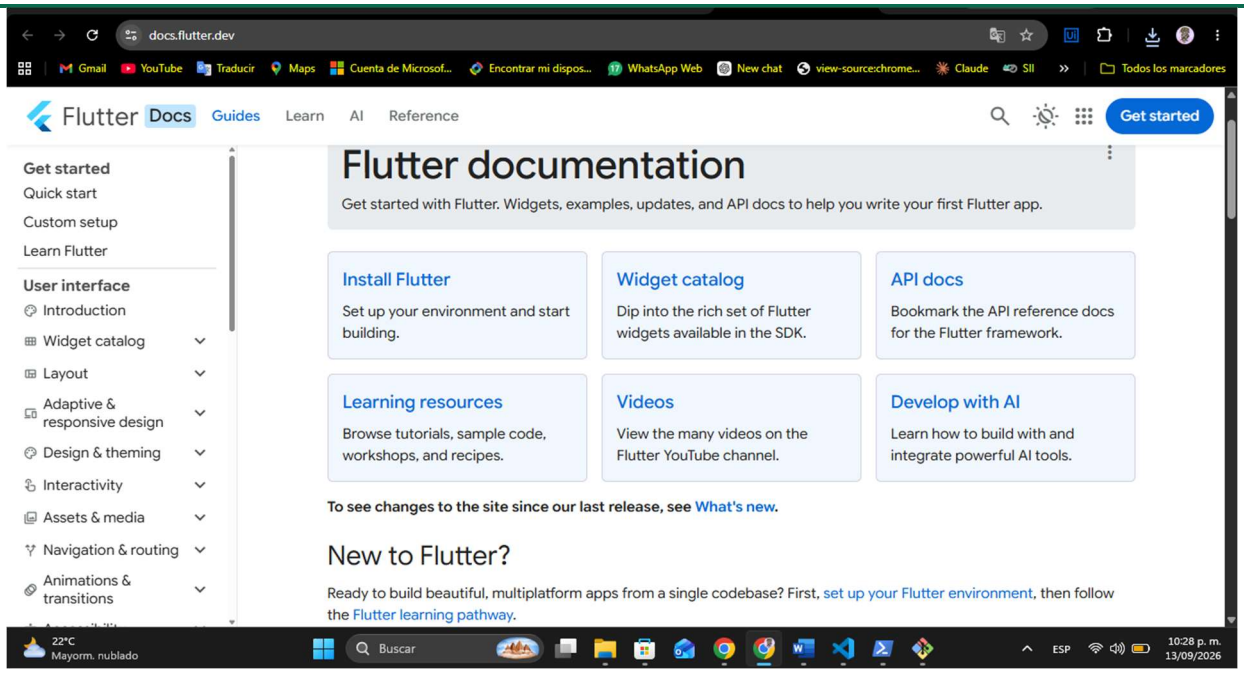


Figura 40. Documentación oficial de Flutter para iniciar la instalación.

Fuente: Elaboración propia.

Paso 3. Una vez dentro de la documentación oficial de Flutter, localiza la sección correspondiente a la instalación del SDK.

Paso 4. Selecciona **Windows** como sistema operativo.

Paso 5. Busca la opción correspondiente al canal **Stable**.

Paso 6. Selecciona la descarga del **Flutter SDK para Windows**.

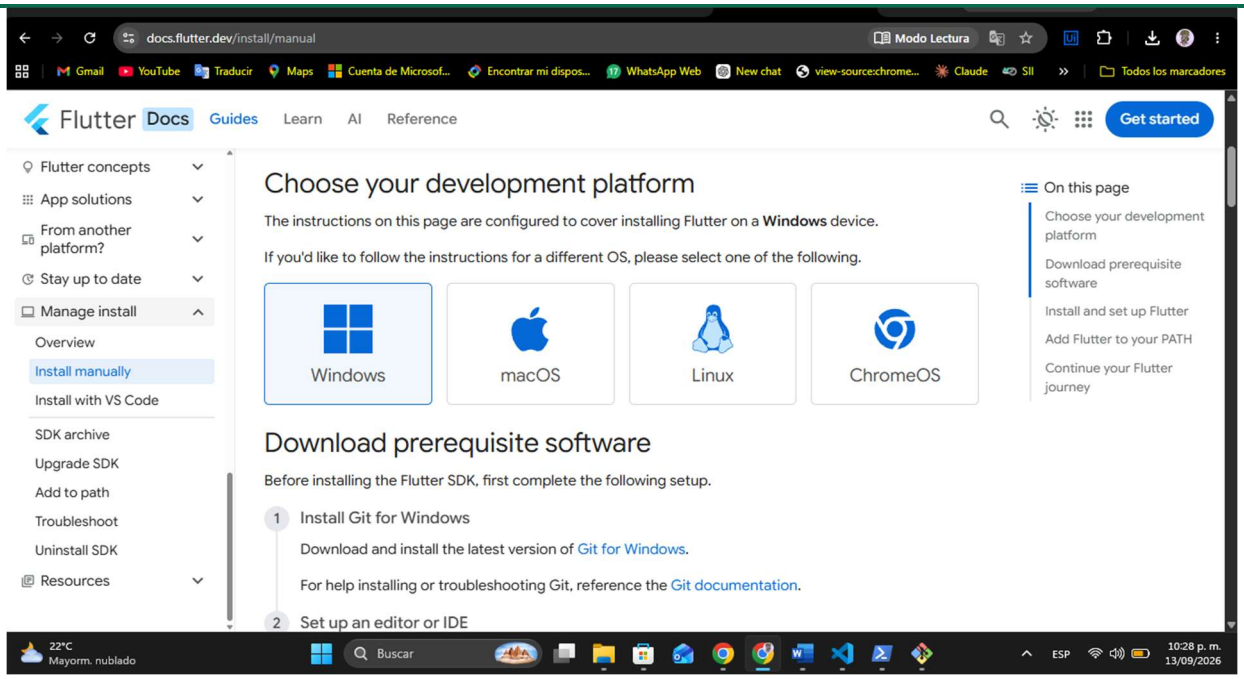


Figura 41. Selección de Windows como plataforma de instalación de Flutter.
Fuente: Elaboración propia.

Paso 7. Espera a que la descarga termine completamente. El archivo descargado normalmente se encontrará en la carpeta **Descargas** de Windows.

Paso 8. No abras ni ejecutes directamente el archivo ZIP. Primero se deberá extraer su contenido.

Resultado esperado: El archivo comprimido del Flutter SDK se encuentra descargado en el equipo.

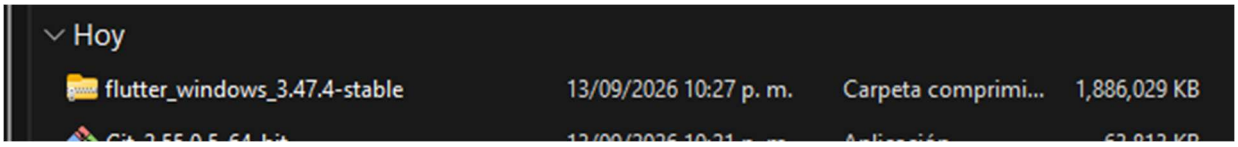


Figura 42. Descarga del SDK de Flutter para Windows.
Fuente: Elaboración propia.

Extraer el Flutter SDK

Paso 1. Abre el **Explorador de archivos**.

Paso 2. Entra en la carpeta **Descargas**.

Paso 3. Localiza el archivo ZIP descargado de Flutter.

Paso 4. Haz clic derecho sobre el archivo.

Paso 5. Selecciona **Extraer todo**.

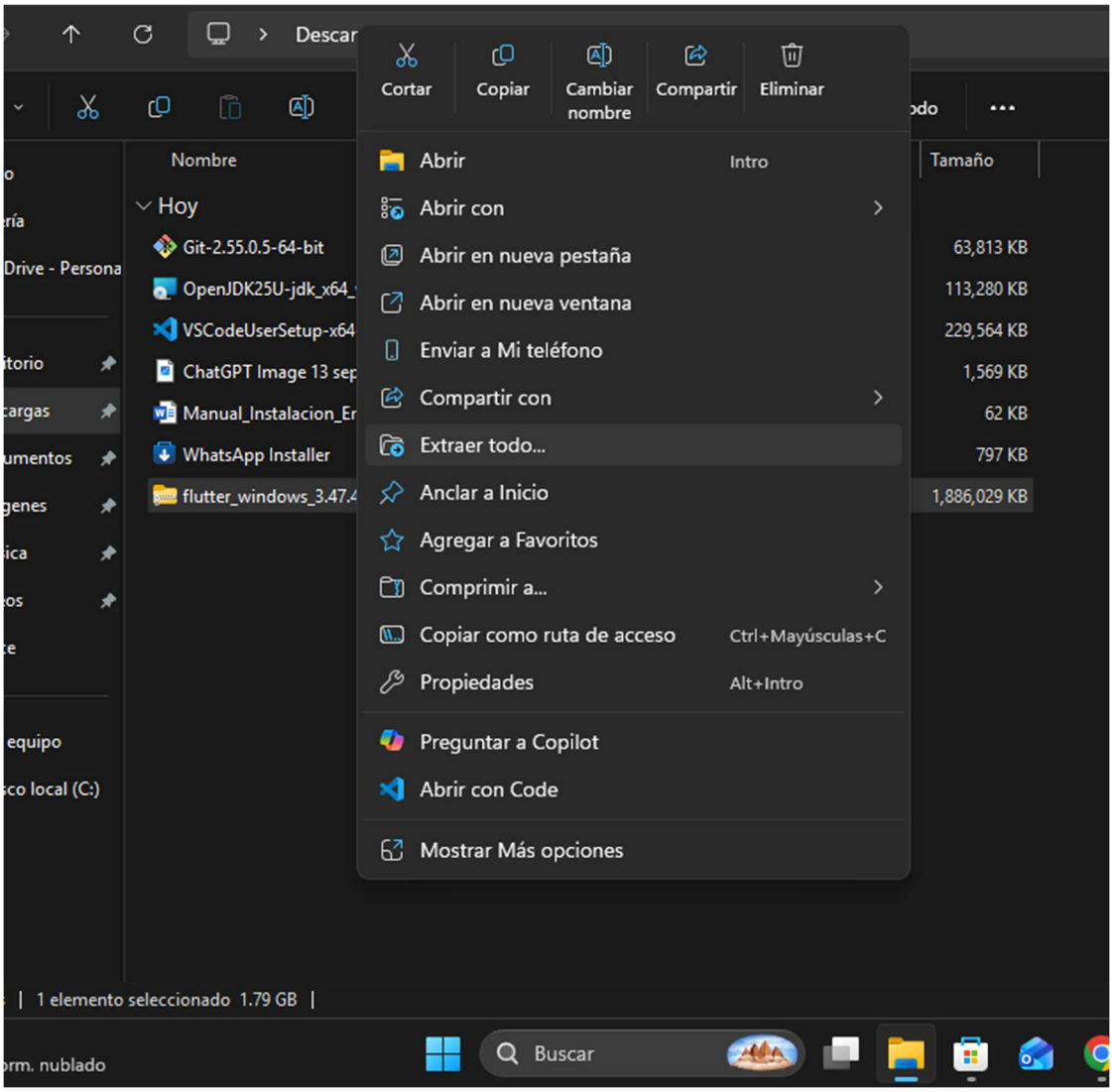


Figura 43. Archivo del SDK de Flutter descargado en Windows.
Fuente: Elaboración propia.

Paso 6. En la ventana que aparece, selecciona la ubicación donde se extraerá Flutter.

Para mantener una ruta sencilla, se recomienda utilizar:

```
C:\src
```

Paso 7. Presiona **Extraer** y espera a que finalice el proceso.

Paso 8. Abre la carpeta:

```
C:\src
```

Paso 9. Comprueba que exista la carpeta:

```
flutter
```

Paso 10. Entra en la carpeta:

```
C:\src\flutter
```

Paso 11. Comprueba que dentro de ella exista la carpeta:

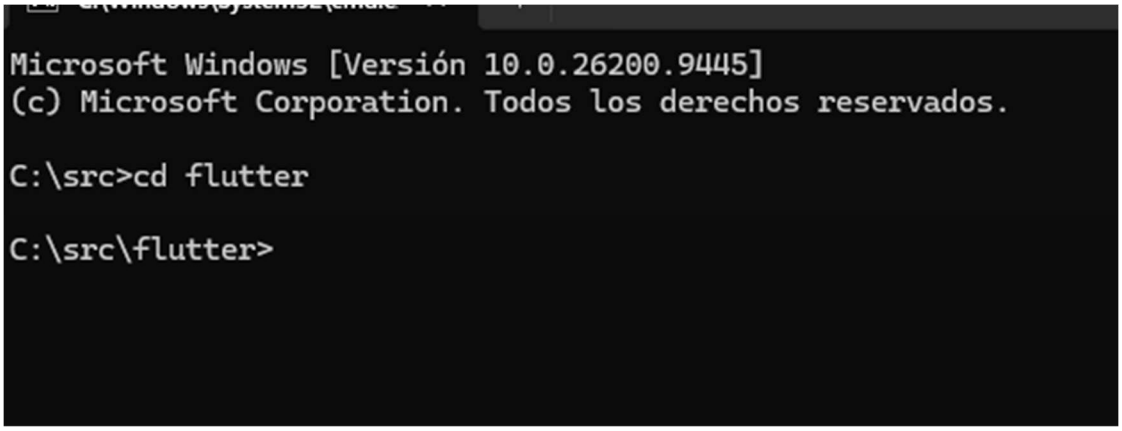


Figura 44. Carpeta del SDK de Flutter extraída en el equipo.
Fuente: Elaboración propia.

```
bin
```

La ruta completa debe ser:

```
C:\src\flutter\bin
```

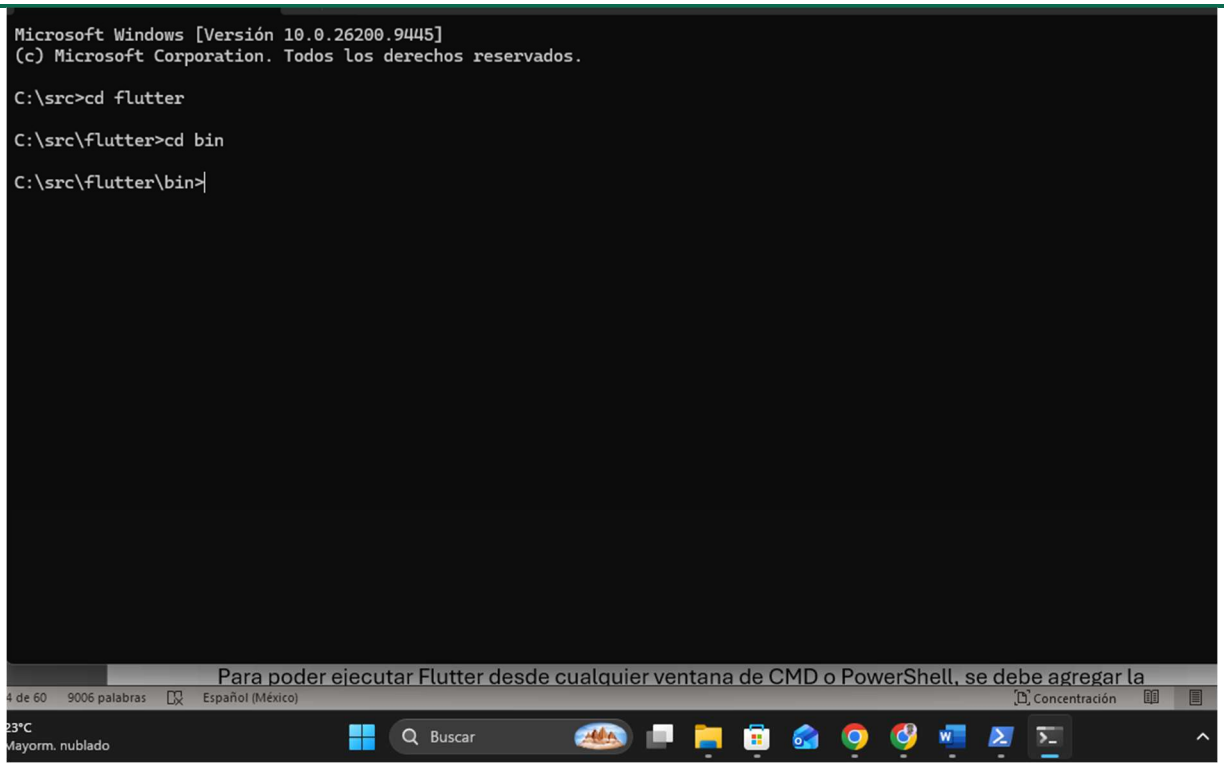


Figura 45. Verificación de la estructura del SDK de Flutter.
Fuente: Elaboración propia.

Resultado esperado: Flutter queda extraído correctamente y la carpeta `bin` existe dentro del directorio de Flutter.

Importante: Evita colocar Flutter dentro de una carpeta que posteriormente pueda ser eliminada, movida o comprimida. La ruta debe mantenerse sin cambios después de configurar el PATH.

Agregar Flutter al PATH de Windows

Para poder ejecutar Flutter desde cualquier ventana de CMD o PowerShell, se debe agregar la carpeta `bin` al PATH de Windows.

Paso 1. Presiona:

Windows + R

Paso 2. Escribe:

`sysdm.cpl`

Paso 3. Presiona **Enter**.

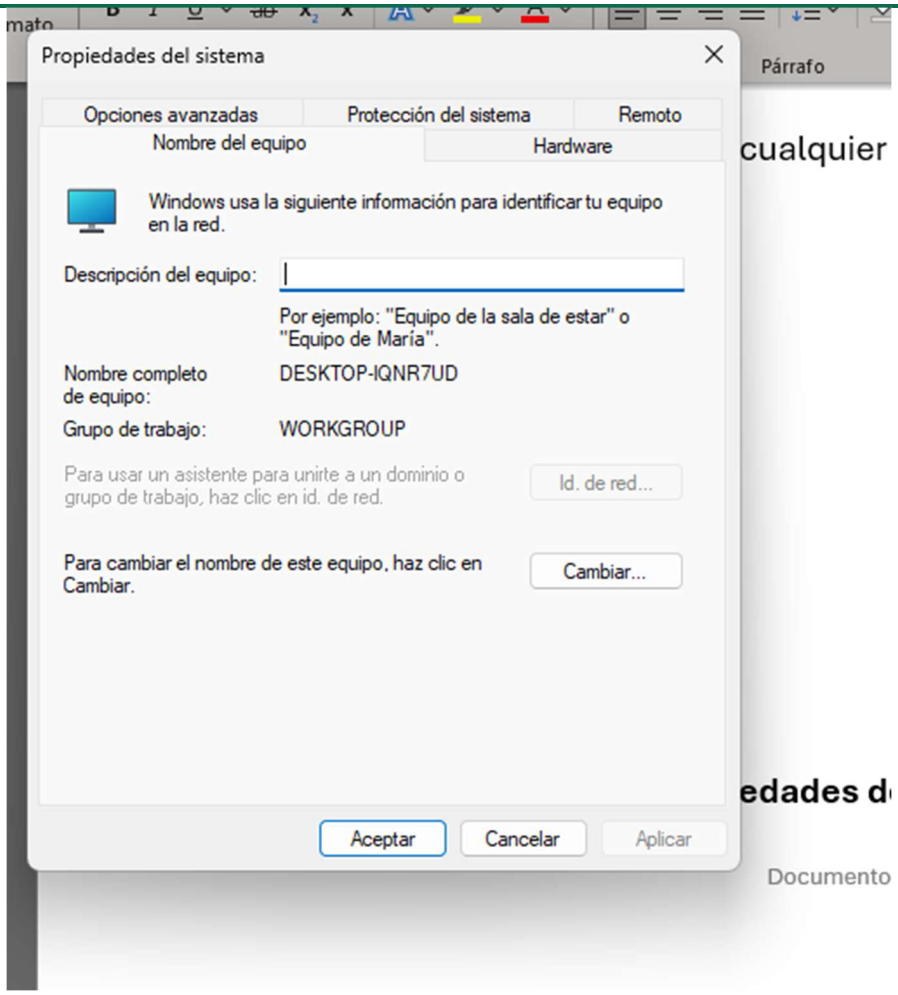


Figura 46. Acceso a la configuración de variables de entorno de Windows.

Fuente: Elaboración propia.

Paso 4. Se abrirá la ventana **Propiedades del sistema**.

Paso 5. Selecciona la pestaña **Opciones avanzadas**.

Paso 6. Presiona el botón **Variables de entorno**.

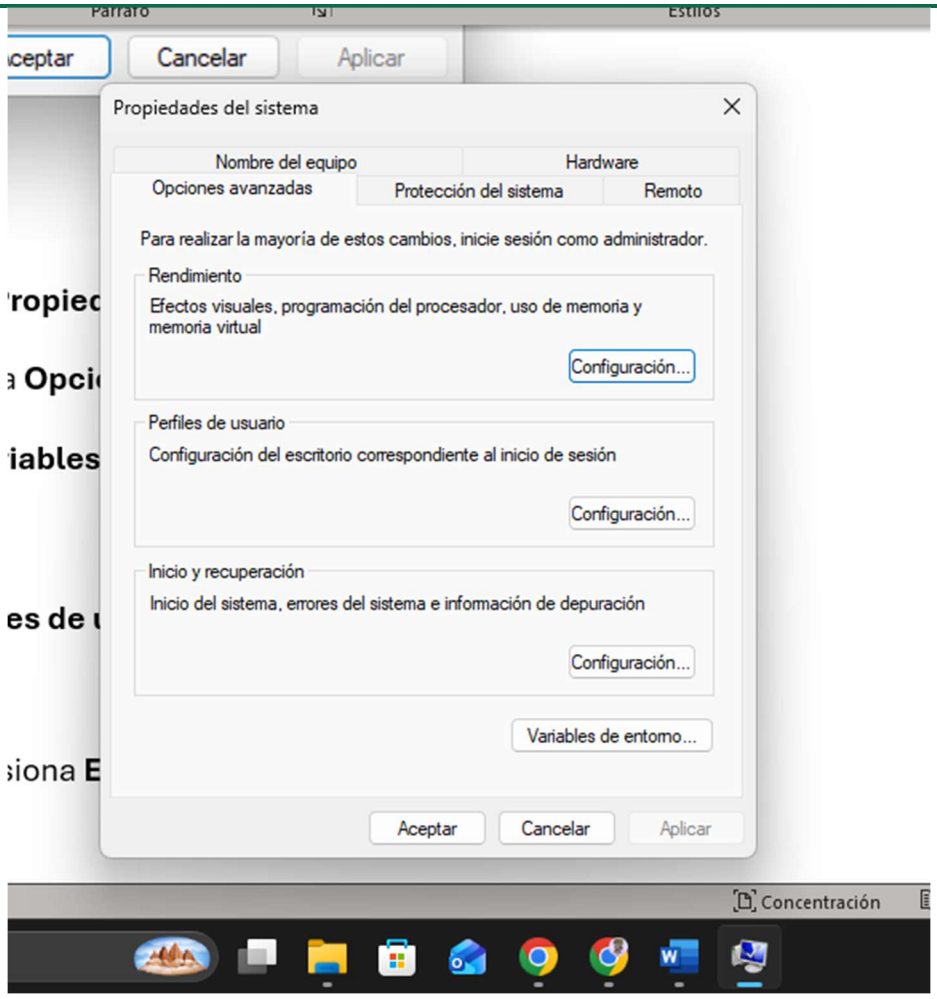


Figura 47. Edición de las variables de entorno del sistema.

Fuente: Elaboración propia.

Paso 7. En la sección **Variables de usuario para [tu usuario]**, localiza la variable:

`Path`

Paso 8. Selecciona `Path` y presiona **Editar**.

Paso 9. Presiona **Nuevo**.

Paso 10. Escribe exactamente:

`C:\src\flutter\bin`

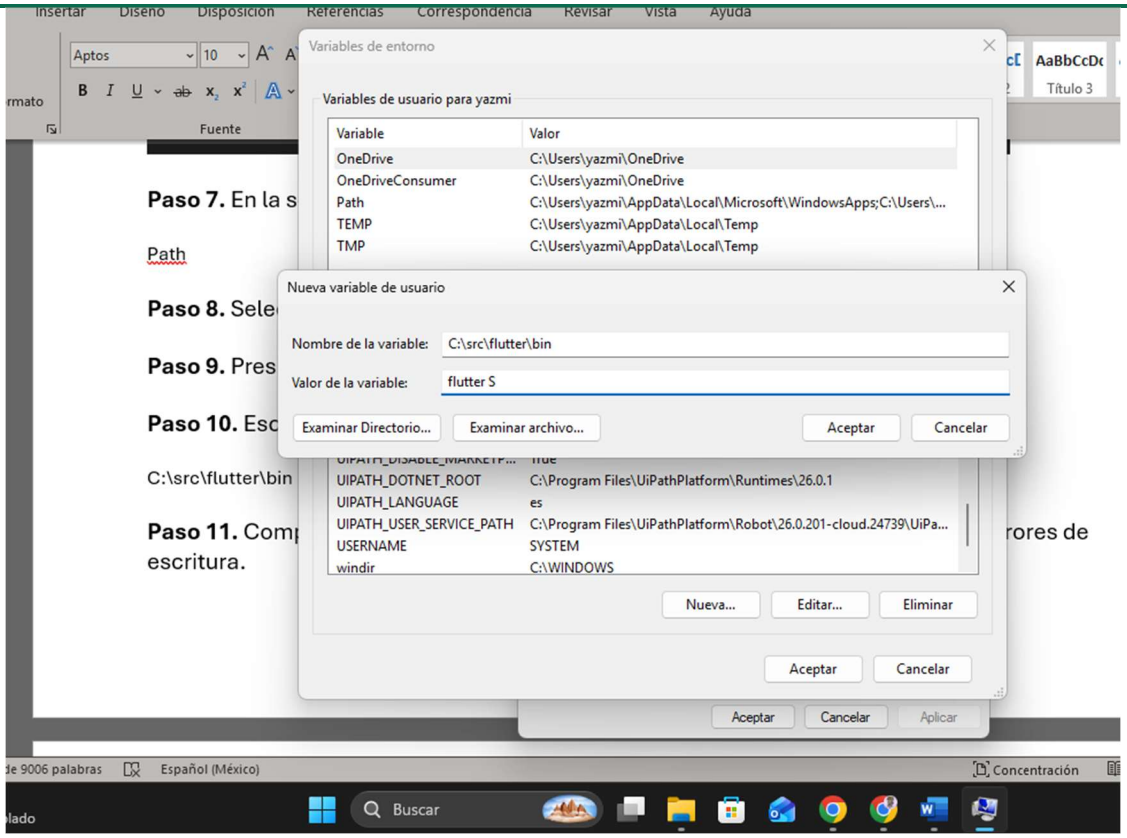


Figura 48. Configuración del PATH para Flutter.

Fuente: Elaboración propia.

Paso 11. Comprueba que la ruta se haya agregado correctamente y que no tenga errores de escritura.

Paso 12. Presiona **Aceptar**.

Paso 13. Presiona nuevamente **Aceptar** en las ventanas restantes hasta cerrar la configuración.

Resultado esperado: La ruta `C:\src\flutter\bin` queda agregada a la variable `Path` de Windows.

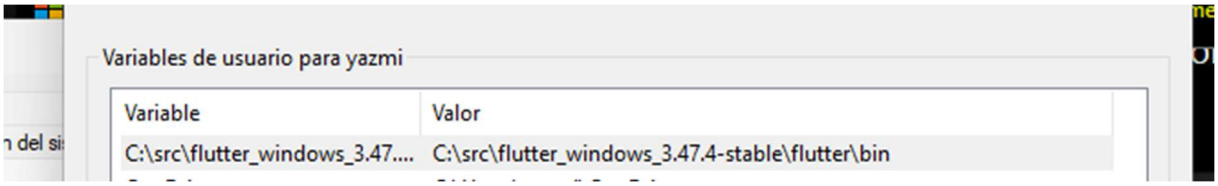


Figura 49. Ruta de Flutter agregada al PATH de Windows.

Fuente: Elaboración propia.

Comprobar que el PATH se actualizó

Para comprobar los cambios, es importante abrir una consola nueva.

Paso 1. Cierra las ventanas de **CMD** o **PowerShell** que estuvieran abiertas antes de modificar el PATH.

Paso 2. Presiona:

Windows + R

Paso 3. Escribe:

```
cmd
```

Paso 4. Presiona **Enter**.

Paso 5. En la nueva ventana ejecuta:

```
flutter --version
```

Paso 6. Presiona **Enter**.

Paso 7. Espera unos segundos mientras Flutter muestra la información de la instalación.

Resultado esperado: El comando `flutter` es reconocido y muestra información relacionada con Flutter y Dart.

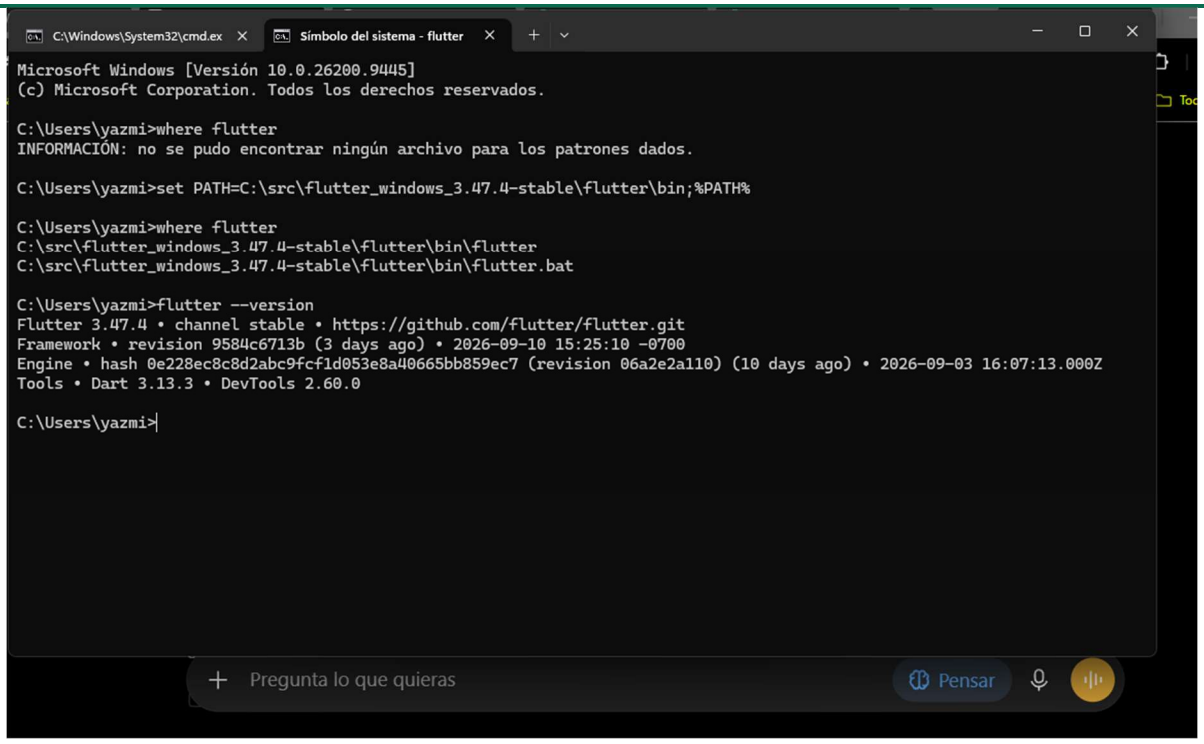


Figura 50. Comprobación de la configuración de Flutter desde la consola.
Fuente: Elaboración propia.

Verificar la versión de Flutter

Después de ejecutar `flutter --version`, revisa la información que aparece en la consola.

Debe mostrarse información como:

- Versión instalada de Flutter.
- Canal utilizado.
- Versión de Dart.
- Información adicional del SDK.

Comprueba especialmente que el canal sea:

`estable`

La presencia de esta información confirma que Windows puede localizar el ejecutable de Flutter mediante el PATH.

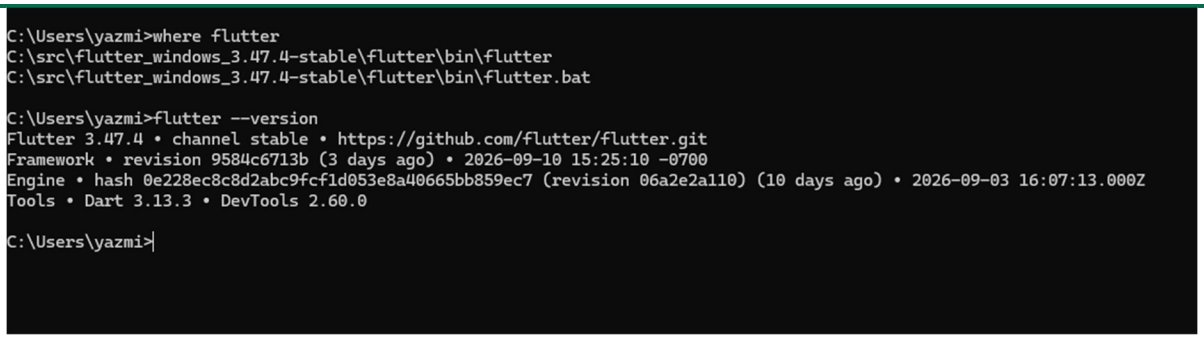


Figura 51. Verificación de la versión de Flutter desde la consola.
Fuente: Elaboración propia.

Comprobar Dart

Como Dart forma parte del entorno de Flutter, también se puede comprobar desde la consola.

Paso 1. En la misma ventana de CMD ejecuta:

```
dart --version
```

Paso 2. Presiona **Enter**.

Resultado esperado: La consola muestra la versión instalada de Dart.

Esta comprobación permite confirmar que el entorno de Flutter incluye y reconoce correctamente Dart.

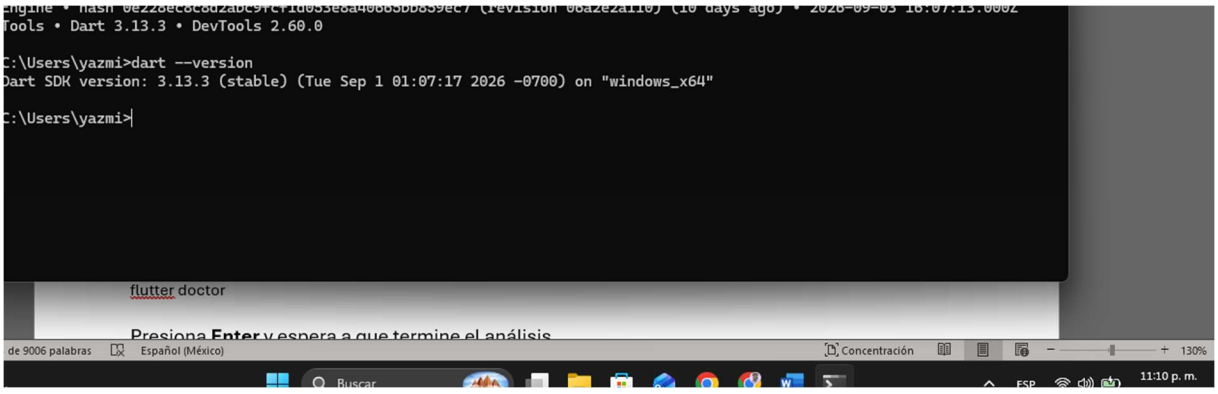


Figura 52. Verificación de Dart desde la consola.
Fuente: Elaboración propia.

Comprobar Flutter con Flutter Doctor

Como comprobación adicional, ejecuta:

```
flutter doctor
```

Presiona **Enter** y espera a que termine el análisis.

Flutter Doctor revisará diferentes componentes relacionados con el entorno de desarrollo y mostrará su estado.

Resultado esperado: Aparece un informe de diagnóstico de Flutter con los componentes detectados y cualquier elemento que requiera configuración adicional.

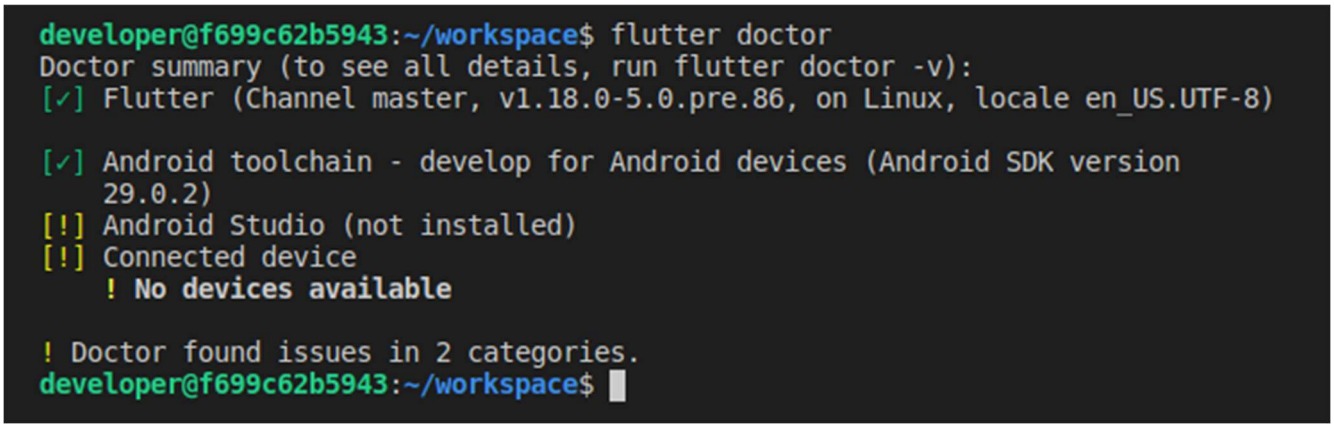


Figura 53. Diagnóstico del entorno mediante Flutter Doctor.
Fuente: Elaboración propia.

Nota: Si Flutter Doctor muestra advertencias o elementos pendientes, no significa necesariamente que la instalación de Flutter haya fallado. Las acciones necesarias dependerán del componente señalado.

Flutter instalado

Momento: Después de ejecutar `flutter --version` y comprobar que la información aparece correctamente.

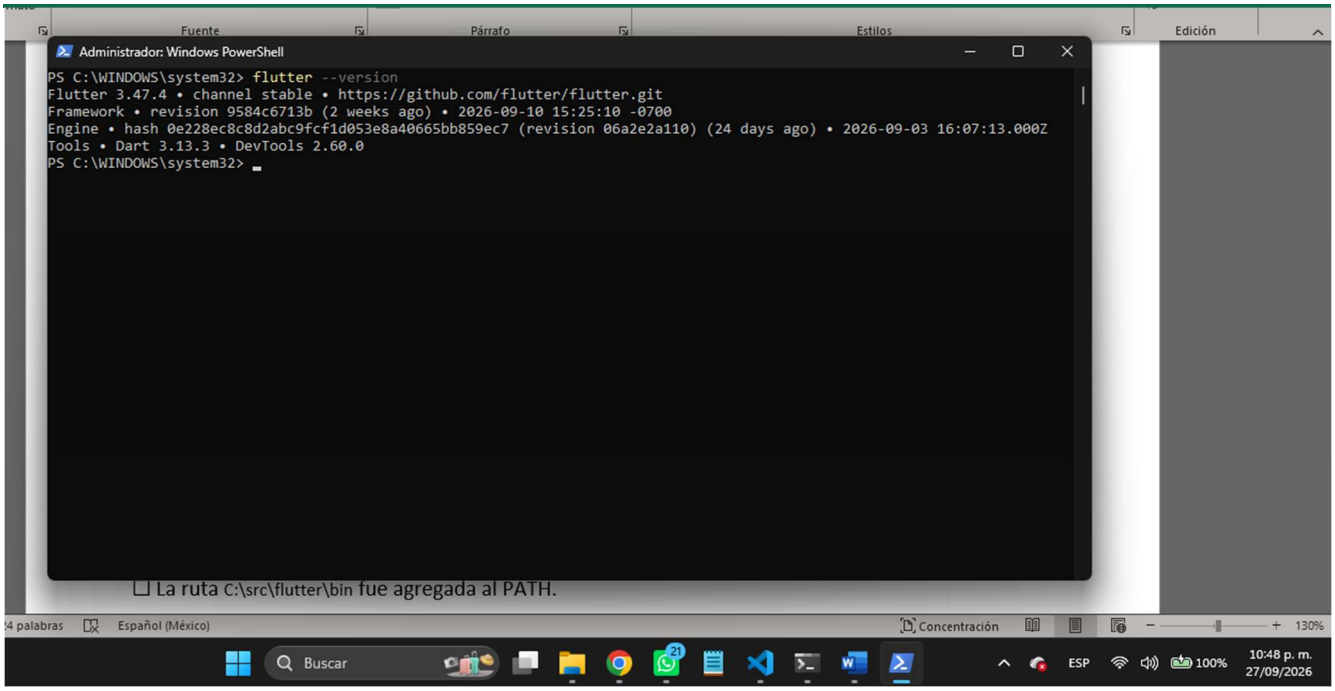


Figura 54. Evidencia de Flutter instalado y reconocido en la terminal.

Fuente: Elaboración propia.

Comprobación final

Antes de continuar con el siguiente punto, verifica lo siguiente:

- ✓ ☐ El Flutter SDK fue descargado desde la documentación oficial.
- ✓ ☐ Flutter fue extraído correctamente.
- ✓ ☐ Existe la carpeta `C:\src\flutter`.
- ✓ ☐ Existe la carpeta `C:\src\flutter\bin`.
- ✓ ☐ La ruta `C:\src\flutter\bin` fue agregada al PATH.
- ✓ ☐ Se abrió una consola nueva después de modificar el PATH.
- ✓ ☐ El comando `flutter --version` funciona correctamente.
- ✓ ☐ El canal mostrado es `stable`.
- ✓ ☐ Se muestra la versión de Dart.
- ✓ ☐ El comando `dart --version` funciona correctamente.
- ✓ ☐ Se ejecutó `flutter doctor`.

Resultado final esperado: Flutter está instalado en Windows, el sistema reconoce los comandos `flutter` y `dart`, y el SDK se encuentra configurado para continuar con la preparación del entorno de desarrollo.

DIAGNÓSTICO DEL ENTORNO FLUTTER

Comprobaciones complementarias

flutter doctor --android-licenses
flutter devices

DÓNDE EJECUTARLO: CMD, PowerShell o terminal de Android Studio.

RESULTADO ESPERADO: Las licencias Android quedan revisadas y Flutter muestra el emulador o dispositivo disponible.

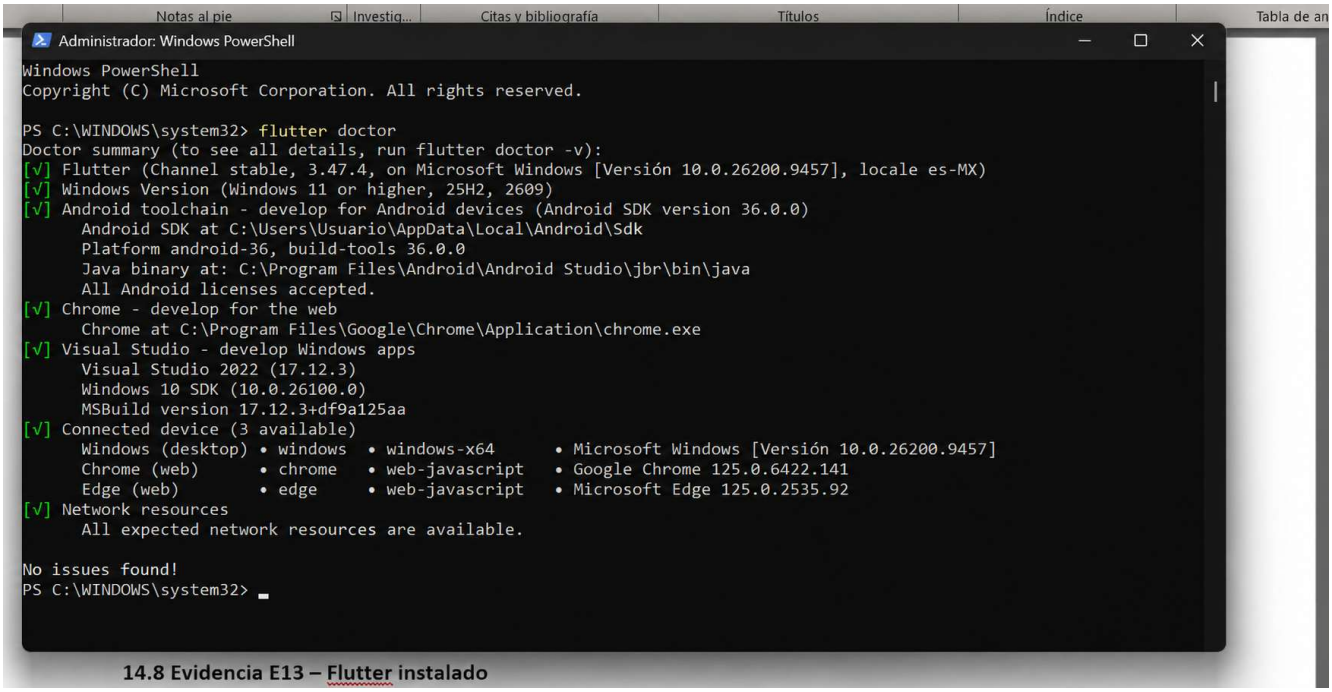


Figura 55. Revisión de licencias Android y dispositivos disponibles en Flutter.
Fuente: Elaboración propia.

VALIDACIÓN DE LA APLICACIÓN FLUTTER EXISTENTE

La aplicación Flutter ya existe. Antes de incorporar Docker se debe comprobar que el proyecto real funciona sin modificaciones, de manera que cualquier problema posterior pueda diferenciarse de un problema previo del código Flutter.

Abrir y revisar el proyecto existente

Localizar la carpeta real del proyecto Flutter.

Abrir la carpeta del proyecto en Visual Studio Code. Mantener Android Studio disponible para iniciar y administrar el emulador Android.

Comprobar la presencia de android/, lib/, test/, pubspec.yaml y pubspec.lock.

lib/ contiene el código Dart principal; android/ contiene la configuración de la plataforma Android; pubspec.yaml declara metadatos y dependencias del proyecto.

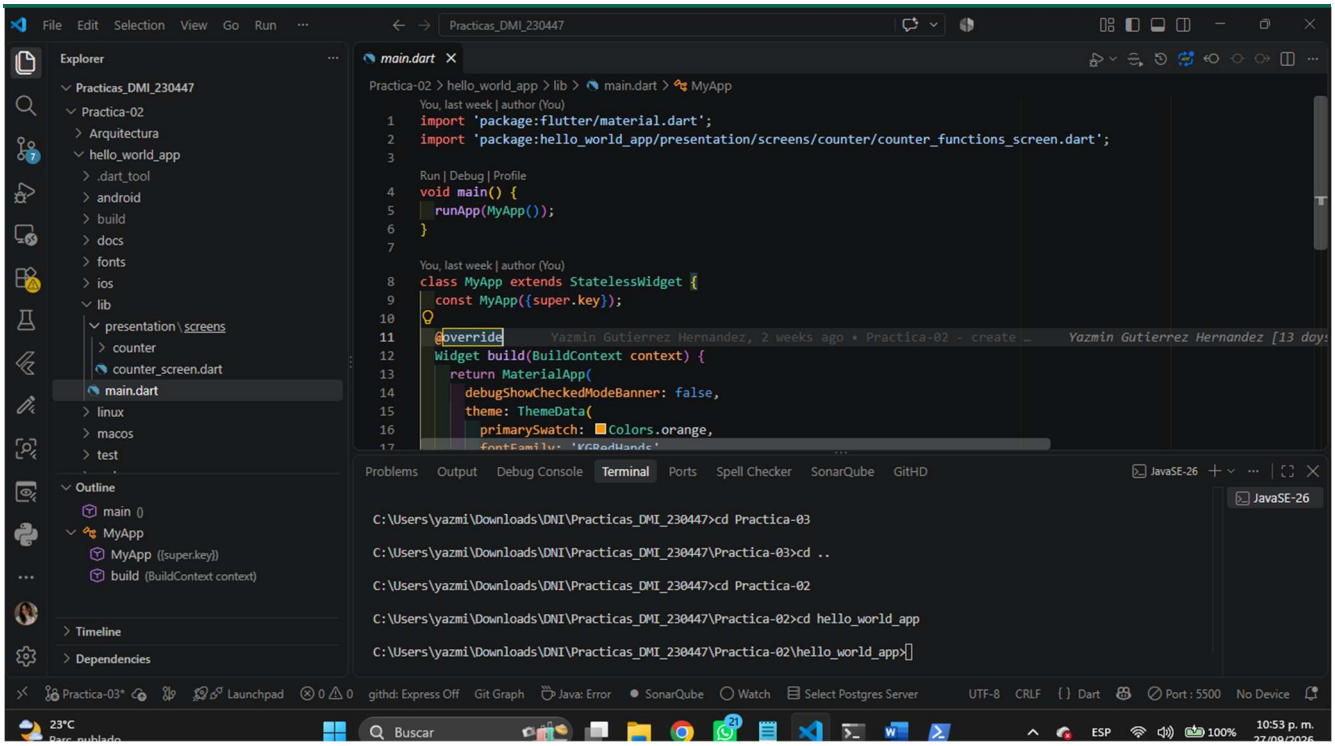


Figura 56. Revisión de la estructura del proyecto Flutter en Visual Studio Code.

Fuente: Elaboración propia.

Resolver dependencias

flutter pub get

DÓNDE EJECUTARLO: En la terminal integrada de Visual Studio Code, ubicada en la raíz del proyecto Flutter.

RESULTADO ESPERADO: Flutter resuelve correctamente las dependencias declaradas en pubspec.yaml.

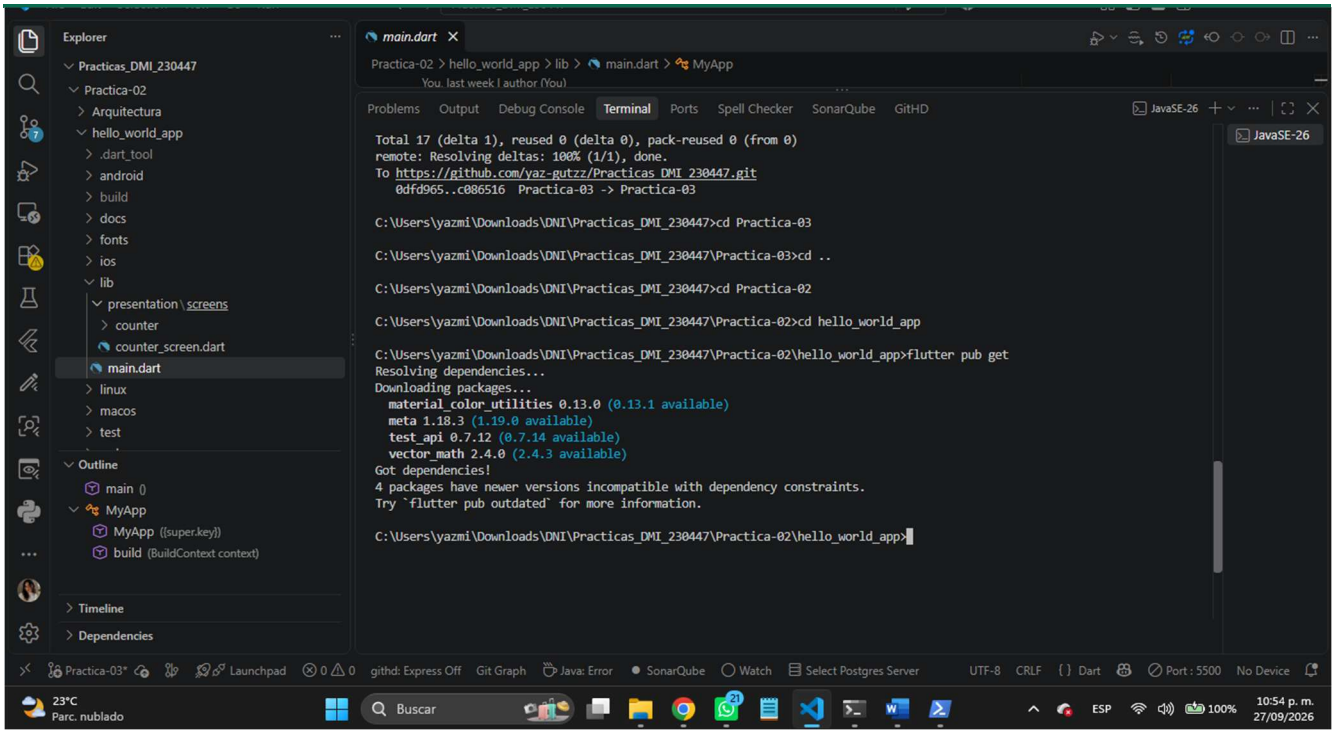


Figura 57. Resolución de dependencias del proyecto con flutter pub get.

Fuente: Elaboración propia.

Detectar el dispositivo Android

flutter devices

RESULTADO ESPERADO: Aparece el emulador iniciado desde Android Studio o el dispositivo Android autorizado.

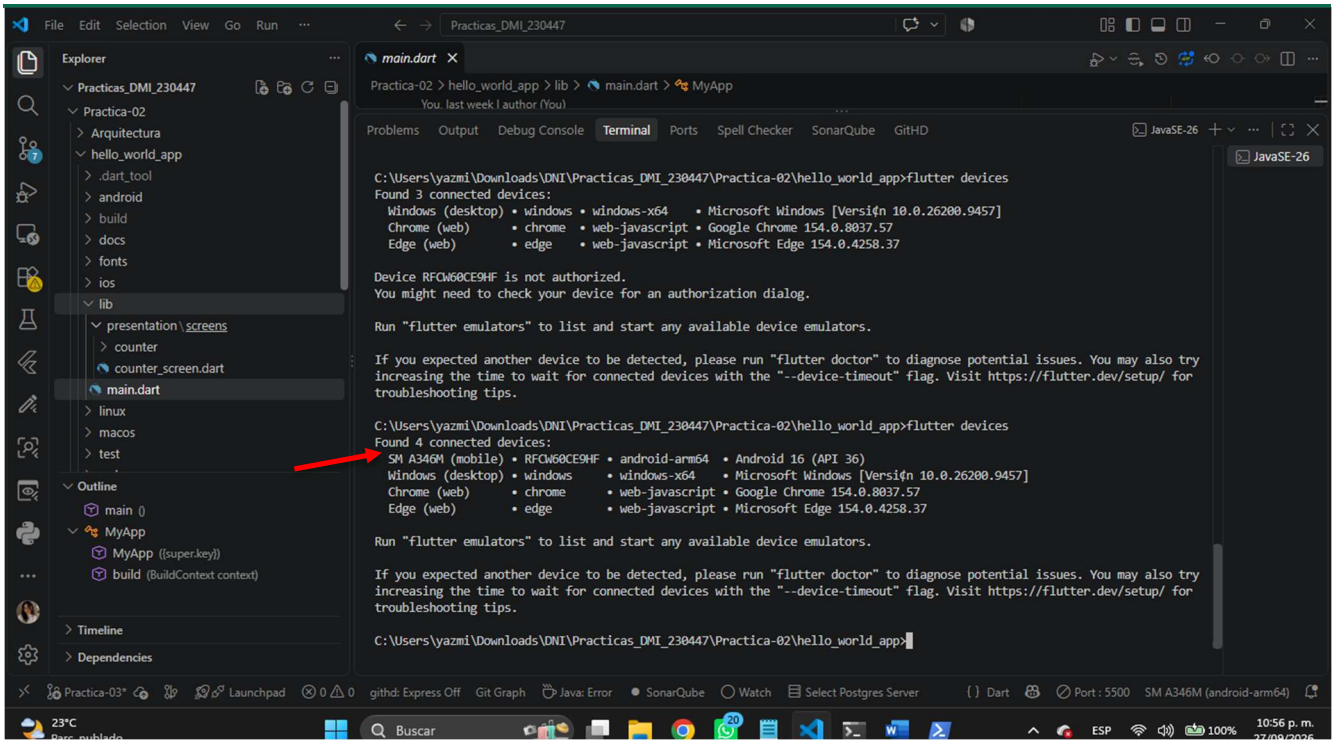


Figura 58. Detección del dispositivo Android desde Flutter.

Fuente: Elaboración propia.

Ejecutar la aplicación real

flutter run

RESULTADO ESPERADO: La aplicación Flutter existente se ejecuta correctamente en Android.

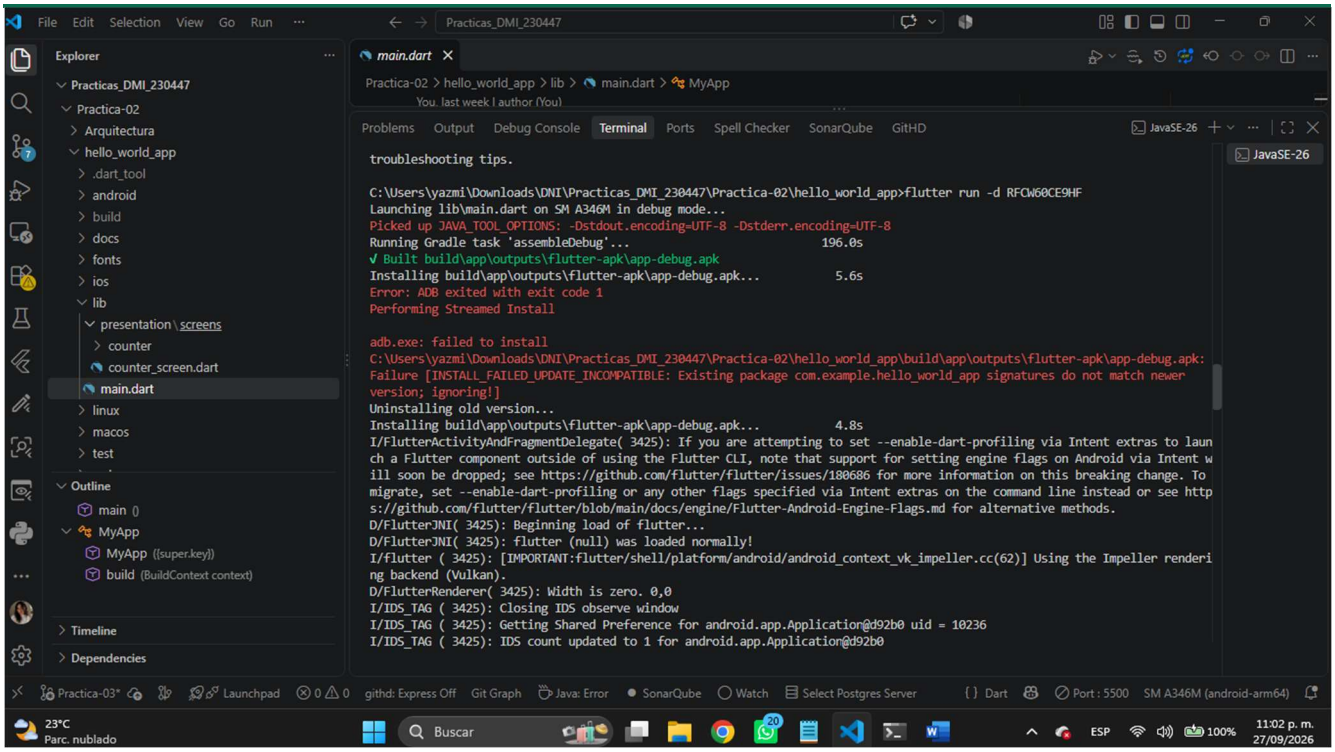


Figura 59. Ejecución de la aplicación Flutter en el dispositivo Android.

Fuente: Elaboración propia.

Evidencia E15 – Aplicación Flutter existente funcionando

MOMENTO: Después de ejecutar flutter run sobre el proyecto real.

ACCIÓN: Capturar la aplicación ejecutándose en el emulador o dispositivo Android.

PANTALLA: Android Emulator o dispositivo físico.

LA CAPTURA DEBE MOSTRAR: La interfaz real de la aplicación desarrollada.

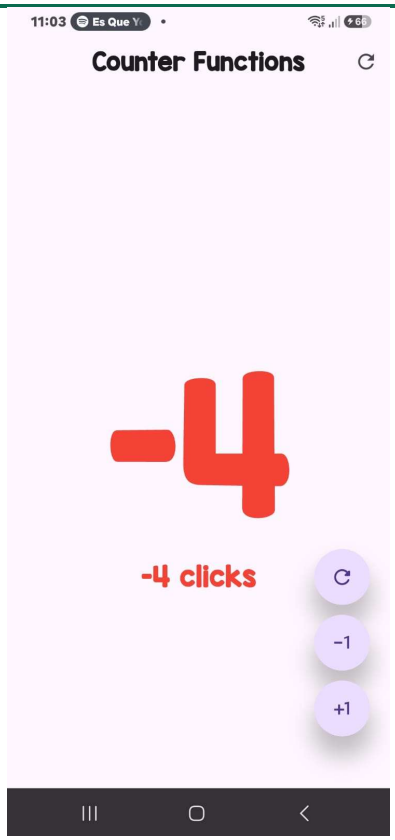


Figura 60. Aplicación Flutter existente funcionando en Android.
Fuente: Elaboración propia.

INSTALACIÓN Y CONFIGURACIÓN DE DOCKER DESKTOP

Docker Desktop proporciona las herramientas necesarias para trabajar con contenedores en Windows mediante **Docker Engine** y **Docker Compose**. En sistemas Windows puede utilizar **WSL2** como backend.

En esta sección únicamente se realizará la instalación y verificación básica de Docker Desktop. La dockerización de la aplicación se realizará posteriormente.

Descargar e instalar Docker Desktop

1. Acceder al sitio oficial de Docker:

   `https://www.docker.com/products/docker-desktop/`

2. Descargar **Docker Desktop para Windows**.
3. Ejecutar el instalador.
4. Mantener habilitada la opción para utilizar **WSL2** cuando corresponda.
5. Completar la instalación con las opciones recomendadas.
6. Abrir **Docker Desktop**.

7. Esperar a que Docker Engine termine de iniciar.

RESULTADO ESPERADO: Docker Desktop debe abrir correctamente y mostrar que Docker Engine se encuentra disponible.

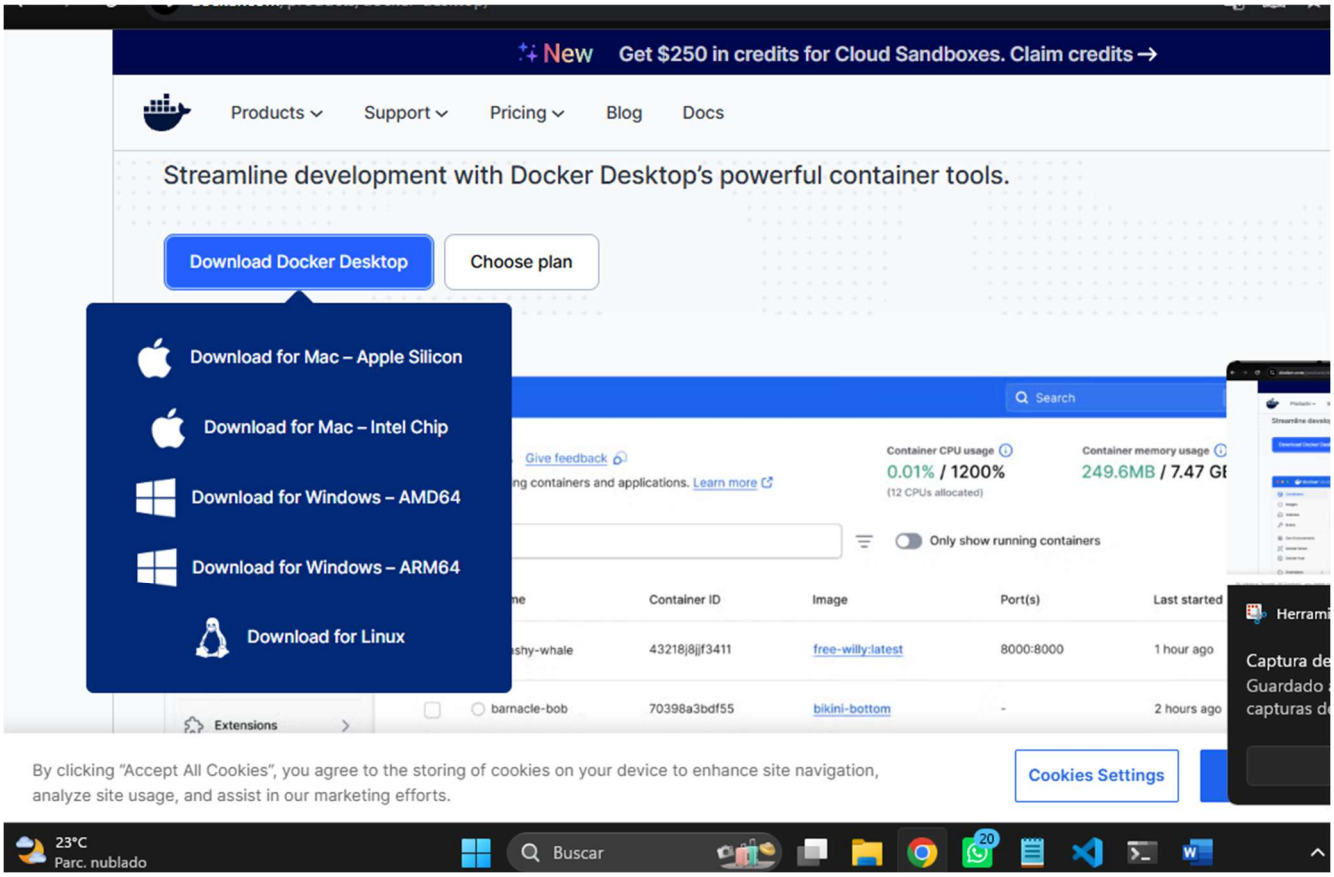


Figura 61. Descarga de Docker Desktop para Windows desde el sitio oficial.

Fuente: Elaboración propia.

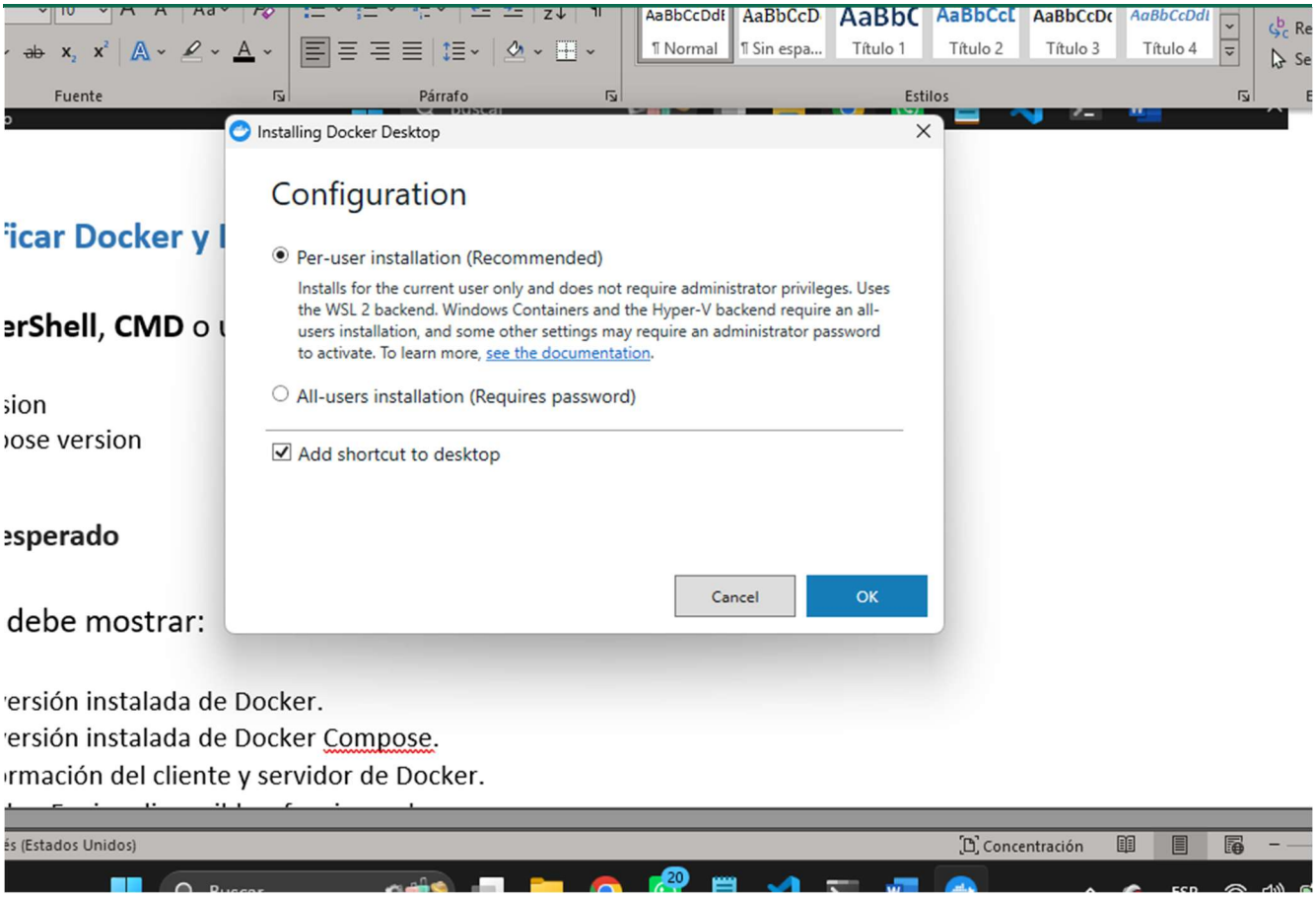


Figura 62. Configuración de instalación de Docker Desktop en Windows.

Fuente: Elaboración propia.

Verificar Docker y Docker Compose

Abrir **PowerShell**, **CMD** o una terminal compatible y ejecutar:

```
docker --version
docker compose version
docker info
```

Resultado esperado

El sistema debe mostrar:

- La versión instalada de Docker.
- La versión instalada de Docker Compose.
- Información del cliente y servidor de Docker.
- Docker Engine disponible y funcionando.

Ejemplo:

Si `docker info` muestra información tanto de **Client** como de **Server**, significa que Docker Engine se está ejecutando correctamente.

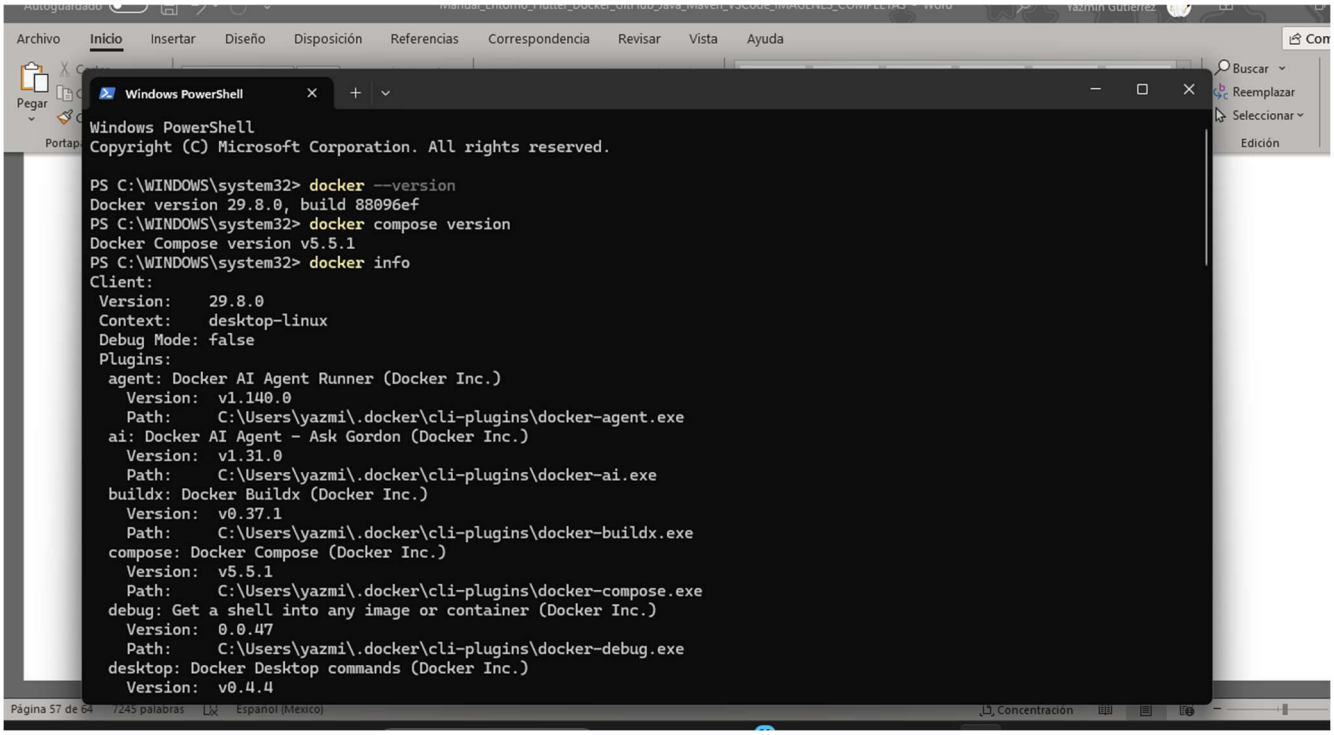


Figura 63. Verificación de Docker Engine y Docker Compose desde la terminal.
Fuente: Elaboración propia.

Docker Desktop funcionando

MOMENTO: Después de instalar Docker Desktop, iniciar Docker Engine y ejecutar los comandos de verificación.

ACCIÓN: Capturar la terminal mostrando los resultados de:

```
docker --version
docker compose version
docker info
```

También puede incluirse Docker Desktop abierto si se desea mostrar visualmente que el motor se encuentra activo.

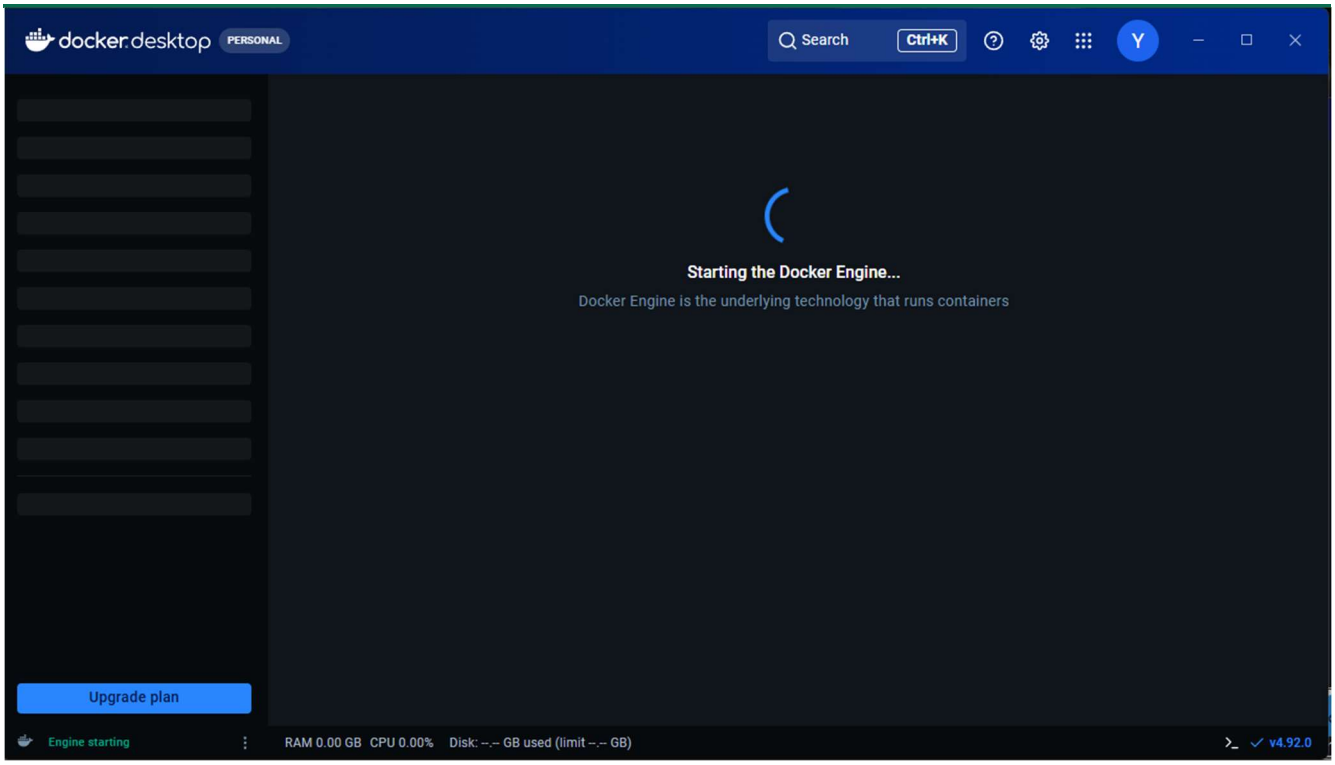


Figura 64. Docker Desktop iniciando Docker Engine.

Fuente: Elaboración propia.

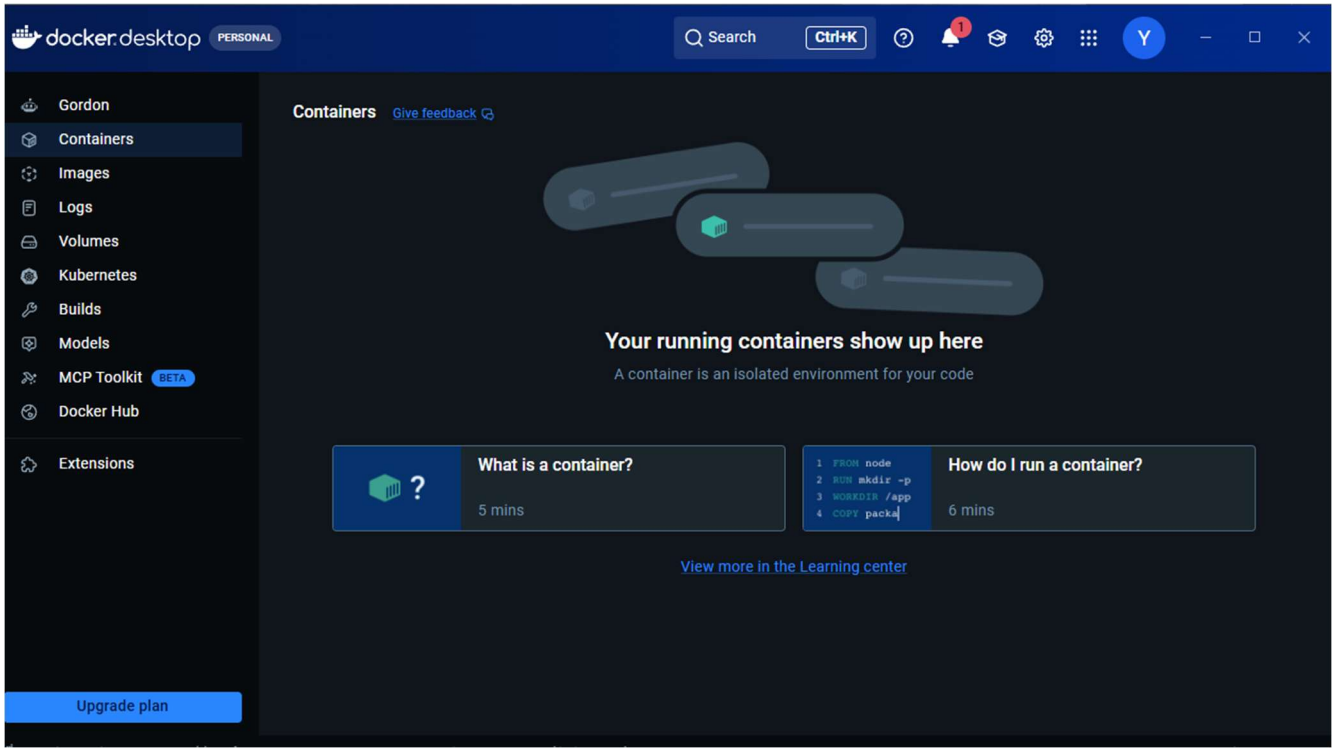


Figura 65. Docker Desktop abierto con el motor disponible.

Fuente: Elaboración propia.

DOCKERIZACIÓN DEL PROYECTO FLUTTER EXISTENTE

En este apartado se configurará Docker para trabajar con el proyecto Flutter existente de forma reproducible.

La aplicación móvil continuará ejecutándose mediante Flutter, Android Studio y Android Emulator. Docker se utilizará únicamente para ejecutar tareas relacionadas con el proyecto que puedan realizarse dentro de un contenedor, como la instalación de dependencias, análisis del código, ejecución de pruebas o procesos de compilación compatibles.

Importante: No se debe afirmar que Android Emulator o la aplicación Android se ejecutan dentro del contenedor Docker si esto no ocurre realmente. Tampoco deben agregarse APIs, bases de datos, servidores o microservicios que no formen parte del proyecto.

Revisar el proyecto Flutter

Abrir la carpeta raíz del proyecto en Visual Studio Code:

```
Practica-03/yes_no_app/
```

Verificar que se encuentre el archivo:

```
pubspec.yaml
```

Abrir una terminal en la raíz del proyecto y ejecutar:

```
flutter pub get
flutter analyze
flutter test
```

Resultado esperado: Flutter reconoce correctamente el proyecto y sus dependencias.

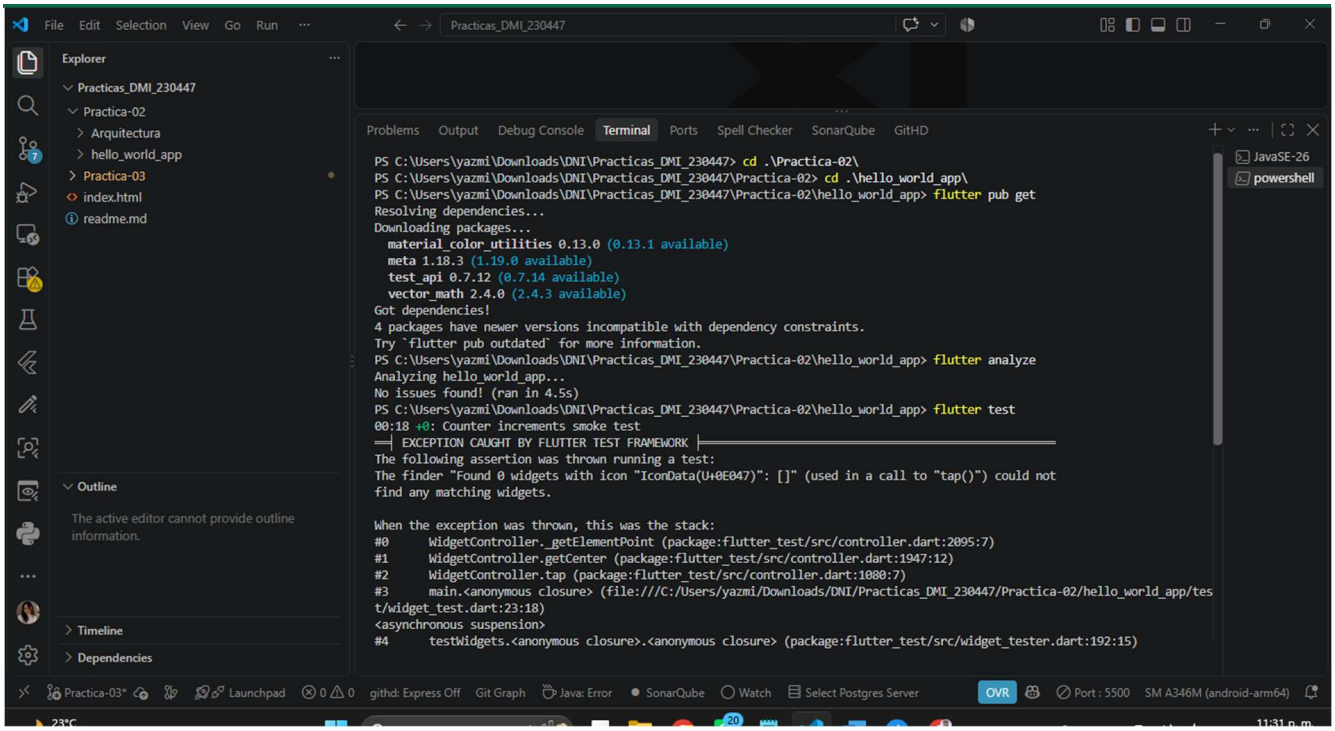


Figura 66. Revisión del proyecto Flutter existente antes de la dockerización.

Fuente: Elaboración propia.

Verificar Docker

Abrir Docker Desktop y esperar a que Docker Engine esté activo.

Ejecutar:

```
docker --version
docker compose version
docker info
```

Resultado esperado: Docker y Docker Compose responden correctamente.

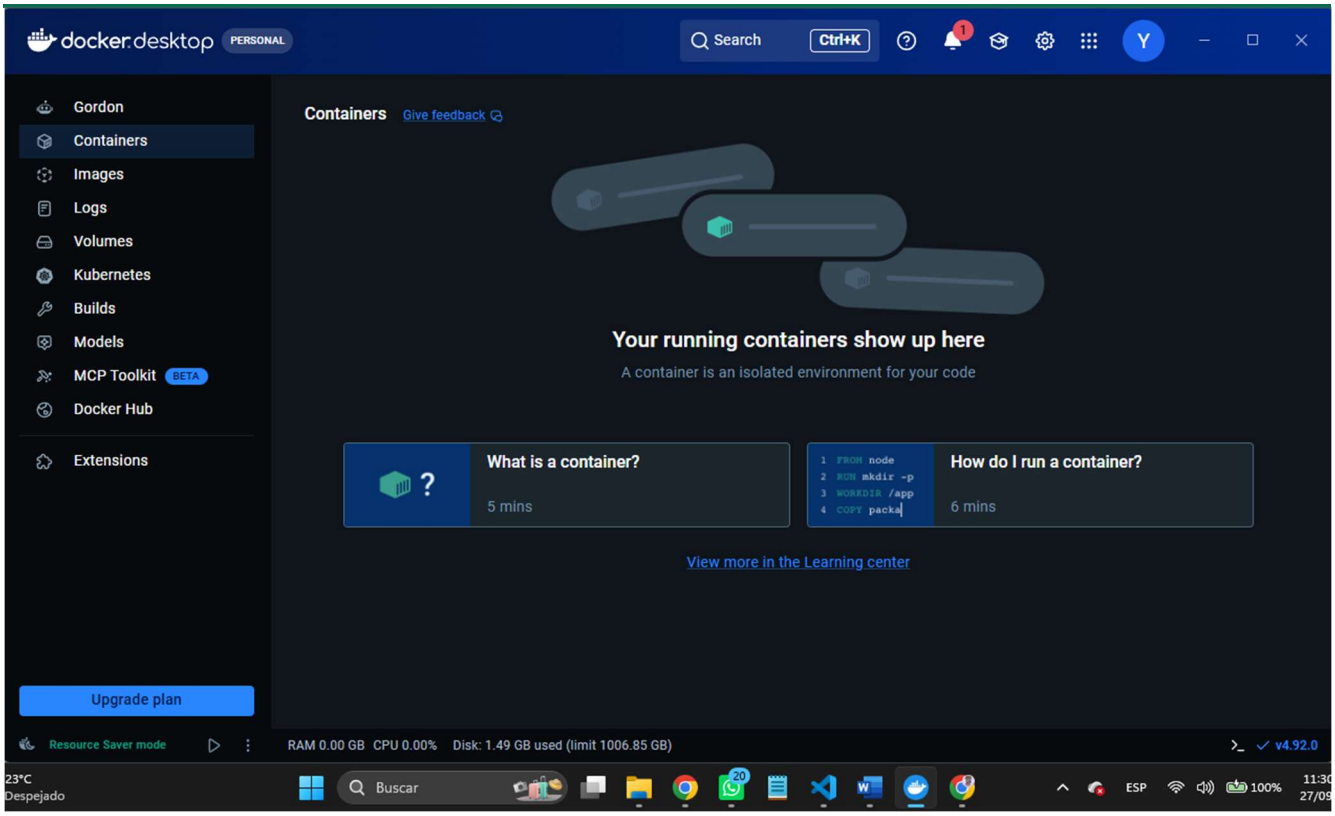


Figura 67. Verificación de Docker y Docker Compose antes de construir la imagen.
Fuente: Elaboración propia.

Crear el Dockerfile

En la raíz del proyecto crear un archivo llamado:

```
Dockerfile
```

La estructura deberá quedar similar a:

```
yes_no_app/
├── lib/
├── test/
├── pubspec.yaml
├── Dockerfile
└── .dockerignore
```

Consultar primero la versión de Flutter:

```
flutter --version
```

El Dockerfile deberá utilizar una imagen compatible con la versión real del proyecto.

Ejemplo de estructura:

```
FROM IMAGEN_FLUTTER_COMPATIBLE

WORKDIR /app

COPY pubspec.yaml pubspec.lock ./

RUN flutter pub get

COPY . .

RUN flutter analyze

CMD ["flutter", "test"]
```

Resultado esperado: El Dockerfile contiene únicamente las instrucciones necesarias para trabajar con el proyecto Flutter.

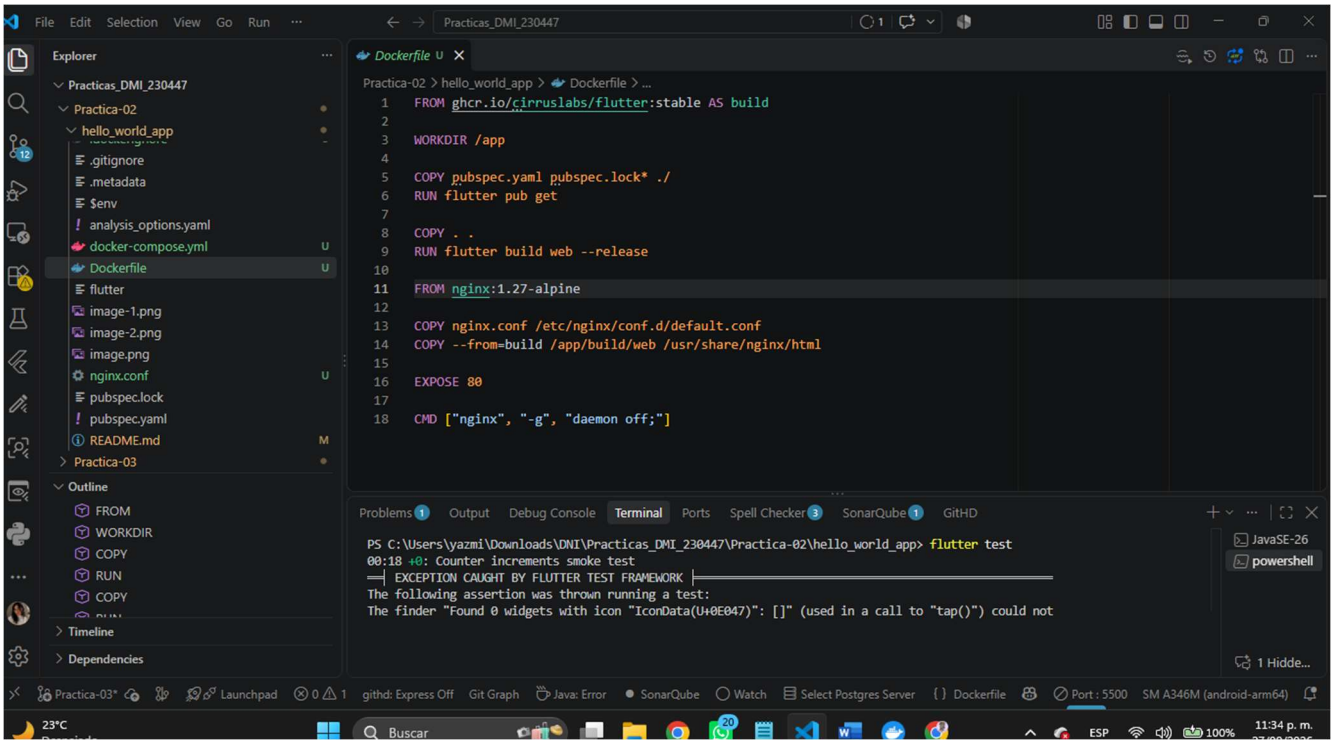


Figura 68. Dockerfile creado para el proyecto Flutter.

Fuente: Elaboración propia.

Crear .dockerignore

En la misma carpeta crear:

```
.dockerignore
```

Agregar como referencia:

```
.git
.gitignore
.idea
.vscode
build
.dart_tool
*.log
```

Resultado esperado: Docker excluirá archivos temporales o innecesarios durante la construcción de la imagen.

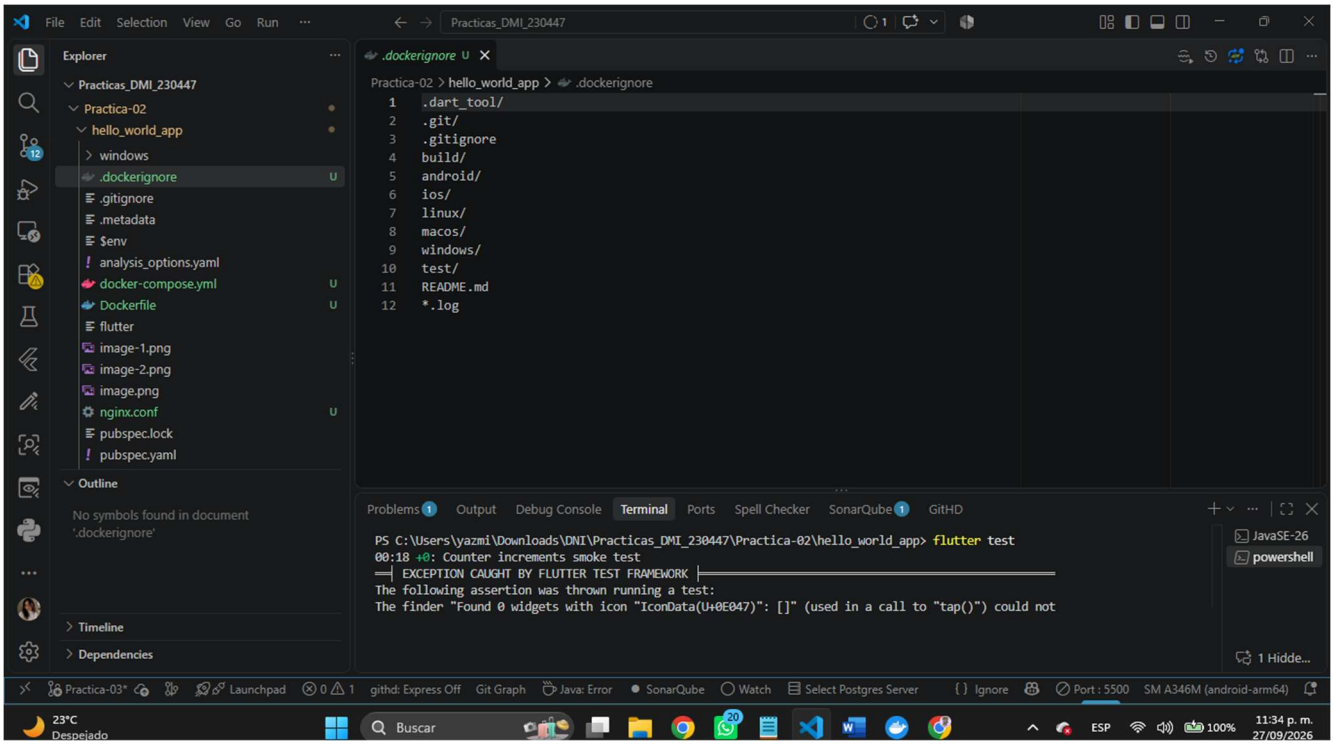


Figura 69. Archivo .dockerignore configurado para el proyecto Flutter.
Fuente: Elaboración propia.

Crear compose.yaml si aplica

Si el proyecto requiere Docker Compose, crear:

```
compose.yaml
```

Ejemplo básico:

```
services:
  flutter:
```

```
  build:
    context: .
    dockerfile: Dockerfile
```

No agregar puertos, bases de datos, servicios o variables que el proyecto no utilice.

Si Docker solamente ejecutará pruebas o análisis de Flutter, Compose puede omitirse.

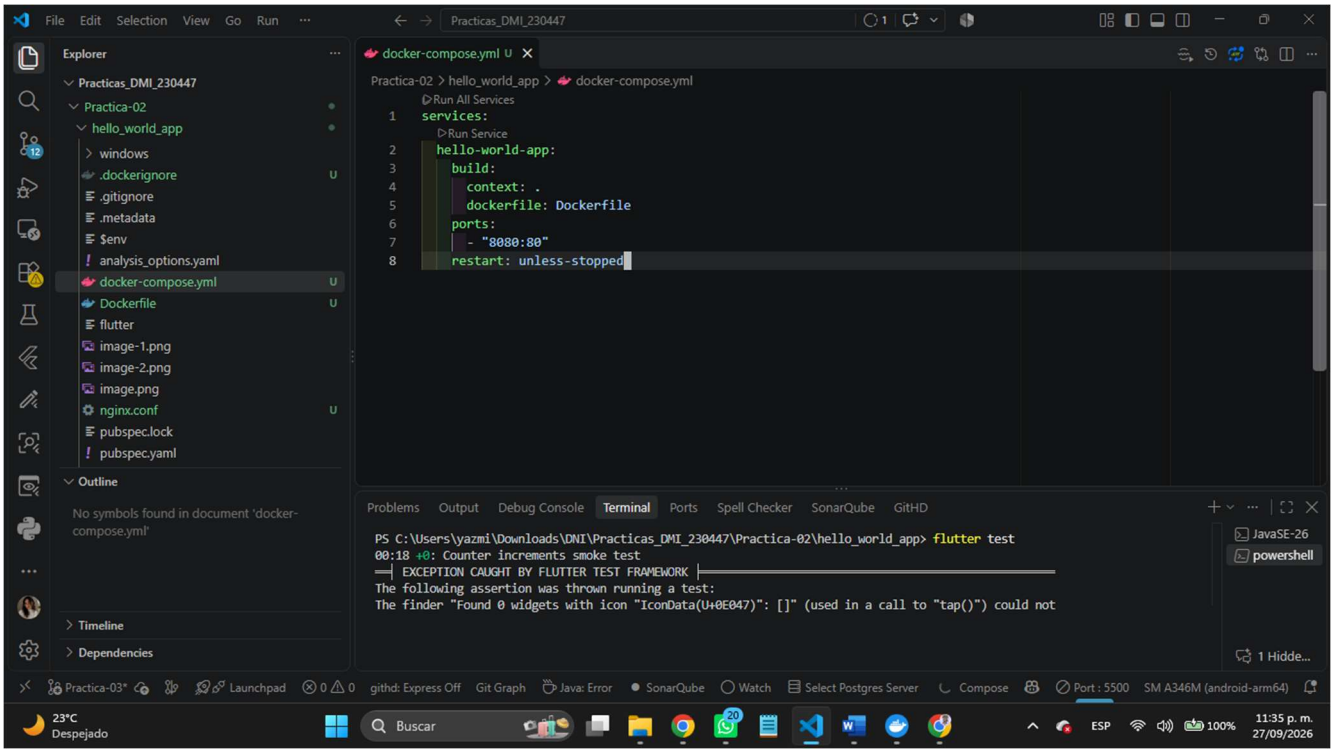


Figura 70. Archivo compose.yaml configurado para el proyecto Flutter.

Fuente: Elaboración propia.

Construir la imagen Docker

Desde la raíz del proyecto ejecutar:

```
docker build -t yes-no-app-flutter .
```

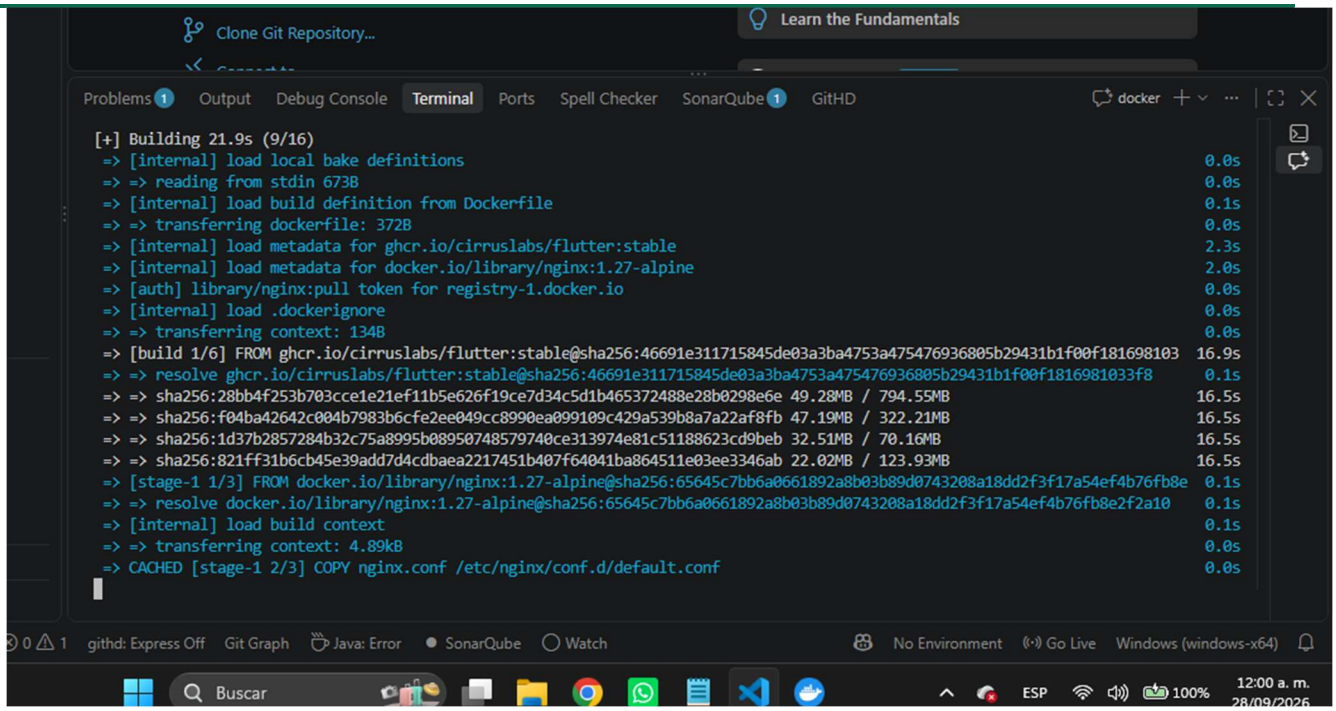


Figura 71. Construcción de la imagen Docker del proyecto Flutter.

Fuente: Elaboración propia.

Si se utiliza Compose:

```
docker compose build
```

Después verificar la imagen:

```
docker images
```

Resultado esperado: La imagen `yes-no-app-flutter` aparece entre las imágenes disponibles.

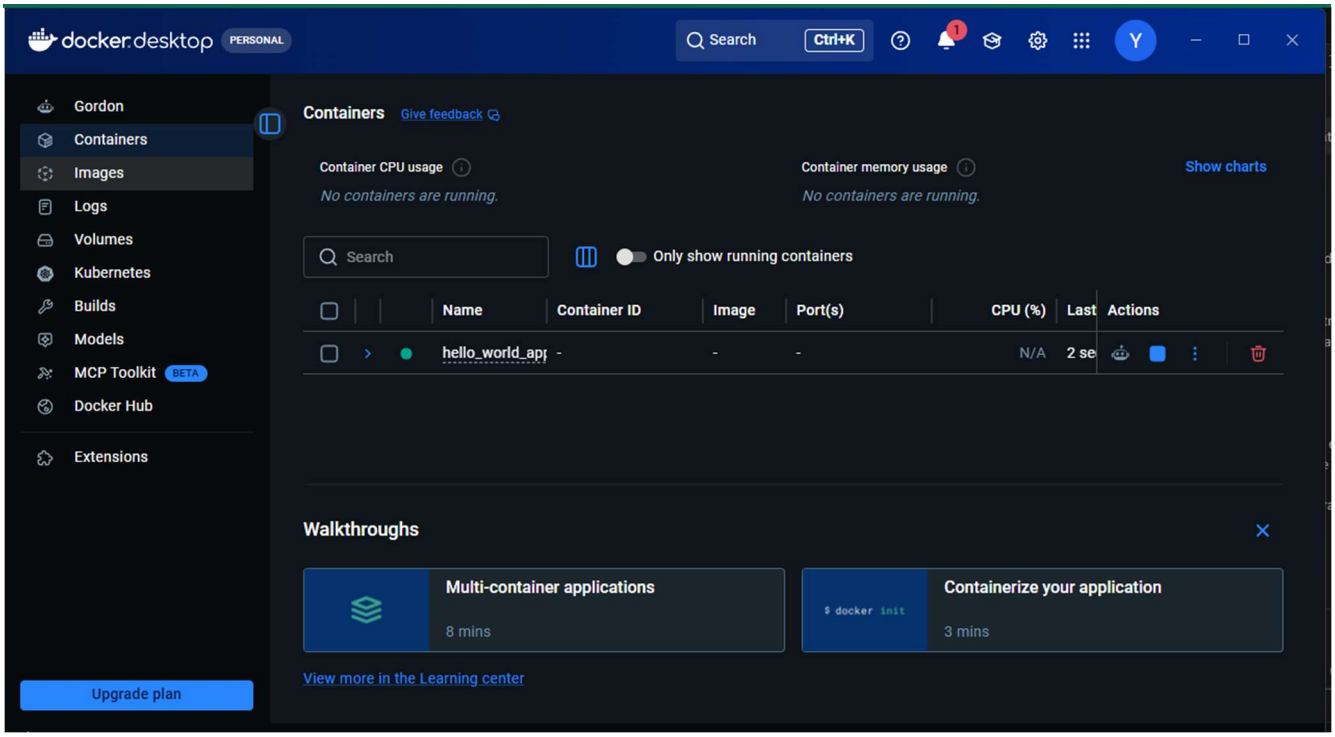


Figura 72. Imagen Docker del proyecto disponible en Docker Desktop.
Fuente: Elaboración propia.

Ejecutar y verificar el contenedor

Ejecutar:

```
docker run --name yes-no-app-container yes-no-app-flutter
```

Verificar los contenedores:

```
docker ps -a
```

Consultar los logs:

```
docker logs yes-no-app-container
```

Si se utiliza Compose:

```
docker compose up
docker compose ps
docker compose logs
```

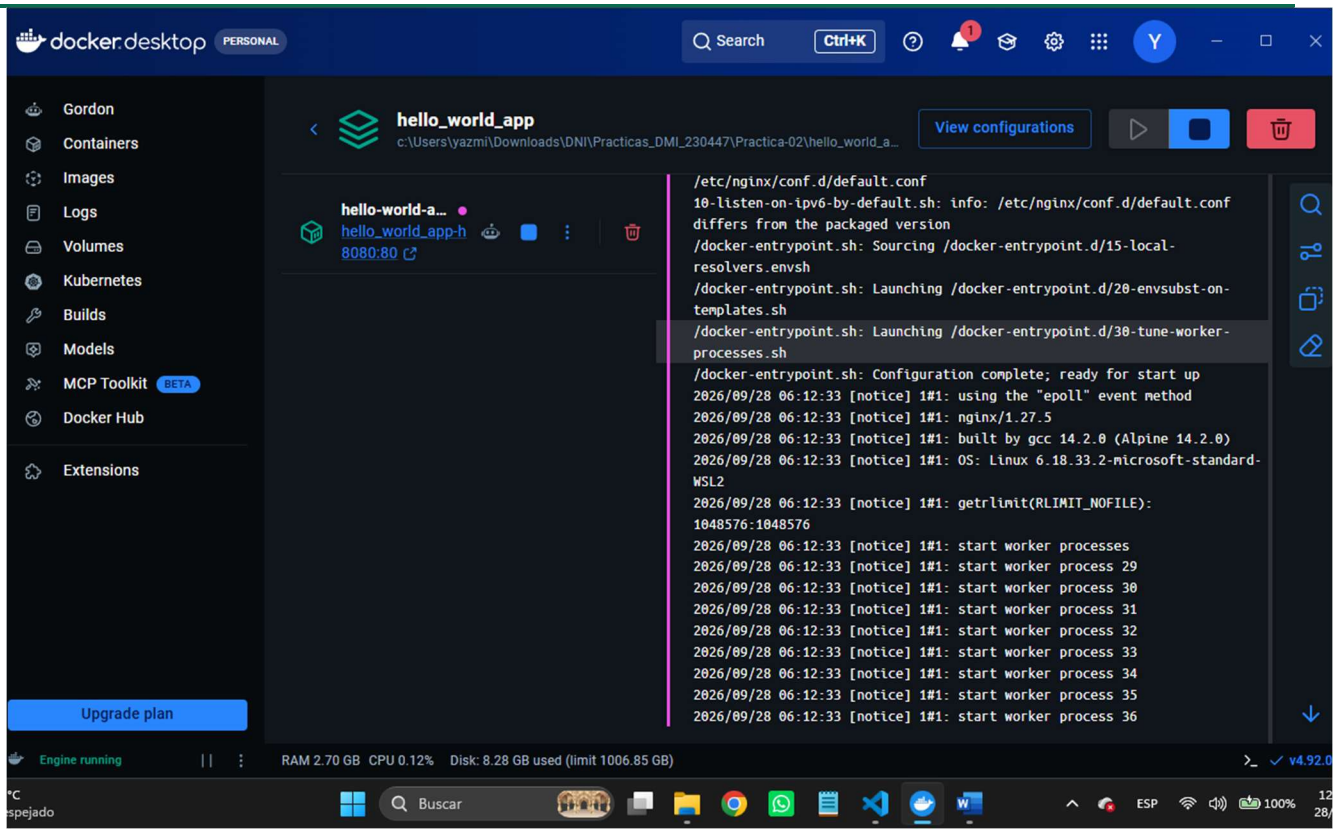


Figura 73. Ejecución del contenedor y revisión de los logs de Docker.

Fuente: Elaboración propia.

Resultado esperado: El contenedor ejecuta correctamente la tarea definida en el Dockerfile.

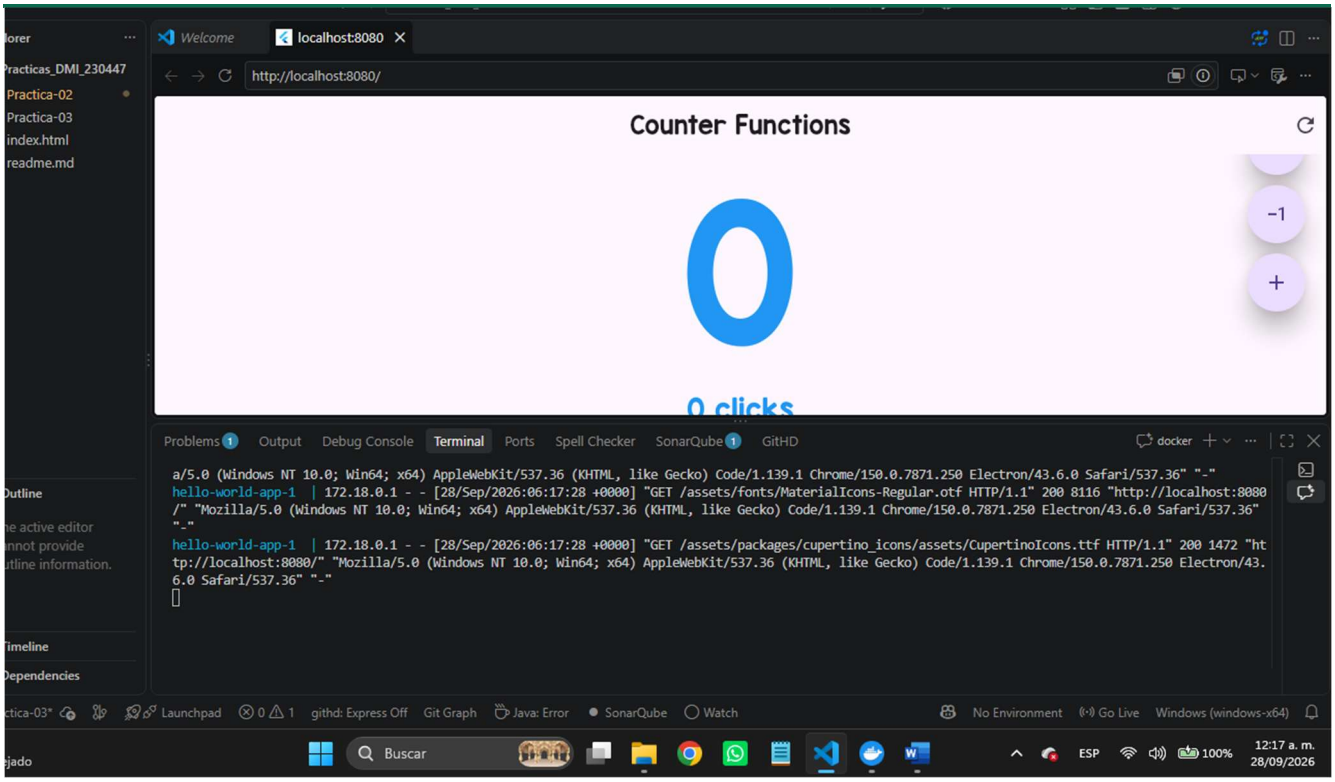


Figura 74. Resultado del contenedor ejecutando la tarea definida en el Dockerfile.

Fuente: Elaboración propia.

Detener los servicios

Para detener y eliminar un contenedor creado directamente:

```
docker stop yes-no-app-container
docker rm yes-no-app-container
```

Si se utiliza Compose:

```
docker compose down
```

Resultado esperado: Los contenedores se detienen sin afectar el código fuente del proyecto.



Figura 75. Detención de los servicios Docker sin afectar el proyecto Flutter.

Fuente: Elaboración propia.

Comprobación Flutter + Docker

La aplicación Flutter continuará ejecutándose de forma independiente:

```
Flutter
→ Android Studio
→ Android Emulator
→ Aplicación
```

Docker seguirá un flujo separado:

```
Proyecto Flutter
→ Dockerfile
→ Imagen Docker
→ Contenedor
→ Pruebas o análisis
```

No se debe indicar que Android Emulator o el APK se ejecutan dentro de Docker si esto no ocurre realmente.

IMAGEN DOCKER

Momento: Después de construir la imagen.

Acción: Ejecutar:

```
docker images
```

La captura debe mostrar: La imagen creada y Docker Desktop o la terminal con la construcción completada.

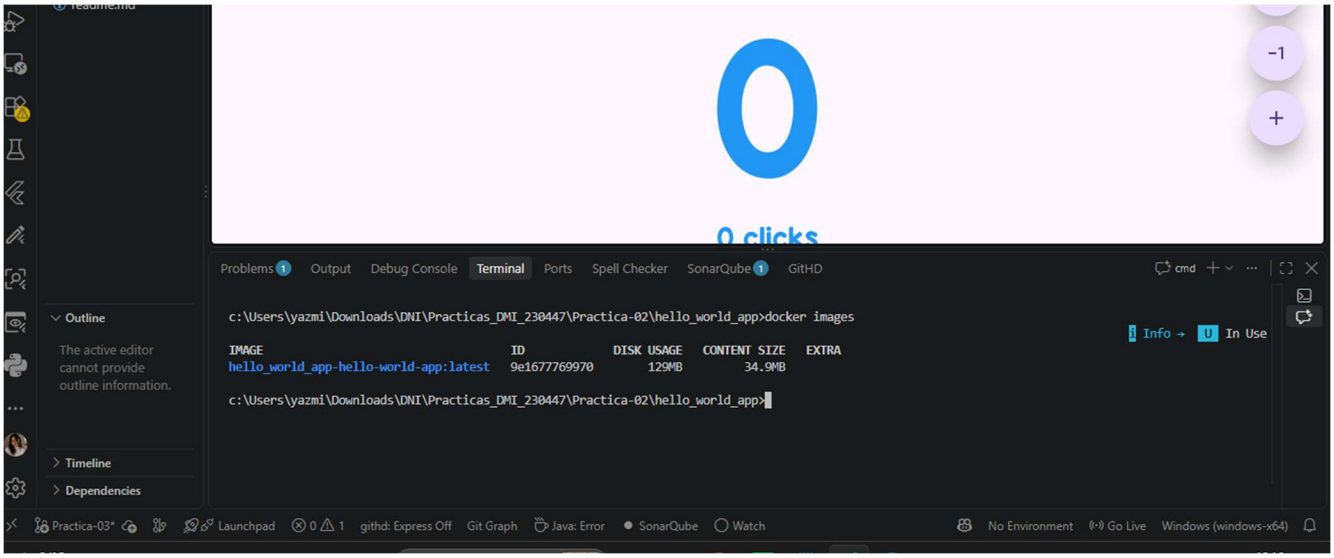


Figura 76. Evidencia de la imagen Docker construida para el proyecto.

Fuente: Elaboración propia.

CONTENEDOR DOCKER

Momento: Después de ejecutar el contenedor.

Acción: Ejecutar:

```
docker ps -a
```

La captura debe mostrar: El contenedor, su estado y el resultado de la tarea configurada.

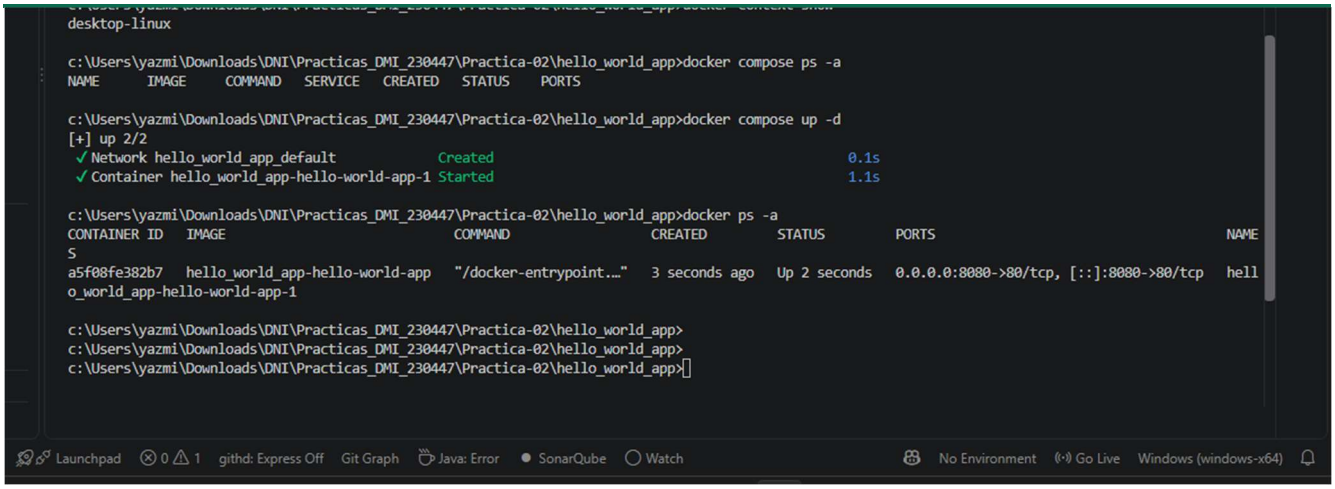


Figura 77. Evidencia del contenedor Docker ejecutado.
Fuente: Elaboración propia.

PUBLICACIÓN DEL PROYECTO EN GITHUB

La publicación debe realizarse después de validar Flutter y completar la configuración Docker que realmente utilice el proyecto. Antes de ejecutar comandos se debe comprobar si el proyecto ya tiene un repositorio Git y si ya existe un remoto origin.

Verificar Git y el repositorio local

git --version
git status

Si git status indica que no existe un repositorio, ejecutar git init. Si el repositorio ya existe, continuar con él y no inicializarlo de nuevo.

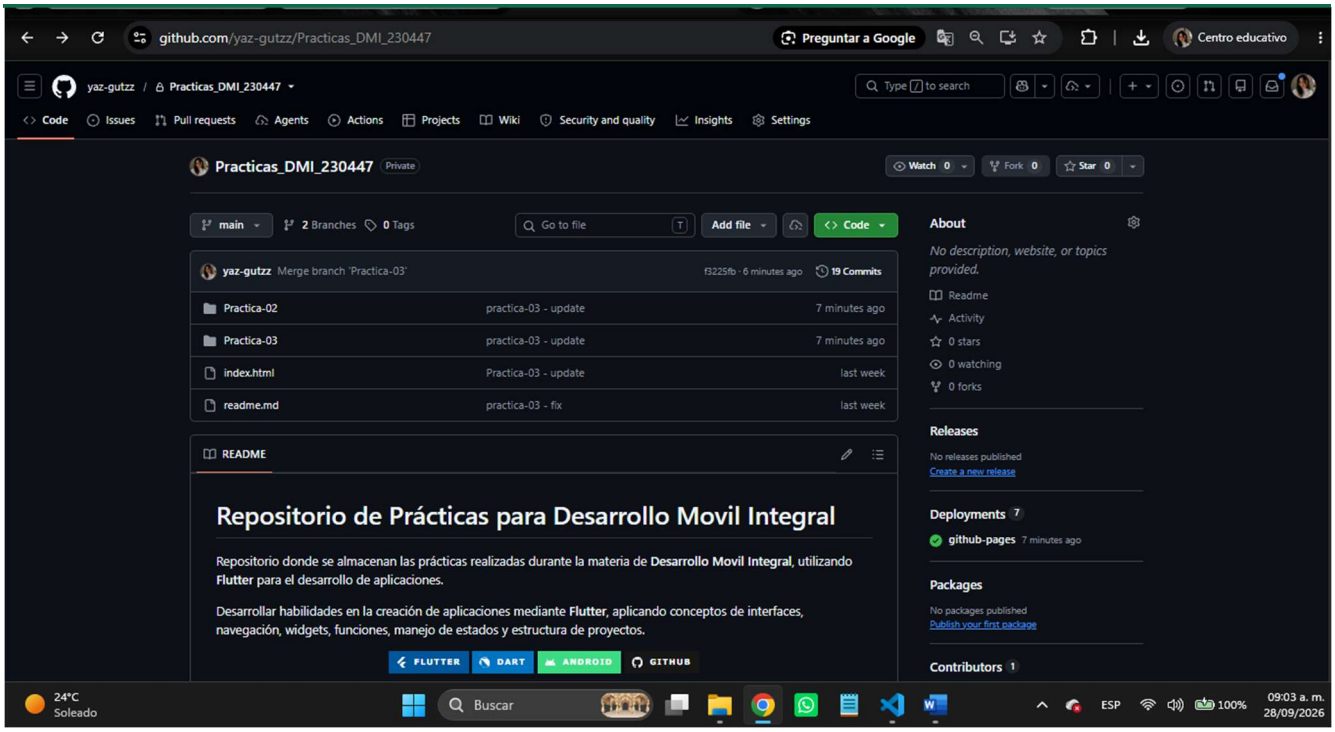


Figura 78. Verificación del repositorio Git local antes de la publicación.

Fuente: Elaboración propia.

Revisar .gitignore

Flutter normalmente genera .gitignore. Revisarlo antes de crear otro. No publicar archivos temporales, compilaciones, credenciales, tokens, claves privadas, secretos ni archivos .env con información sensible. Mantener Dockerfile, .dockerignore y compose.yaml cuando formen parte del proyecto.

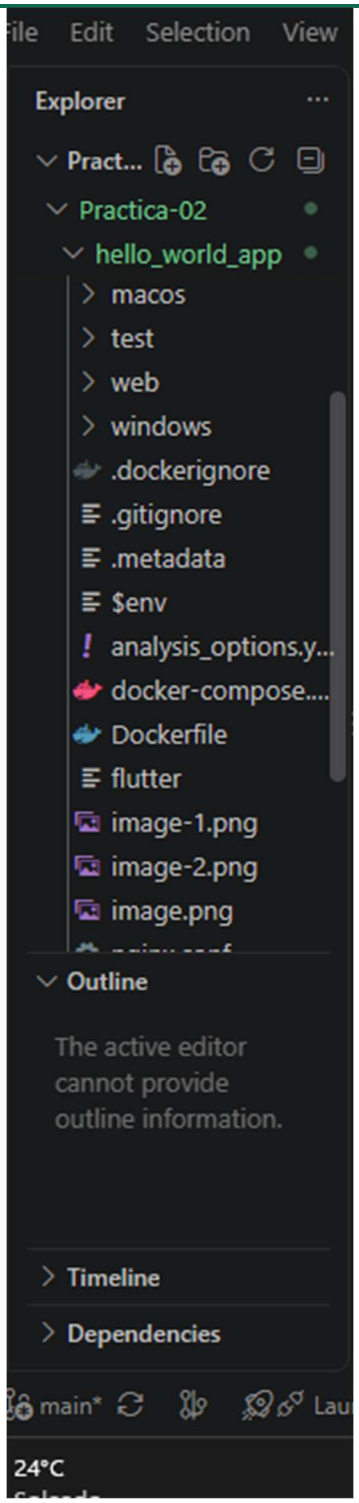


Figura 79. Revisión del archivo .gitignore del proyecto Flutter.

Fuente: Elaboración propia.

Preparar los cambios

```
git status
git add .
git status
```

El primer git status permite identificar archivos Untracked o Modified; después de git add ., el segundo permite comprobar qué archivos quedaron Staged.

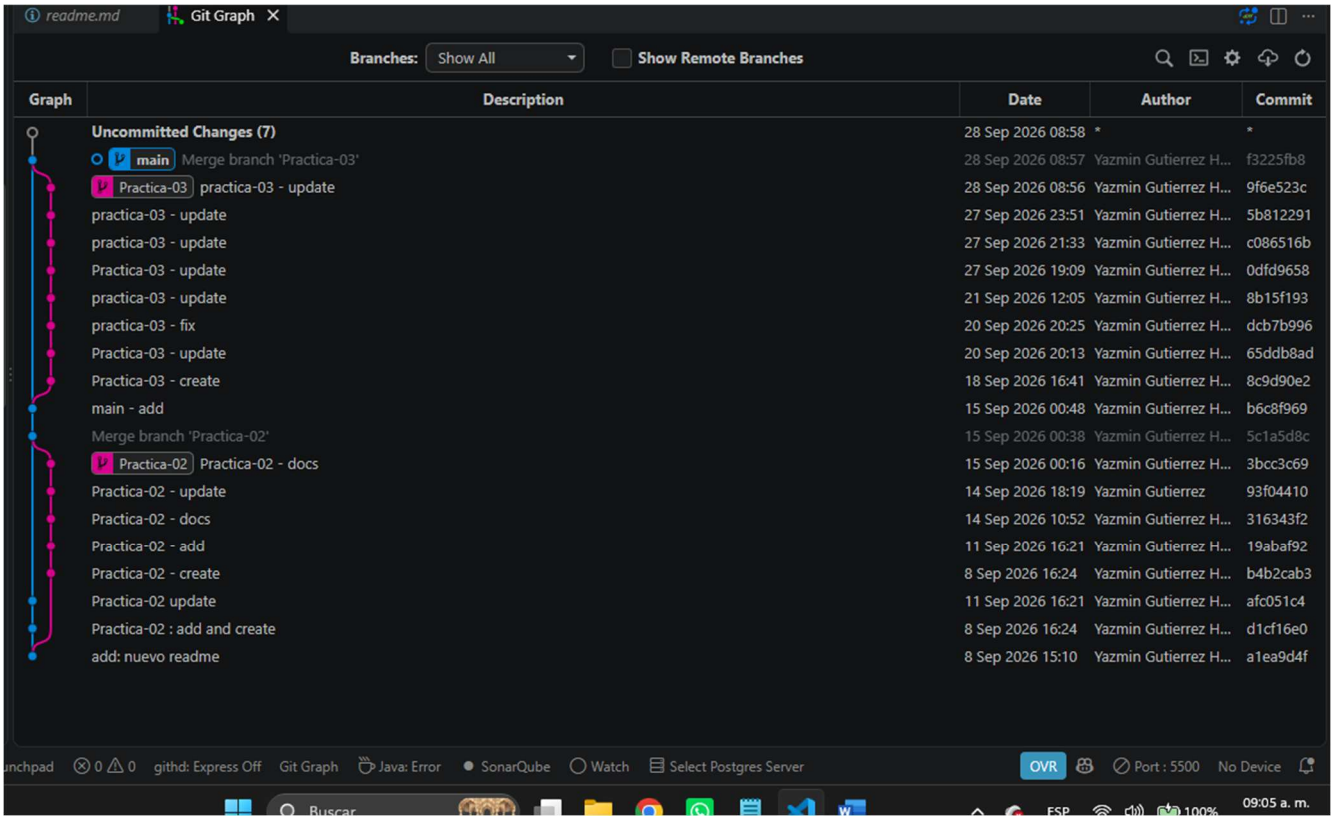


Figura 80. Preparación de cambios con git status y git add.

Fuente: Elaboración propia.

Crear el commit

git branch --show-current

Utilizar el nombre exacto de la rama mostrado por el comando anterior. El formato del commit debe ser:

git commit -m "NOMBRE_EXACTO_DE_LA_RAMA - NOMENCLATURA" -m "DESCRIPCIÓN DE LO REALIZADO"

Ejemplo únicamente si la rama realmente se llama main:

git commit -m "main - add" -m "Se agregó la configuración de Docker al proyecto Flutter y se prepararon los archivos necesarios para ejecutar los contenedores."

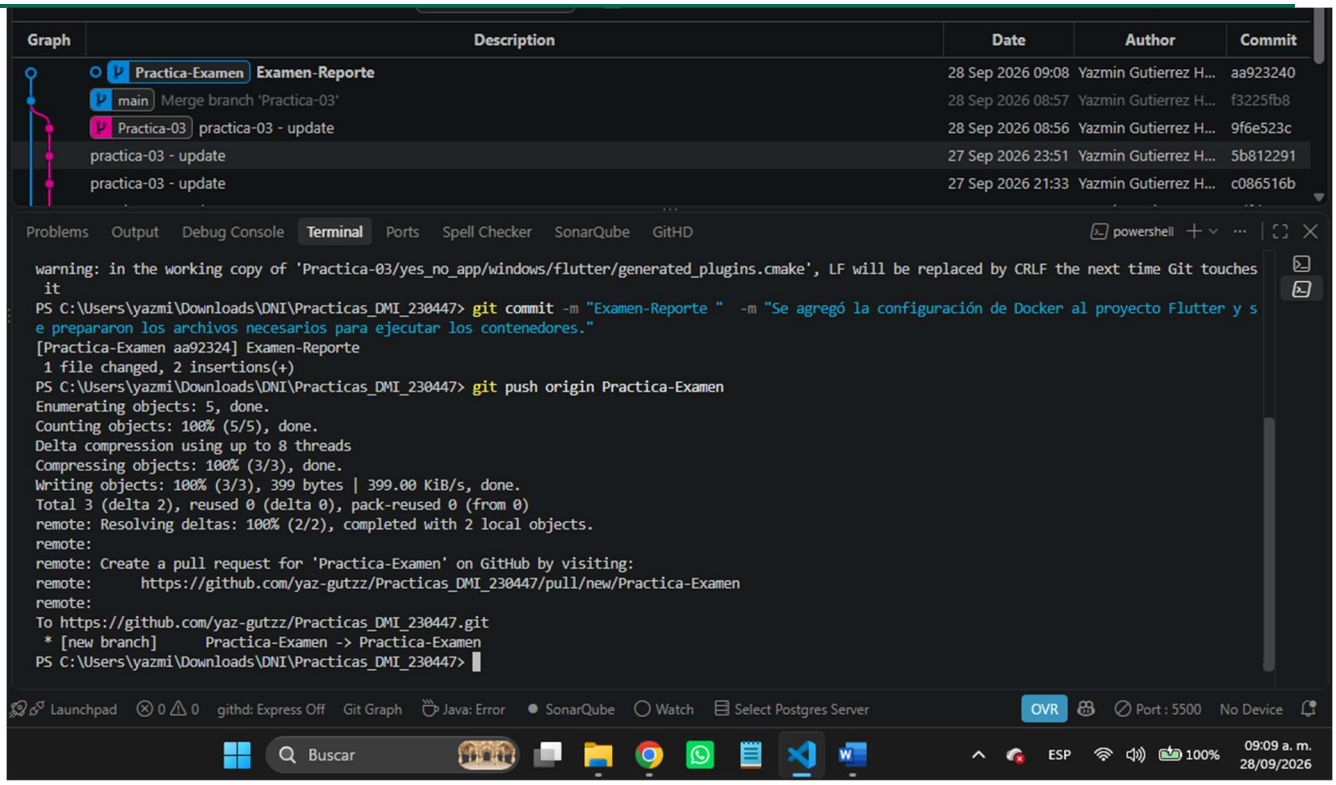


Figura 81. Creación del commit del proyecto mediante Git.

Fuente: Elaboración propia.

Crear o conectar el repositorio de GitHub

Crear un repositorio en GitHub únicamente si todavía no existe.

Evitar agregar automáticamente README o .gitignore desde GitHub cuando el proyecto local ya los contiene y esto pueda generar conflictos.

Copiar la URL del repositorio.

git remote -v

Si no existe origin:

git remote add origin URL_DEL_REPOSITORIO

Si origin ya existe, no volver a agregarlo.

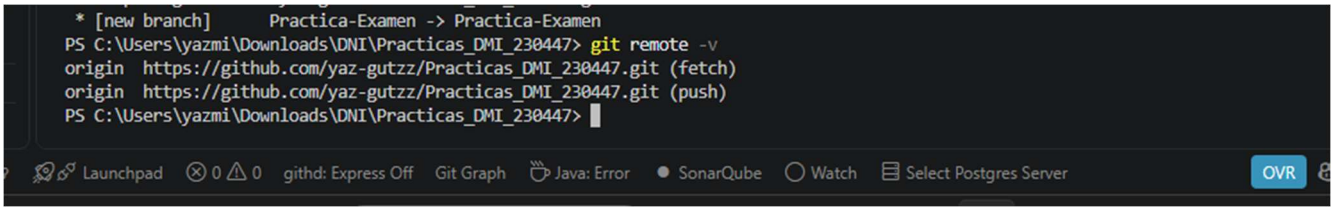


Figura 82. Configuración del repositorio remoto de GitHub.

Fuente: Elaboración propia.

Subir la rama actual

git branch --show-current

Utilizar el nombre real de la rama en el push. Ejemplo solo si la rama es main:

git push -u origin main

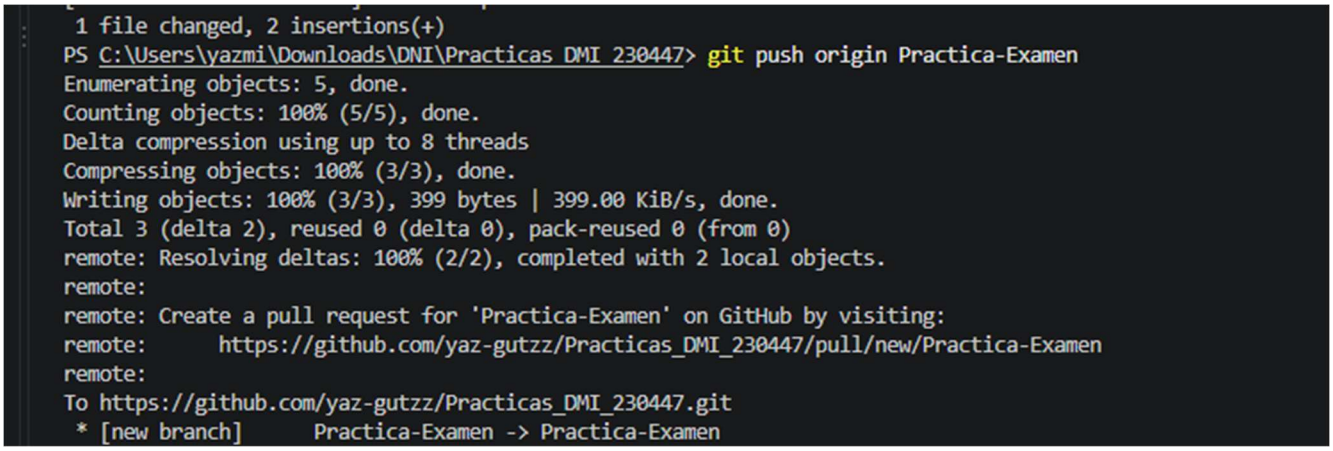


Figura 83. Publicación de la rama actual en GitHub mediante git push.

Fuente: Elaboración propia.

Comprobar el proyecto en GitHub

Actualizar la página del repositorio.

Verificar que estén los archivos necesarios del proyecto y Docker.

Comprobar que no aparezcan credenciales, tokens, claves privadas ni información sensible.

Abrir el historial de commits y confirmar el commit realizado.

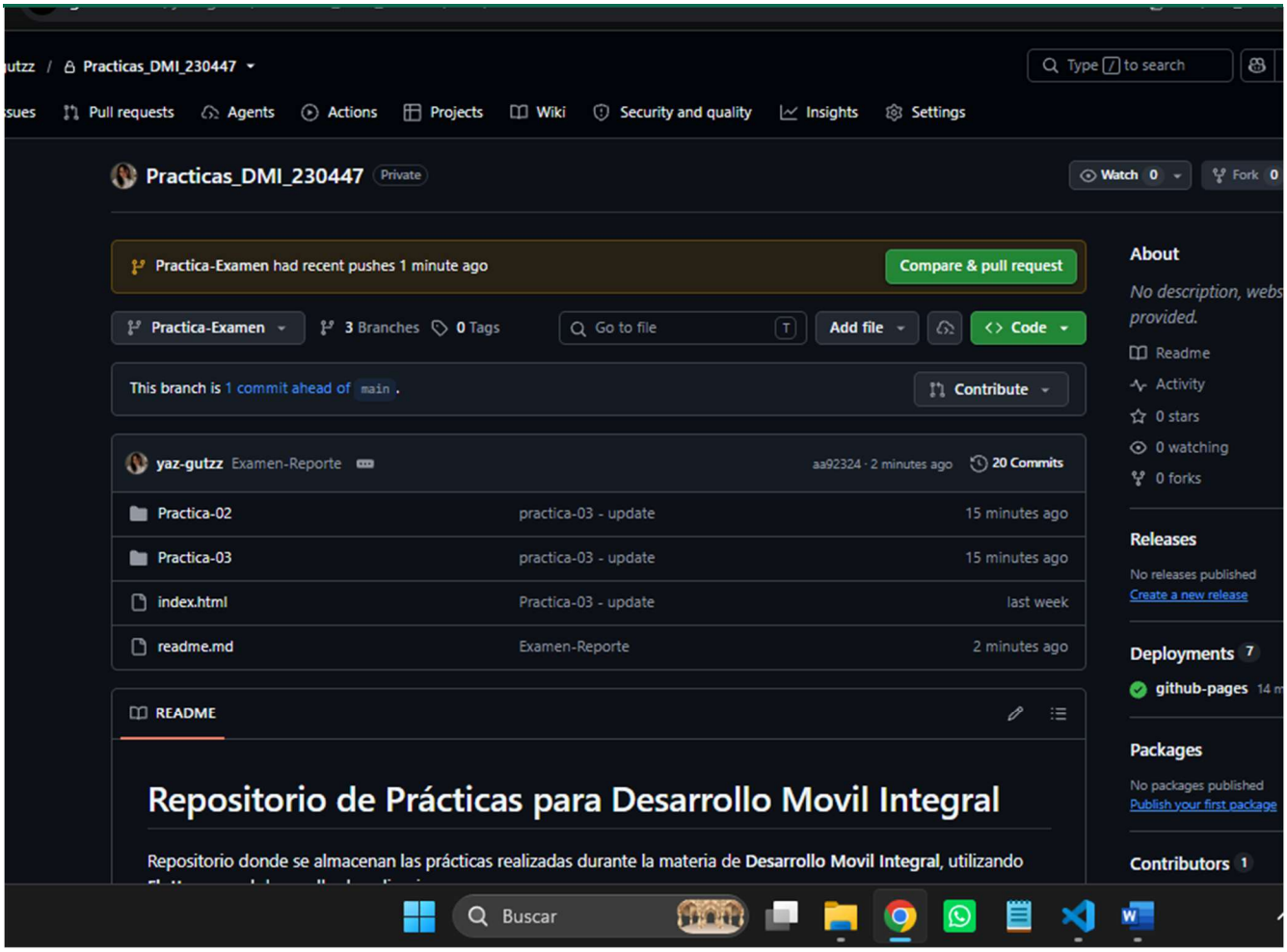


Figura 84. Verificación del proyecto publicado en GitHub.

Fuente: Elaboración propia.

PRUEBA FINAL Y PROBLEMAS FRECUENTES

Secuencia de validación final

Prueba	Comando / herramienta	Resultado esperado
Visual Studio Code	Aplicación / Extensions	Editor abre correctamente y extensiones principales disponibles.
Java	java -version / JAVA_HOME	Java 21 visible y JAVA_HOME configurado.
Maven	mvn -version / MAVEN_HOME	Maven 3.9.x visible y utiliza Java 21.
Node.js y npm	node --version / npm --version	Ambas versiones visibles.
Flutter	flutter --version / flutter doctor	SDK reconocido y diagnóstico revisado.
Android	flutter devices	Emulador o dispositivo detectado.

Aplicación	flutter run	Aplicación real ejecutándose.
Docker	docker --version / docker info	Docker Engine disponible.
Compose	docker compose version	Compose disponible.
Contenedores	docker compose ps	Servicios reales en ejecución.
Git	git status	Repositorio local consistente.
GitHub	Repositorio web	Archivos y commit visibles.

Problemas frecuentes

Problema	Revisión recomendada
Visual Studio Code no abre o no aparece	Revisar instalación y acceso desde el menú Inicio.
java no reconocido	Revisar JDK 21, PATH, JAVA_HOME y abrir una consola nueva.
mvn no reconocido	Revisar MAVEN_HOME, %MAVEN_HOME%\bin en PATH y abrir una consola nueva.
flutter no reconocido	Revisar PATH y abrir una consola nueva.
Flutter no detecta dispositivo	Iniciar el emulador y revisar flutter devices / flutter doctor.
WSL no inicia	Revisar wsl --status y wsl --list --verbose.
node o npm no reconocidos	Abrir una consola nueva y revisar la instalación de Node.js.
Docker no responde	Abrir Docker Desktop y revisar el estado de Docker Engine.
Docker no conecta con WSL2	Revisar la integración WSL2 de Docker Desktop.
Contenedor detenido	Revisar docker compose ps y docker compose logs.
git push falla	Revisar rama actual, remoto origin, autenticación y estado del repositorio.

Reglas permanentes para evidencias

Antes de insertar cualquier captura, comprobar que no aparezcan contraseñas, tokens, claves privadas, credenciales, correos personales innecesarios u otra información sensible. Si aparece un dato sensible, recortarlo o difuminarlo. No generar capturas falsas ni afirmar que un paso fue completado si todavía está pendiente.

CHECKLIST FINAL

Area	Comprobaciones
SISTEMA	☐ Windows compatible ☐ Arquitectura x64 ☐ Virtualización habilitada ☐ WSL2 configurado
HERRAMIENTAS	☐ Visual Studio Code instalado ☐ Extensiones Flutter y Dart disponibles ☐ JDK 21 instalado ☐ JAVA_HOME configurado ☐ Maven 3.9.x instalado ☐ MAVEN_HOME configurado
NODE.JS	☐ Node.js instalado ☐ node --version funciona ☐ npm instalado ☐ npm --version funciona
ANDROID	☐ Android Studio instalado ☐ Android SDK configurado ☐ SDK Tools instaladas ☐ AVD creado ☐ Emulador funcionando
FLUTTER	☐ Flutter instalado ☐ Dart disponible ☐ PATH configurado ☐ flutter doctor revisado ☐ Dispositivo detectado ☐ Aplicación real ejecutándose
DOCKER	☐ Docker Desktop instalado ☐ Docker Engine iniciado ☐ Docker Compose disponible ☐ Imagen creada ☐ Contenedores ejecutándose ☐ docker compose ps verificado ☐ docker compose down probado
GIT / GITHUB	☐ git status revisado ☐ Commit creado ☐ Remoto origin verificado ☐ Push completado ☐ Proyecto visible en GitHub

REFERENCIAS OFICIALES

• Microsoft Learn — Install WSL: https://learn.microsoft.com/windows/wsl/install
• Visual Studio Code — https://code.visualstudio.com/
• Eclipse Adoptium / Temurin — https://adoptium.net/
• Apache Maven — https://maven.apache.org/install
• Microsoft Learn — Basic commands for WSL: https://learn.microsoft.com/windows/wsl/basic-commands
• Node.js — Downloads: https://nodejs.org/en/download/
• npm — Documentation: https://docs.npmjs.com/
• Android Developers — Android Studio: https://developer.android.com/studio
• Android Developers — Emulator: https://developer.android.com/studio/run/emulator
• Flutter — Documentation: https://docs.flutter.dev/
• Dart — Documentation: https://dart.dev/guides
• Docker Desktop — Windows: https://docs.docker.com/desktop/setup/install/windows-install/
• Docker Compose: https://docs.docker.com/compose/
• Git — Documentation: https://git-scm.com/doc
• GitHub Docs: https://docs.github.com/

Nota final

El objetivo del manual es documentar un entorno funcional, reproducible y verificable. Los procedimientos pendientes deben conservar sus espacios de evidencia hasta que se ejecuten realmente. Las capturas existentes se mantienen como evidencia de los puntos ya realizados; las nuevas capturas de Docker y GitHub deberán añadirse cuando corresponda.

EVIDENCIA FINAL DEL ENTORNO

MOMENTO: Al finalizar Flutter, Docker y publicación en GitHub.

ACCIÓN: Documentar en una captura o composición académica los resultados finales que el docente solicite, sin falsificar pasos.

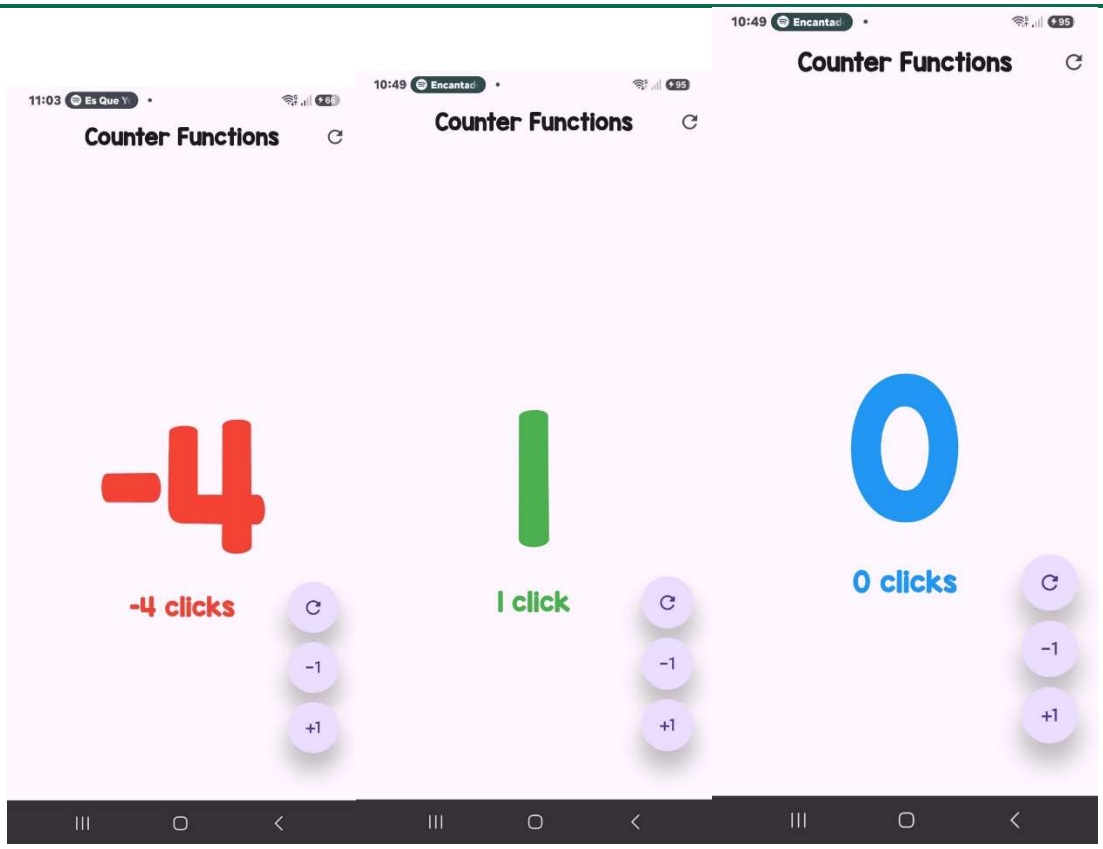


Figura 85. Aplicación Flutter Dockerizada

Fuente: Elaboración propia.